

Solving Numerical Constraints

Alexander Bockmayr

Volker Weispfenning

SECOND READERS: Michael Maher.

Contents

1	Introduction	753
1.1	Constraints in automated reasoning	753
1.2	A classification of numerical constraints	754
1.3	Preliminaries	755
2	Linear constraints over fields	758
2.1	Linear equations	758
2.2	Linear inequalities	761
2.3	Disequalities	774
2.4	Extensions	775
3	Linear diophantine constraints	779
3.1	Equations over $\mathbb{Z}$	779
3.2	Equations over $\mathbb{N}$, inequalities over $\mathbb{Z}$	782
3.3	Cutting planes	785
3.4	Branch-and-infer	788
3.5	Gröbner bases in integer programming	792
3.6	Computing Hilbert bases	793
3.7	Linear 0-1 inequalities	798
3.8	Presburger arithmetic	800
4	Non-linear constraints over continuous domains	802
4.1	Zero-sets of univariate polynomials	802
4.2	Zero-sets of multivariate polynomials	805
4.3	Parametric zero-sets	812
4.4	Complex and real quantifier elimination	816
4.5	Further Topics	818
5	Non-linear diophantine constraints	819
5.1	Hilbert's Tenth Problem	819
5.2	Positive results	821
5.3	Decision problems for first-order theories of the integers	822
	Bibliography	823
	Index	838

1. Introduction

1.1. Constraints in automated reasoning

Numerical constraints have been at the heart of mathematics for about 4000 years. By the second millennium B.C., the Babylonians and Egyptians solved simple examples of linear equations. The Babylonians, Greeks, and Chinese knew the idea of elimination of unknowns for both linear and quadratic equations. Over the course of history, many fields of mathematics have contributed to the study of numerical constraints, e.g., algebra, geometry, number theory, analysis, and optimization theory. During the last 40 years, numerical constraints have also become a major topic of interest in computer science. This paper considers numerical constraint solving in the context of automated reasoning. We are mainly interested in logical aspects of numerical constraints. We view them as atomic formulae in first-order predicate logic that are interpreted over some numerical domain like the real, rational, or integer numbers. We will present various inference systems for reasoning with numerical constraints, describe the algebraic and geometric structure of their solution set, and present fundamental algorithms for deciding satisfiability, computing one or all solutions, and deducing new constraints. We will focus on exact symbolic methods, i.e. we will not consider approximative algorithms or interval techniques that are normally studied in numerical analysis. We will also not discuss numerical problems like rounding errors.

Enhancing general theorem proving by specialized procedures for a particular domain has been a pertinent research theme in automated reasoning, see e.g. [Huet 1972, Plotkin 1972, Stickel 1985, Bürckert 1991] or for more recent work [Bjørner, Stickel and Uribe 1997, Armando and Ranise 1998, Bertoli, Calmet, Giunchiglia and Homann 1998, Dowek, Hardin and Kirchner 1998, Janičić, Bundy and Green 1999]. Cooper [1972] studied theorem proving in Presburger arithmetic. Linear arithmetic over the rational numbers was investigated, e.g., in [Bledsoe 1975, Shostak 1977, Shostak 1979, Boyer and Moore 1988, Käuß 1988, Kapur and Nie 1994]. Methods for the combination of decision procedures for different theories were developed by Nelson and Oppen [1979] and Shostak [1984]. The notion of deduction with constraints can be traced back at least to [Huet 1972]. It became very popular through the work on constraint logic programming [Colmerauer 1987, Jaffar and Lassez 1987]. A framework for constrained deduction was developed in [Kirchner, Kirchner and Rusinowitch 1990], see also [Kirchner 1995]. A methodological view of constraint solving is presented in [Jouannaud and Kirchner 1991, Comon 1991, Comon, Dincbas, Jouannaud and Kirchner 1999]. Armando, Melis and Ranise [1998] compare constraint solving in logic programming and automated deduction. For more information on the integration of constraint solving into different theorem proving methods, we refer to the [Bachmair and Ganzinger 2001, Nieuwenhuis and Rubio 2001, Degtyarev and Voronkov 2001a], i.e., Chapters 2, 7, and 10 in this handbook. The special case of unification is discussed in [Baader and Snyder 2001] (Chapter 8).

The field that we are trying to cover in this chapter is huge. We can only give

Satisfiability	over $\mathbb{R}$	over $\mathbb{Z}$
Linear constraints	polynomial	NP-complete
Non-linear constraints	decidable	undecidable

Table 1: Solving numerical constraints

an outline of what we think are the most important issues. Each section or even subsection of this chapter would deserve a paper or book on its own. We have included an extensive bibliography for further reference. We invite the reader to consult some of the monographs and articles listed there in order to learn more about this fascinating field.

1.2. A classification of numerical constraints

Numerical domains are the field $\mathbb{C}$ of complex numbers, the field $\mathbb{R}$ of real numbers, the field $\mathbb{Q}$ of rational numbers, the ring $\mathbb{Z}$ of integer numbers, the semiring $\mathbb{N}$ of natural numbers (including zero), and the two-element set $\{0, 1\}$. We will use the symbol $\mathbb{D}$ to denote a numerical domain, and the symbol $\mathbb{F}$ to denote one of the fields $\mathbb{C}$, $\mathbb{R}$, or $\mathbb{Q}$. A *numerical function* is a mapping $f : \mathbb{D}_1 \times \cdots \times \mathbb{D}_n \rightarrow \mathbb{D}$ from a Cartesian product of numerical domains $\mathbb{D}_1 \times \cdots \times \mathbb{D}_n$ into a numerical domain $\mathbb{D}$. We are mainly interested in *linear functions*

$$f(x_1, \dots, x_n) = \sum_{i=1}^n c_i x_i, \quad c_i \in \mathbb{D}, \quad (1.1)$$

and *polynomials*

$$p(x_1, \dots, x_n) = \sum_{\substack{(i_1, \dots, i_n) \in \mathbb{N}^n \\ i_1 + \dots + i_n \leq k}} c_{i_1, \dots, i_n} x_1^{i_1} \cdots x_n^{i_n}, \quad c_{i_1, \dots, i_n} \in \mathbb{D}, k \in \mathbb{N}. \quad (1.2)$$

A *numerical constraint* is an equation, inequality, or disequality between two numerical functions. We exclude inequalities between complex-valued functions. By $\text{Sol}_{\mathbb{D}}(C)$ we denote the set of solutions of the constraint system C in the domain $\mathbb{D}$. We are mainly interested in conjunctions of atomic formulae, but we will also discuss arbitrary Boolean combinations and quantifier elimination in the corresponding first-order theories.

We classify constraints according to whether the numerical functions are linear or non-linear, and whether they are solved over a continuous domain, i.e. $\mathbb{R}$, $\mathbb{C}$, or a discrete domain, i.e. $\mathbb{Z}$, $\mathbb{N}$ or $\{0, 1\}$. Very roughly, we get the picture represented in Table 1. The rational numbers $\mathbb{Q}$ play a special role and are not included in this table. In the linear case, they behave like the reals, in the non-linear case more like a discrete domain. In order to get a more accurate picture in the linear case, we have to distinguish between equations and inequalities, see Table 2.

Satisfiability	over $\mathbb{Q}$	over $\mathbb{Z}$	over $\mathbb{N}$
Linear equations	polynomial	polynomial	NP-complete
Linear inequalities	polynomial	NP-complete	NP-complete

Table 2: Solving linear constraints

1.3. Preliminaries

We start with some preliminaries from linear algebra, complexity theory, commutative algebra, and first-order logic. Basic references are [Schrijver 1986, Grötschel, Lovász and Schrijver 1988, Becker, Weispfenning and Kredel 1993, von zur Gathen and Gerhard 1999].

1.3.1. Linear algebra

For $n \in \mathbb{N}$, $n \geq 1$, we denote by $\mathbb{D}^n$ the set of vectors with n components and entries in $\mathbb{D}$. Addition of vectors and multiplication of vectors with scalars are defined as usual. A vector is always considered as a column vector. The superscript $\cdot^T$ denotes transposition, i.e. for $x \in \mathbb{D}^n$, x^T is a row vector.

For $m, n \in \mathbb{N}$, $m, n \geq 1$, we denote by $\mathbb{D}^{m \times n}$ the set of $m \times n$ -matrices with entries in $\mathbb{D}$. For a matrix $A \in \mathbb{D}^{m \times n}$, we usually assume that the row index set of A is $M = \{1, \dots, m\}$ and that the column index set is $N = \{1, \dots, n\}$. For $I \subseteq M$, $J \subseteq N$, we denote by A_{IJ} the submatrix of A induced by the rows in I and the columns in J . Instead of A_{MJ} and A_{IN} , we will also write A_{*J} and A_{I*} . A_{i*} is the i -th row of A , and A_{*j} the j -th column. Unless specified otherwise, the elements of $A \in \mathbb{D}^{m \times n}$ are denoted by a_{ij} , $1 \leq i \leq m$, $1 \leq j \leq n$. Vectors with n components are also considered as $n \times 1$ -matrices. Given $A \in \mathbb{D}^{m \times l}$ and $B \in \mathbb{D}^{l \times n}$, the product $C = A \cdot B \in \mathbb{D}^{m \times n}$ is defined by $c_{ij} = \sum_{k=1}^l a_{ik} \cdot b_{kj}$.

A vector $x \in \mathbb{R}^n$ is called a *linear combination* of the vectors $x_1, \dots, x_k \in \mathbb{R}^n$ if $x = \lambda_1 x_1 + \dots + \lambda_k x_k$, for some $\lambda_1, \dots, \lambda_k \in \mathbb{R}$. If, in addition

$$\left\{ \begin{array}{l} \lambda_1, \dots, \lambda_k \geq 0, \\ \lambda_1 + \dots + \lambda_k = 1, \end{array} \right\} \quad \text{we call } x \text{ a } \left\{ \begin{array}{l} \text{conic} \\ \text{affine} \\ \text{convex} \end{array} \right\} \text{ combination.}$$

These combinations are *proper* if neither $\lambda_1 = \dots = \lambda_k = 0$ nor $x = x_i$, for some $i = 1, \dots, k$. For a nonempty subset $S \subseteq \mathbb{R}^n$, we denote by $\text{lin}(S)$ (resp. $\text{cone}(S)$, $\text{aff}(S)$, $\text{conv}(S)$) the *linear* (resp. *conic*, *affine*, *convex*) *hull* of S , i.e. the set of all linear (resp. conic, affine, convex) combinations of finitely many vectors of S . For the empty set, we define $\text{lin}(\emptyset) = \text{cone}(\emptyset) = \{0\}$ and $\text{aff}(\emptyset) = \text{conv}(\emptyset) = \emptyset$. A subset $S \subseteq \mathbb{R}^n$ is called a *linear subspace* (resp. *cone*, *affine subspace*, *convex set*) if $S = \text{lin}(S)$ (resp. $S = \text{cone}(S)$, $S = \text{aff}(S)$, $S = \text{conv}(S)$).

A subset $S \subseteq \mathbb{R}^n$ is called *linearly* (resp. *affinely*) *independent* if none of its members is a proper linear (resp. affine) combination of elements of S . The *dimen-*

sion $\dim(P)$ of a subset $P \subseteq \mathbb{R}^n$ is the maximum number of affinely independent vectors in P minus one. A set $P \subseteq \mathbb{R}^n$ is *full-dimensional* if $\dim(P) = n$. The *rank* of an $m \times n$ -matrix A is the maximal number of linearly independent columns (or equivalently rows) of A . The matrix A has *full row rank* (resp. *full column rank*) if $\text{rank}(A) = m$ (resp. $\text{rank}(A) = n$).

1.3.2. Complexity

The *encoding length* of rational numbers, vectors, and matrices is defined as follows:

$$\begin{aligned} \langle n \rangle &= 1 + \lceil \log_2(|n| + 1) \rceil, & n &\in \mathbb{Z} \\ \langle r \rangle &= \langle p \rangle + \langle q \rangle, & r &= p/q \in \mathbb{Q}, q > 0, \gcd(p, q) = 1 \\ \langle a \rangle &= \langle a_1 \rangle + \cdots + \langle a_n \rangle, & a &= (a_1, \dots, a_n)^T \in \mathbb{Q}^n \\ \langle A \rangle &= \langle a_{11} \rangle + \cdots + \langle a_{mn} \rangle, & A &= (a_{ij}) \in \mathbb{Q}^{m \times n} \end{aligned}$$

Strongly polynomial algorithms. When analyzing numerical algorithms it is often natural to use the *arithmetic model* of computation, where we count the number of elementary arithmetic operations, i.e. addition, subtraction, multiplication, division, and comparison, instead of the number of steps of a Turing machine. In the arithmetic model, the *encoding length* of a problem instance is simply the number of input numbers, not the encoding length of these numbers.

The running time of an algorithm may be polynomially bounded in the Turing machine model, but not in the arithmetic model, and vice versa. For example, the Euclidean algorithm to compute the greatest common divisor of two integers is polynomial in the Turing machine model, but not in the arithmetic model. Conversely, a polynomial number of elementary arithmetic operations cannot be executed in polynomial time on a Turing machine if the encoding length of the numbers occurring in the computation is not polynomially bounded. We say that an algorithm is *strongly polynomial* if it is a polynomial space algorithm in the Turing machine model and also a polynomial time algorithm in the arithmetic model. This definition depends on how we encode rational numbers and the result of arithmetic operations. We will assume that if a and b are two integers such that we know in advance that b divides a , then the rational number a/b will be represented by the corresponding integer [Grötschel et al. 1988, p.32-33].

1.3.3. Polynomials

The polynomials in an indeterminate x with coefficients in the field $\mathbb{F}$ form a commutative ring, the *univariate polynomial ring* $\mathbb{F}[x]$. The polynomials in several indeterminates $x_1, \dots, x_n$ with coefficients in the field $\mathbb{F}$ form another commutative ring, the *multivariate polynomial ring* $\mathbb{F}[x_1, \dots, x_n]$. For an extension ring S of $\mathbb{F}$, and elements $c_1, \dots, c_n$ in S , the simultaneous substitution of c_i for x_i is compatible with the ring operations. This important fact will be used tacitly whenever needed. As usual, the result of this substitution for a polynomial $f = f(x_1, \dots, x_n)$ will be denoted by $f(c_1, \dots, c_n) \in S$. The n -tuple $(c_1, \dots, c_n)$ is a *zero* of f if $f(c_1, \dots, c_n) = 0$. The set of all common zeros of a set $G \subseteq \mathbb{F}[x_1, \dots, x_n]$ in $\mathbb{F}^n$

is called the $\mathbb{F}$ -variety of G and is denoted by $V_{\mathbb{F}}(G)$. A multivariate polynomial f is *homogeneous* if all monomials occurring in f with non-vanishing coefficient have the same degree. Homogeneous polynomials always have the origin as trivial zero; every other zero is called a *non-trivial zero*. An extension field $\mathbb{K}$ of $\mathbb{F}$ is *algebraically closed*, if every non-constant univariate polynomial $f \in \mathbb{F}[x]$ has a zero in $\mathbb{K}$. In particular, the field $\mathbb{C}$ of complex numbers is algebraically closed. An ordered extension field $\mathbb{K}$ of $\mathbb{F}$ is *real-closed*, if the intermediate value theorem of analysis holds for all polynomial functions on $\mathbb{K}$. In particular, the field $\mathbb{R}$ of real numbers is real-closed.

1.3.4. First-order theories

The basic concepts of first-order logic that we require are as follows. For a given numerical domain such as $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$ we specify the constants, operations, and relations to be considered. A formal list of symbols for these objects together with the arities of the operation symbols and relation symbols forms a *first-order language* L . With every such language we associate a countably infinite set of symbols called *variables*, and some auxiliary symbols such as parentheses and commas. Among all finite strings of L -symbols we distinguish in particular the L -terms and L -formulas: L -terms are formal superpositions of operation symbols starting from variables and constants. In the following we consider only languages L having only binary relations symbols. Then *atomic* L -formulas are strings of the form $s \rho t$, where s, t are L -terms and ρ is a relation symbol in L . L -formulas are obtained from atomic L -formulas by composition with the propositional operators $\wedge$ (and), $\vee$ (or), $\neg$ (not), and quantifiers $\exists x, \forall x$ for variables x with appropriate use of commas and parentheses. An occurrence of a variable x in the scope of a corresponding quantifier $\exists x, \forall x$ is called *bound*. All other occurrences of a variable in an L -formula are *free*. An L -formula is *quantifier-free*, if it contains no quantifiers. An L -formula φ is *prenex*, if it is of the form $Q_1 x_1 \dots Q_n x_n \psi$, where ψ is a quantifier-free formula and Q_i are quantifiers. The part $Q_1 x_1 \dots Q_n x_n$ is called the *prefix* of the formula φ . In this prefix the quantifiers $Q_i x_i$ can be grouped into *blocks* of similar quantifiers (i.e. all existential or all universal). An *existential formula* is a prenex formula with a single block of existential quantifiers. Similarly, a *universal formula* is a prenex formula with a single block of universal quantifiers. A *closed* L -formula is an L -formula in which all variables x are within the scope of a matching quantifier $\exists x$ or $\forall x$. In other words, a closed L -formula contains no *parameters*.

Every closed L -formula has a natural meaning in a numerical domain $\mathbb{D}$ where the constants, operations and relations named in L are defined. So it assumes exactly one of the truth values ‘true’ or ‘false’. For arbitrary L -formulas the meaning depends on an assignment of numerical values in $\mathbb{D}$ for all free occurrences of variables. Two L -formulas are *equivalent* in $\mathbb{D}$ if they assume the same truth value for every variable assignment in $\mathbb{D}$. A basic fact of first-order logic asserts that every L -formula is equivalent over an arbitrary domain to a prenex L -formula. Moreover any quantifier-free L -formula is equivalent to an L -formula in *disjunctive normal form*, i.e. to a disjunction of conjunctions of atomic L -formulas and negations of

atomic L -formulas. We say that an arbitrary formula holds in $\mathbb{D}$ if it assumes the truth value ‘true’ for every variable assignment in $\mathbb{D}$. The *first-order L -theory* of some domain $\mathbb{D}$ is the set of all L -formulas that hold in $\mathbb{D}$.

1.3.5. Quantifier elimination and decision procedures

A *quantifier elimination* procedure for a theory T of some domain $\mathbb{D}$ is an algorithm that assigns to every L -formula φ a quantifier-free L -formula ψ that is equivalent to φ in $\mathbb{D}$. A *decision procedure* for T is an algorithm that decides for a given closed formula whether or not this formula holds in the domain $\mathbb{D}$. From a given quantifier elimination procedure for T for a domain $\mathbb{D}$, where the truth value of closed atomic L -formulas can be evaluated algorithmically, it is easy to produce a decision procedure: on input of a closed formula φ , the quantifier elimination procedure computes an equivalent closed quantifier-free and hence variable-free formula ψ that is a Boolean combination of relations between constant terms. So the truth value of ψ can be determined by evaluating all atomic parts of ψ . While the decision problem is concerned with closed (i.e. parameter-free) formulas, the quantifier elimination problem deals with simplification of formulas involving parameters to a form that is easy to evaluate. For applications, a quantifier elimination procedure is much more useful than a decision procedure. The reason is that a decision procedure gives only a ‘yes’ or ‘no’ answer to a specific problem, while a quantifier elimination procedure describes for which numerical values of the parameters in $\mathbb{D}$ an assertion holds; thus quantifier elimination provides a structural explanation in place of a simple yes/no-answer.

2. Linear constraints over fields

We begin our discussion with classical results from linear algebra and linear optimization. The presentation in Sections 2.1 to 2.2.3 is based on [Schrijver 1986, Grötschel et al. 1988, Bertsimas and Tsitsiklis 1997], where the reader may find many additional details and the proofs of the main theorems. First we study linear equations over a field $\mathbb{F}$.

2.1. Linear equations

2.1.1. Basic theory

Consider a system of linear equations

$$\begin{array}{rcl} a_1^T x & = & \beta_1 \\ \vdots & & \vdots \\ a_m^T x & = & \beta_m \end{array} \iff Ax = b, \quad (2.1)$$

with $a_1, \dots, a_m \in \mathbb{F}^n$, $\beta_1, \dots, \beta_m \in \mathbb{F}$, $A \in \mathbb{F}^{m \times n}$, and $b \in \mathbb{F}^m$. The basic inference principle to derive a new equation $c^T x = \delta$, with $c \in \mathbb{F}^n$, $\delta \in \mathbb{F}$, is taking a *linear*

combination of the given equations. More precisely, we can multiply each equation $a_i^T x = \beta_i$ by a number $u_i \in \mathbb{F}$ and add up the resulting equations $u_i a_i^T x = u_i \beta_i$, for $i = 1, \dots, m$. In matrix notation, this can be expressed by the inference rule

$$\text{lin_com} : \frac{Ax = b}{(u^T A)x = u^T b} \text{ if } u \in \mathbb{F}^m. \quad (2.2)$$

If $Ax = b$ is solvable in $\mathbb{F}^n$, the rule `lin_com` allows us to derive any implied equation $c^T x = \delta$. If $Ax = b$ is not solvable, we can get the contradiction $0 = 1$.

2.1. PROPOSITION (Solvability and entailment). *The system $Ax = b$ is not solvable in $\mathbb{F}^n$ if and only if there exists $u \in \mathbb{F}^m$ such that $u^T A = 0$ and $u^T b = 1$. If $Ax = b$ is solvable, an equation $c^T x = \delta$ is implied by $Ax = b$ in $\mathbb{F}^n$ if and only if there exists $u \in \mathbb{F}^m$ such that $u^T A = c^T$ and $u^T b = \delta$.*

The solution set of a single linear equation $a^T x = \beta$ defines a *hyperplane* in the n -dimensional affine space $\mathbb{F}^n$. The solution set S of a system of linear equations $Ax = b$ corresponds to an intersection of hyperplanes, i.e. an *affine subspace*. For $x^0 \in S$ the set $U = \{x - x^0 \mid x \in S\}$ defines a linear subspace of the n -dimensional vector space $\mathbb{F}^n$. If $\dim U = k$, then U is generated by k linearly independent vectors $x^1, \dots, x^k \in \mathbb{F}^n$.

2.2. PROPOSITION (Structure of the solution set). *If the system $Ax = b$ of linear equations has a solution $x^0 \in \mathbb{F}^n$, then there exist vectors $x^1, \dots, x^k \in \mathbb{F}^n$ such that*

$$\text{Sol}_{\mathbb{F}}(Ax = b) = x^0 + \mathbb{F} \cdot x^1 + \dots + \mathbb{F} \cdot x^k. \quad (2.3)$$

2.1.2. Gaussian elimination

Gaussian elimination is probably the best-known algorithm in linear algebra. To solve a system of linear equations

$$\begin{array}{ccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \dots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \dots & + & a_{2n}x_n & = & b_2 \\ \vdots & & \vdots & & & & \vdots & & \vdots \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \dots & + & a_{mn}x_n & = & b_m, \end{array} \quad (2.4)$$

the basic idea is to eliminate successively the variables in the problem. Suppose $a_{11} \neq 0$. Then we can subtract appropriate multiples of the first equation from the other equations, which corresponds to applications of the rule `lin_com`, in order to get

$$\begin{array}{ccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \dots & + & a_{1n}x_n & = & b_1 \\ & & a'_{22}x_2 & + & \dots & + & a'_{2n}x_n & = & b'_2 \\ & & \vdots & & & & \vdots & & \vdots \\ & & a'_{m2}x_2 & + & \dots & + & a'_{mn}x_n & = & b'_m. \end{array} \quad (2.5)$$

Now we solve recursively the system consisting of the last $m - 1$ equations. Substituting the solution into the first equation yields a value for x_1 .

Forward and backward phase. In matrix form, Gaussian elimination transforms a matrix $A \in \mathbb{F}^{m \times n}$ into a new matrix of the form

$$\begin{pmatrix} \Delta & D \\ 0 & 0 \end{pmatrix}, \quad (2.6)$$

where Δ is a diagonal $r \times r$ -matrix and r is the rank of A . In the *forward phase*, we compute matrices $A_0, A_1, \dots, A_r$, where A_k is of the form

$$A_k = \begin{pmatrix} B & C \\ 0 & D \end{pmatrix}, \quad (2.7)$$

with a non-singular upper triangular matrix $B \in \mathbb{F}^{k \times k}$, for $k = 0, \dots, r$. We start with $A_0 = A$. Given A_k of form (2.7), we choose a non-zero element of D , called the *pivot element*, and permute the rows and columns of A , so that this element becomes d_{11} . Now we add multiples of the first row of D to the other rows of D such that d_{11} becomes the only nonzero element in the first column of D , i.e.

$$d_{ij} := d_{ij} - \frac{d_{i1}}{d_{11}} d_{1j}, \text{ for } i = 1, \dots, m - k, j = 1, \dots, n - k. \quad (2.8)$$

The resulting matrix is called A_{k+1} . This is repeated until D contains no nonzero element. In the *backward phase*, we start with the matrix $E_r = A_r$ and compute matrices $E_{r-1}, \dots, E_0$ of the form

$$E_k = \begin{pmatrix} B & 0 & C \\ 0 & \Delta & D \\ 0 & 0 & 0 \end{pmatrix}, \quad (2.9)$$

such that B is a non-singular upper triangular $k \times k$ -matrix and Δ is a diagonal $(r - k) \times (r - k)$ -matrix. Given the matrix E_k , we add multiples of the k -th row to the other rows of E_k so that b_{kk} will be the only nonzero entry in the k -column of E_k . The resulting matrix is E_{k-1} . The final matrix E_0 has the form (2.6).

Complexity. Obviously, the number of elementary arithmetic operations (addition, subtraction, multiplication, division, comparison) in Gaussian elimination is bounded by a polynomial in m and n . If we assume $n = m$, then we need $O(n^2)$ divisions and $O(n^3)$ additions and multiplications. Note that this is not the best possible. Starting with the classical work by Strassen [1969], fast matrix multiplication algorithms have been developed that require less than $O(n^3)$ arithmetic operations. The best currently known value is less than $O(n^{2.376})$ [Coppersmith and Winograd 1990, Bürgisser, Clausen and Shokrollahi 1997, von zur Gathen and Gerhard 1999].

To show that Gaussian elimination is polynomial, we have to prove in addition that the encoding length of the numbers produced by the algorithm is polynomially

bounded. This can be guaranteed if rational numbers are represented in *coprime* form, i.e. by fractions p/q with $\gcd(p, q) = 1$.

2.3. THEOREM (Edmonds 67). *For any rational matrix $A \in \mathbb{Q}^{m \times n}$, the Gaussian elimination method (using coprime representation of rationals) runs in polynomial time.*

There exists a representation scheme for rational numbers such that the Gaussian elimination method runs even in strongly polynomial time [Edmonds 1967, Bareiss 1968, Grötschel et al. 1988, Geddes, Czapor and Labahn 1992]. This implies:

2.4. COROLLARY. *There exist strongly polynomial algorithms for the following problems:*

- *Finding a solution of a rational equation system $Ax = b$,*
- *Determining the rank of a rational matrix,*
- *Determining the determinant of a rational matrix,*
- *Determining the inverse of a non-singular rational matrix,*
- *Testing whether m given rational vectors are linearly independent.*

Gauss-Jordan elimination is another polynomial time method that combines forward and backward steps. In the k -th step, we have a matrix of the form

$$A_k = \begin{pmatrix} I & C \\ 0 & D \end{pmatrix}, \quad (2.10)$$

where I is the $k \times k$ -identity matrix. Again, we choose a non-zero pivot element in D , and permute rows and columns so that this element becomes d_{11} . Then we divide the $(k+1)$ -th row of A_k by d_{11} , and add rational multiples of this row to all the other rows, such that the 1 in position $(k+1, k+1)$ is the only non-zero element in the $(k+1)$ -th column.

2.2. Linear inequalities

Throughout this subsection, $\mathbb{F}$ denotes the field $\mathbb{R}$ of real numbers, or the field $\mathbb{Q}$ of rational numbers.

2.2.1. Basic theory

Given a system of linear inequalities $Ax \leq b, x \in \mathbb{F}^n$, the basic principle to derive a new inequality $c^T x \leq \delta$ is taking a *non-negative* linear combination of the given inequalities. Another possibility is *weakening* the right-hand side. We capture this by the following two inference rules:

$$\text{nonneg_lin_com} : \frac{Ax \leq b}{(u^T A)x \leq u^T b} \text{ if } \begin{cases} u \in \mathbb{F}^m, \\ u \geq 0 \end{cases} \quad (2.11)$$

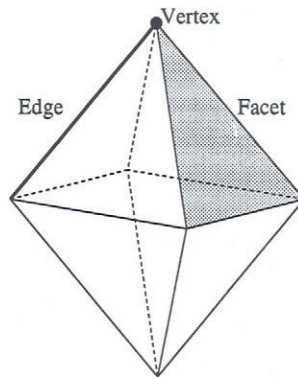


Figure 1: Polyhedron

$$\text{weak_rhs} : \frac{a^T x \leq \beta}{a^T x \leq \beta'} \text{ if } \beta \leq \beta' \quad (2.12)$$

Again there is a soundness and completeness result, which in the theory of linear optimization is known as *Farkas Lemma*. Any implied inequality of a solvable system can be obtained by one application of the rule `nonneg_lin_com` followed by one application of `weak_rhs`.

2.5. PROPOSITION (Solvability and entailment). *The system $Ax \leq b$ has no solution $x \in \mathbb{F}^n$ if and only if there exists $u \in \mathbb{F}^m, u \geq 0$ such that $u^T A = 0$ and $u^T b = -1$. If $Ax \leq b$ is solvable, then an inequality $c^T x \leq \delta$ is implied by $Ax \leq b$ in $\mathbb{F}^n$ if and only if there exists $u \in \mathbb{F}^m, u \geq 0$ such that $u^T A = c^T$ and $u^T b \leq \delta$.*

The solution set of a linear inequality $a^T x \leq \beta, a \neq 0$ defines a *halfspace* in n -dimensional affine space $\mathbb{F}^n$. The solution set $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ of a system of linear inequalities $Ax \leq b$, i.e. an intersection of finitely many halfspaces, is called a *polyhedron*. A bounded polyhedron is called a *polytope*. An inequality $c^T x \leq \delta$ is called *valid* for a polyhedron P if $P \subseteq \{x \in \mathbb{F}^n \mid c^T x \leq \delta\}$. A set $F \subseteq P$ is called a *face* of P if there exists an inequality $c^T x \leq \delta$ valid for P such that $F = P \cap \{x \in P \mid c^T x = \delta\}$. A point v in P such that $\{v\}$ is a face of P is called a *vertex* of P . A polyhedron is called *pointed* if it has a vertex. A non-empty polyhedron $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ is pointed if and only if A has full column rank. A *facet* of P is an inclusionwise maximal face F with $\emptyset \neq F \neq P$. Equivalently, a facet is a nonempty face F of P with $\dim(F) = \dim(P) - 1$, see Fig. 1. If a polyhedron P is full-dimensional then P has a representation $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ such that each inequality in $Ax \leq b$ defines a facet of P and such that each facet of P is defined by exactly one of these inequalities. If we normalize the inequalities, then this representation of P is unique.

In addition to the outer description of a polyhedron as the solution set of a linear inequality system, there is also an inner description, which is based on vertices and extreme rays. A vector $r \in \mathbb{F}^n$ is called a *ray* of the polyhedron P if for each $x \in P$ the set $\{x + \lambda r \mid \lambda \geq 0\}$ is contained in P . A ray r of P is *extreme* if there do not exist two linearly independent rays r^1, r^2 of P such that $r = \frac{1}{2}(r^1 + r^2)$.

2.6. THEOREM. *For each polyhedron $P \subseteq \mathbb{F}^n$ there exist finitely many points $p^1, \dots, p^k$ in P and finitely many rays $r^1, \dots, r^l$ of P such that*

$$P = \text{conv}(p^1, \dots, p^k) + \text{cone}(r^1, \dots, r^l). \quad (2.13)$$

If the polyhedron P is pointed then $p^1, \dots, p^k$ may be chosen as the uniquely determined vertices of P , and $r^1, \dots, r^l$ as representatives of the up to scalar multiplication uniquely determined extreme rays of P .

This result yields a parametric description of the solution set of a system of linear inequalities. In particular, every polytope can be described as the convex hull of its vertices.

Algebraically, the vertices of a polyhedron can be characterized as follows: Let $A \in \mathbb{F}^{m \times n}$ be a matrix of rank r . A non-singular $r \times r$ -submatrix B of A is called a *basis* of A . Let $A \in \mathbb{F}^{m \times n}$ have full column rank n and let $b \in \mathbb{F}^m$. For any basis $B = A_{I*}$ of A , let b_I be the subvector of b corresponding to B . Then the vector $B^{-1}b_I$ is called a *basic solution* of $Ax \leq b$. Note that $B^{-1}b_I$ need not satisfy $Ax \leq b$. If $B^{-1}b_I$ satisfies $Ax \leq b$, then it is called a *basic feasible solution* of $Ax \leq b$. A vector v is a vertex of the polyhedron $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$, A with full column rank, if and only if it is a basic feasible solution of $Ax \leq b$, for some basis B of A . Note that this basis need not be unique.

In many practical applications, one is interested in computing a solution of a system of linear inequalities that is optimal with respect to some linear objective function $f(x) = c^T x$, $c \in \mathbb{F}^n$. This leads to a *linear optimization* or *linear programming* problem. Given a matrix $A \in \mathbb{F}^{m \times n}$ and vectors $b \in \mathbb{F}^m$, $c \in \mathbb{F}^n$, find a vector $x^* \in P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ maximizing the linear function $c^T x$ over P , i.e.

$$c^T x^* = \max\{c^T x \mid Ax \leq b, x \in \mathbb{F}^n\}. \quad (2.14)$$

A vector $x \in \mathbb{F}^n$ satisfying $Ax \leq b$ is called a *feasible* solution. A feasible solution x^* is called an *optimal solution* if $c^T x \leq c^T x^*$, for all feasible solutions $x \in P$. With every linear program of the form (2.14) we can associate another linear program

$$\min\{y^T b \mid y^T A = c^T, y \geq 0\}, \quad (2.15)$$

which is called the *dual* of the *primal* problem (2.14). Using trivial transformations, the dual program can be brought into the form (2.14).

2.7. THEOREM (Duality). *Let (P) $\max\{c^T x \mid Ax \leq b\}$ be a linear program and (D) $\min\{y^T b \mid y^T A = c^T, y \geq 0\}$ be its dual. If (P) and (D) both have feasible solutions,*

then both programs have optimal solutions and the optimum values of the objective functions are equal. If one of the programs (P) or (D) has no feasible solution, then the other is either unbounded or has no feasible solution. If one of the programs (P) or (D) is unbounded, then the other has no feasible solution.

There exist various different formulations of linear optimization problems, which are all equivalent. One possibility is the *standard form*

$$c^T x^* = \max\{c^T x \mid Ax = b, x \geq 0\}. \quad (2.16)$$

To obtain this form from (2.14), we can replace each inequality $a^T x \leq \beta$ by the equation $a^T x + x' = \beta$ using a new variable $x' \geq 0$, which is called a *slack variable*. Furthermore, any unbounded variable x in (2.14) can be written as the difference $x = x^+ - x^-$ of two new non-negative variables $x^+, x^- \geq 0$.

2.2.2. The Simplex method

The classical method for solving linear programs is the Simplex algorithm, which was proposed by Dantzig in 1947. While this method is usually described for linear programs in standard form, we follow here the presentation in [Grötschel et al. 1988] and consider a linear program of the form

$$\max\{c^T x \mid Ax \leq b, x \in \mathbb{F}^n\}. \quad (2.17)$$

Assume first that the matrix $A \in \mathbb{F}^{m \times n}$ has full column rank. This implies that the polyhedron $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ has a vertex. In geometric terms, the Simplex method can be described as follows. We start at a vertex of P and check whether there is an edge leaving this vertex along which the objective function can be increased. If there is no such edge, the current vertex is optimal. Otherwise, we follow one of these edges. Either we find a new vertex, in which case we can repeat the procedure, or the edge is infinite, in which case the linear program (2.17) is unbounded.

We now translate these geometric ideas into an algebraic framework. A *basic feasible index set* is a row index set $I \subseteq \{1, \dots, m\}$ of cardinality n for which A_{I*} is a basis of A and for which $A_{I*}^{-1}b_I$ is feasible. As we have mentioned before, each vertex v of P can be represented by a basic feasible index set I such that $v = A_{I*}^{-1}b_I$. Instead of creating a sequence of vertices, the Simplex method constructs a sequence $I_1, I_2, \dots$ of basic feasible index sets. Remember, however, that this representation of a vertex need not be unique. Therefore, in the sequence $A_{I_1*}^{-1}b_{I_1}, A_{I_2*}^{-1}b_{I_2}, \dots$ two consecutive vertices need not be different.

We now describe one iteration of the Simplex method. Suppose we have a basic feasible index set I with corresponding vertex $v = A_{I*}^{-1}b_I$. We calculate

$$u^T := c^T A_{I*}^{-1}. \quad (2.18)$$

Note that u is an n -dimensional vector indexed by I . We distinguish two cases:

1. If $u \geq 0$, then v is an optimal solution of (2.17), because for each feasible solution x

$$c^T x = u^T A_{I*} x \leq u^T b_I = u^T A_{I*} v = c^T v. \quad (2.19)$$

2. If $u \not\geq 0$, we choose $i \in I$ such that $u_i < 0$ and define the direction

$$d := -A_{I*}^{-1} e_i, \quad (2.20)$$

where e_i is the i -th unit basis vector in $\mathbb{R}^I$.

The idea is to increase the value of the objective function by going from v in direction d while maintaining feasibility.

Note that $c^T(v + \lambda d) = c^T v - \lambda u^T e_i = c^T v - \lambda u_i > c^T v$, for $\lambda > 0$.

- (a) If $Ad \not\leq 0$, then the largest $\lambda \geq 0$ for which $v + \lambda d$ is still feasible is given by

$$\lambda^* = \min \left\{ \frac{b_l - A_{l*} v}{A_{l*} d} \mid l \in \{1, \dots, m\}, A_{l*} d > 0 \right\}. \quad (2.21)$$

Let this minimum be attained at index k . Then $k \notin I$ because $A_{I*} d = -e_i \leq 0$. We define the new basic feasible index $I' = (I \setminus \{i\}) \cup \{k\}$, which corresponds to the vertex $v + \lambda^* d$. Then we replace I by I' and repeat the iteration.

- (b) If $Ad \leq 0$, then $v + \lambda d$ is feasible, for all $\lambda \geq 0$. Moreover,

$$c^T d = -c^T A_{I*}^{-1} e_i = -u^T e_i = -u_i > 0. \quad (2.22)$$

Thus the objective function can be increased along d to infinity and (2.17) is unbounded.

The method terminates if the indices i and k are chosen in the right way (such choices are called *pivoting rules*). For example, following the rule of Bland [1977], we can always choose the smallest i such that $u_i < 0$ and the smallest k attaining the minimum in (2.21). For most known pivoting rules, sequences of examples have been constructed such that the number of iterations is exponential in $m + n$ [Klee and Minty 1972]. Although no pivoting rule is known to yield a polynomial time algorithm, the Simplex method turns out to work very well in practice. Borgwardt [1982] proved that in some natural probabilistic model, and with an appropriate pivoting rule, the expected running time of the Simplex method is polynomial.

In the general case, where we know nothing about the matrix A , we may first apply Gaussian elimination in order to satisfy the hypothesis that A has full column rank. In order to find an initial basic feasible index set, we can consider an auxiliary linear program

$$\max \{y \mid \begin{array}{rcl} Ax - by & \leq & 0 \\ -y & \leq & 0 \\ y & \leq & 1 \end{array} \}, \quad (2.23)$$

where y is a new variable. Given an arbitrary basis A_{K*} of A , we obtain a basic feasible index set I for (2.23) by choosing $I = K \cup \{m+1\}$. The corresponding basic feasible solution is 0. Now, we apply the Simplex method to (2.23). If the optimum value is 0, then (2.17) is infeasible. Otherwise, the optimum value has to be 1. If I' is the final basic feasible index set of (2.23), then $K' = I' \setminus \{m+2\}$ can be used as an initial basic feasible index set for (2.17).

2.2.3. The ellipsoid method and the complexity of linear programming

For a long time, it was an open question whether there is a polynomial algorithm for solving linear inequalities over the rational numbers. Only in 1979, Khachiyan showed that the *ellipsoid method* for nonlinear programming can be adapted to solve linear inequalities and linear programming in polynomial time.

2.8. THEOREM (Khachiyan 79). *The following problems are solvable in polynomial time:*

- Given a matrix $A \in \mathbb{Q}^{m \times n}$ and a vector $b \in \mathbb{Q}^m$, decide whether $Ax \leq b$ has a solution $x \in \mathbb{Q}^n$, and if so, find one.
- Given a rational matrix $A \in \mathbb{Q}^{m \times n}$ and a rational vector $b \in \mathbb{Q}^m$, decide whether $Ax = b$ has a nonnegative solution $x \in \mathbb{Q}^n, x \geq 0$, and if so, find one.
- (Linear programming problem) Given a matrix $A \in \mathbb{Q}^{m \times n}$ and vectors $b \in \mathbb{Q}^m, c \in \mathbb{Q}^n$, decide whether $\max\{c^T x \mid Ax \leq b, x \in \mathbb{Q}^n\}$ is infeasible, finite, or unbounded. If it is finite, find an optimal solution. If it is unbounded, find a feasible solution x_0 , and find a vector $z \in \mathbb{Q}^n$ with $Az \leq 0$ and $c^T z > 0$.

A set $E \subseteq \mathbb{R}^n$ is called an *ellipsoid* if there exists a vector $c \in \mathbb{R}^n$, which is called the *center* of E , and a positive definite matrix $D \in \mathbb{R}^{n \times n}$ such that

$$E = E(c, D) = \{x \in \mathbb{R}^n \mid (x - c)^T D^{-1} (x - c) \leq 1\}. \quad (2.24)$$

Remember that a symmetric matrix $D \in \mathbb{R}^{n \times n}$ is *positive definite*, if $x^T D x > 0$, for all $x \in \mathbb{R}^n, x \neq 0$. Any positive definite matrix is non-singular, and D^{-1} is again positive definite. The unit ball $B(0, 1) = \{x \in \mathbb{R}^n \mid x_1^2 + \dots + x_n^2 \leq 1\}$ around 0 in the Euclidean norm is identical with the ellipsoid $E(0, I)$. For every positive definite matrix D there exists a unique positive definite matrix $D^{1/2}$ such that $D = D^{1/2} D^{1/2}$. It follows that $E(c, D) = D^{1/2} B(0, 1) + c$. Thus every ellipsoid is the image of the unit ball under a bijective affine transformation. The *volume* $\text{vol}(E)$ of an ellipsoid $E = E(c, D)$ is given by $\text{vol}(E(c, D)) = \sqrt{\det D} \cdot V_n$, where V_n denotes the volume of the unit ball $B(0, 1)$ in $\mathbb{R}^n$.

We now give an outline of the ellipsoid method for deciding whether a system of linear inequalities $Ax \leq b$ with integer coefficients is solvable over the real or rational numbers. Consider the polyhedron

$$P = \{x \in \mathbb{F}^n \mid Ax \leq b\}, \quad A \in \mathbb{Z}^{m \times n}, b \in \mathbb{Z}^m, \quad (2.25)$$

and let us assume that P is either empty or bounded and full-dimensional. The basic idea is to construct a sequence of ellipsoids $E_t, t \in \mathbb{N}$, such that all E_t contain P and such that $\text{vol}(E_t)$ gets smaller and smaller. Suppose that, in the t -th iteration, we have computed the ellipsoid $E_t = E(c_t, D_t)$. If $c_t \in P$, then P is non-empty and the algorithm terminates. If $c_t \notin P$, there exists an inequality $a^T x \leq \beta$ in the system $Ax \leq b$ such that $a^T c_t > \beta$. It follows that P is contained in the intersection $E_t \cap H$ of E_t with the half-space $H = \{x \in \mathbb{R}^n \mid a^T x \leq a^T c_t\}$. Now we can construct a new ellipsoid E_{t+1} containing the half-ellipsoid $H \cap E_t$ such that the volume of E_{t+1} is only a fraction of the volume of E_t , see Fig. 2 for illustration.

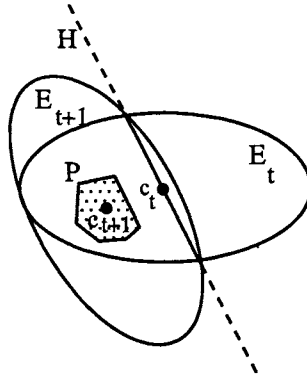


Figure 2: One iteration of the ellipsoid method

2.9. THEOREM. Let $E_t = E(c_t, D_t) \subseteq \mathbb{R}^n$ be an ellipsoid and let $a \in \mathbb{R}^n$ be a non-zero vector. Consider the halfspace $H = \{x \in \mathbb{R}^n \mid a^T x \leq a^T c_t\}$ defined by the hyperplane in direction a through c_t . Let

$$c_{t+1} = c_t - \frac{1}{n+1} d_t \quad \text{and} \quad D_{t+1} = \frac{n^2}{n^2-1} \left(D_t - \frac{2}{n+1} d_t d_t^T \right), \quad (2.26)$$

where

$$d_t = \frac{1}{\sqrt{a^T D_t a}} D_t a. \quad (2.27)$$

Then $E_{t+1} = E(c_{t+1}, D_{t+1})$ is an ellipsoid such that

- $E_t \cap H \subset E_{t+1}$
- $\text{vol}(E_{t+1}) < e^{-\frac{1}{2n}} \text{vol}(E_t)$

Repeating this process, we either find a point in P or we conclude that the volume of P is smaller than some lower bound on the volume of P , if P is nonempty. This implies that P is empty. One can show that if P is full-dimensional, then $\text{vol}(P) \geq 2^{-(n+1)\langle A \rangle + n^3}$, where $\langle A \rangle$ is the encoding length of A . An initial ellipsoid $E_0 = E(0, R^2 I)$ that contains a point of P , if P is non-empty, can be obtained by observing that all vertices of P , if any, are contained in the ball around 0 with radius $R = \sqrt{n} 2^{\langle A, b \rangle - n^2}$. Remember that a non-empty P has a vertex if and only if A has full column rank. Combining the previous results one can see that after

$$N = 2n((2n+1)\langle A \rangle + n\langle b \rangle - n^3) \quad (2.28)$$

iterations of the ellipsoid method, either we have found a point in P or we can conclude that P is empty because $\text{vol}(E_N) < \text{vol}(P)$. In order to prove that the ellipsoid method with rational arithmetic runs in polynomial time, we must also show that the square root in (2.27) can be approximated in polynomial time without

losing correctness of the algorithm, and that the intermediate rational numbers do not get too large. These results are rather technical and do not offer much insight, see [Schrijver 1986, Grötschel et al. 1988] for more details. The assumption that P is full-dimensional, if it is not empty, can be removed by replacing P with the polyhedron $P_\epsilon = \{x \in \mathbb{R}^n \mid Ax \leq b + \epsilon \cdot e\}$, where $\epsilon = \frac{1}{2n} 2^{-4n^2 \langle A, b \rangle}$ and $e = (1, \dots, 1)^T \in \mathbb{R}^m$. Then P_ϵ is empty if P is empty, and P_ϵ is full-dimensional, if P is non-empty.

The ellipsoid method is an important theoretical tool for analyzing the complexity of solving linear arithmetic constraints. Unfortunately, it does not result in an algorithm that is useful in practice. The convergence rate is slow and the precision demands are very high. Therefore, from a practical perspective, the theoretically inefficient Simplex algorithm behaves much better.

The ellipsoid method is not strongly polynomial. The number of elementary arithmetic operations depends on the encoding length of the input data. Tardos [1986] showed that it is possible to solve any linear program $\max\{c^T x \mid Ax \leq b\}$ in $p(\langle A \rangle)$ elementary arithmetic operations on numbers with encoding length polynomially bounded by $\langle A, b, c \rangle$, where p is a polynomial. In other words, the number of elementary arithmetic operations does not depend on $\langle b \rangle$ and $\langle c \rangle$. This implies that combinatorial linear programs, where A is a $\{0, \pm 1\}$ matrix, have a strongly polynomial algorithm. In [Frank and Tardos 1987], this result is extended to problems, where the number of inequalities may be exponential in the dimension. Megiddo [1984] proved that for each fixed n there is an algorithm for solving linear programs in n variables and m constraints that requires only $O(m)$ elementary arithmetic operations on numbers with polynomial encoding length. Combinatorial algorithms for linear programming in two variables per inequality have been developed among others in [Cohen and Megiddo 1994], [Hochbaum and Naor 1994], and [Wayne 1999].

2.2.4. Interior point methods

The Simplex method solves a linear optimization problem by visiting extreme points, all lying on the boundary of the corresponding polyhedron. *Interior point algorithms* find an optimal solution while moving through the relative interior of the feasible region. They were first described by Dikin [1967], current interest started with a new polynomial-time algorithm for linear programming by Karmarkar [1984]. Interior point methods combine the advantages of the Simplex algorithm and the ellipsoid method. From a theoretical point of view, many interior point algorithms have polynomial complexity like the ellipsoid method. In practice, however, they behave much better. They are competitive with the Simplex method and sometimes even superior, in particular when solving linear optimization problems with a large number of variables and constraints. There exists a variety of interior point methods based on different geometric ideas. We can sketch here only some of the key concepts and refer to the literature for a more detailed description [Bertsimas and Tsitsiklis 1997, Vanderbei 1997, Ye 1997].

- *The affine scaling algorithm* is probably the simplest interior point algorithm. However, it has already good practical performance. The basic idea is to con-

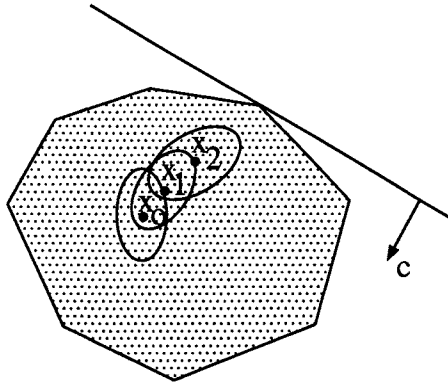


Figure 3: The affine scaling algorithm

struct a sequence of ellipsoids lying inside the feasible region such that the centers of these ellipsoids converge to an optimal solution (Fig. 3).

- The *potential reduction algorithm* measures the progress towards optimality by the reduction in a certain nonlinear potential function, which attempts to balance two conflicting objectives: improving the objective function value and staying away from the boundary of the feasible set.
- *Path following algorithms* first transform the constrained linear program to an unconstrained one, by incorporating the constraints in a *logarithmic barrier function* that imposes a growing penalty as one gets closer to the boundary. Then the unconstrained problem is solved approximately by applying Newton's method. As the strength of the barrier function is decreased, the optimum of the unconstrained problem follows a path that ends at an optimal solution of the original problem.

We describe briefly the *primal-dual path following algorithm*, which may be the most popular interior-point method. It has excellent performance in large scale applications and is used in various commercial implementations. We start with a linear programming problem in standard form

$$\min\{c^T x \mid Ax = b, x \geq 0\} \quad (2.29)$$

and its dual

$$\max\{y^T b \mid y^T A + z^T = c^T, z \geq 0\}. \quad (2.30)$$

For any positive constant $\mu > 0$, we consider the *barrier optimization problem*

$$\min\{c^T x - \mu \sum_{j=1}^n \log(x_j) \mid Ax = b\}. \quad (2.31)$$

Here, $\log$ denotes the natural logarithm. Note that if some x_j approaches 0, then the objective function value goes to $+\infty$. The objective function is defined to be $+\infty$,

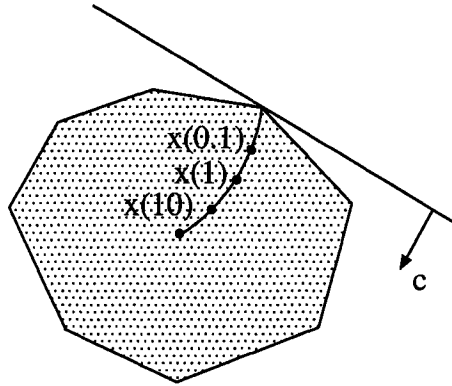


Figure 4: The central path

if some $x_j \leq 0$. The barrier problem has a unique solution $x(\mu)$ if the original linear program has a compact set of optimal solutions. The set of all optimal solutions for varying μ is known as the *central path*. It can be shown that $x^* = \lim_{\mu \rightarrow 0} x(\mu)$ exists and is an optimal solution for the original linear program, see Fig. 4.

The barrier problem from the dual problem is

$$\max\{y^T b + \mu \sum_{j=1}^n \log(z_j) \mid y^T A + z^T = c^T\}. \quad (2.32)$$

The optimal solutions $(x(\mu), y(\mu), z(\mu))$ to the barrier problems (2.31) and (2.32) have to satisfy the *Karush-Kuhn-Tucker conditions*

$$\begin{aligned} Ax(\mu) &= b \\ x(\mu) &\geq 0 \\ A^T y(\mu) + z(\mu) &= c \\ z(\mu) &\geq 0 \\ X(\mu)Z(\mu)e &= \mu e, \end{aligned} \quad (2.33)$$

where $X(\mu) = \text{diag}(x_1(\mu), \dots, x_n(\mu))$, $Z(\mu) = \text{diag}(z_1(\mu), \dots, z_n(\mu))$ are diagonal matrices and $e = (1, \dots, 1)^T$.

The path-following method starts with a feasible solution (x, y, z) close to the point $(x(\mu), y(\mu), z(\mu))$ on the central path. The basic idea is to decrease the value of μ to $\bar{\mu}$ and to apply Newton's method in order to obtain a new feasible solution $(\bar{x}, \bar{y}, \bar{z})$ close to $(x(\bar{\mu}), y(\bar{\mu}), z(\bar{\mu}))$, see Fig. 5. The iterative update of the solution (x, y, z) involves the following steps:

1. Determine an appropriate value for $\bar{\mu}$.
2. Apply Newton's method to the non-linear system (2.33) in order to obtain step directions $(\Delta x, \Delta y, \Delta z)$ pointing approximately to the point $(x(\bar{\mu}), y(\bar{\mu}), z(\bar{\mu}))$ on the central path.

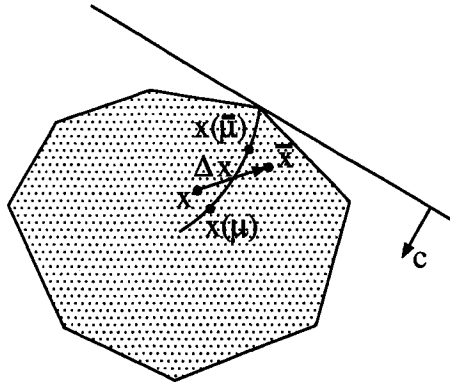


Figure 5: The path following algorithm

3. Compute step length parameters θ_P and θ_D such that the new point

$$\bar{x} = x + \theta_P \Delta x, \bar{y} = y + \theta_D \Delta y, \bar{z} = z + \theta_D \Delta z$$

satisfies $\bar{x}, \bar{z} > 0$.

4. Replace (x, y, z) with $(\bar{x}, \bar{y}, \bar{z})$.

The key operation is the application of Newton's method in Step 2. This requires the solution of a system of linear equations based on *Cholesky factorization*. For additional details and the convergence analysis we refer to [Bertsimas and Tsitsiklis 1997, Vanderbei 1997, Ye 1997].

2.2.5. Fourier-Motzkin elimination

Fourier [1826] found a method for solving systems of linear inequalities by successive elimination of variables. The method may be regarded as an extension of Gauss elimination (see Section 2.1.2) for systems of linear equations. It was forgotten until rediscovered in this century by Dines and Motzkin [Dantzig 1963, Dines 1919, Motzkin 1936].

The basic idea is simple: Suppose we are given a system S of m linear inequalities (constraints) in n real variables $x = (x_1, \dots, x_n)^T$ with real coefficients of the form $Ax \leq b$. Then by isolating one variable, say x_1 on the left hand side and dividing by the coefficient of x_1 , we arrive at an equivalent system S' together with constraints of the form $f_i \leq x_1$ ($1 \leq i \leq k$), $x_1 \leq g_j$ ($k < j \leq m$), where S' is a system of linear constraints in the variables $(x_2, \dots, x_n)$, f_i, g_j are linear polynomials in the variables $(x_2, \dots, x_n)$. So if the given system S has a real solution $(x_1, \dots, x_n)$ then the new system S' together with the constraints $f_i(x_2, \dots, x_n) \leq g_j(x_2, \dots, x_n)$ ($1 \leq i \leq k, k < j \leq m$), has a solution. Conversely, every solution $(x_2, \dots, x_n)^T$ of the new system can be completed to a solution of the old system S by putting $x_1 = \max(f_1, \dots, f_k)$. Notice that the method admits additive parameters (in the entries of b) corresponding geometrically to arbitrary translations of the linear constraints.

Using the trick of the “sliding objective function”, the method can be used to solve linear programming (LP) problems $\min\{c^T x \mid Ax \leq b\}$. Form a new system $Ax \leq b, c^T x \leq z$ of linear constraints with an additive parameter z . Then the elimination of all variables x will result in a system of linear constraints in the single parameter z that can be solved easily. The minimal feasible value of z will then agree with the minimal value z_0 of $c^T x$ under the given constraints. Plugging in this value z_0 for z in the next to last system, we can determine a corresponding value of x_n . Plugging in this value for x_n in the previous system yields a value for x_{n-1} and so on. In this manner we get the coordinates of a minimum point for $c^T x$ under the given constraints.

The major drawback of the method for practical purposes is the rapid increase of the number of constraints that happens during elimination: If in the notation above we have $k = m/2$, then the elimination of the variable x_1 from an input system consisting of m inequalities results in a system of $m^2/4$ inequalities. In the worst-case, this phenomenon can happen roughly in every elimination step, resulting in a doubly exponential increase of the number of inequalities. As a result the worst-case complexity of this method is doubly exponential in the number of variables, even if the computations on the coefficients are taken at unit cost [Weispfenning 1994].

So it is not surprising that - despite some optimization strategies due e.g. to Černikov, Kohler and Duffin [Černikov 1963, Williams 1986, Duffin 1974] - the method is usually unable to solve linear optimization problems of interesting size. Only for problems of very special type, e.g. at most two variables in each inequality there are polynomial upper bounds, see [Chandru 1993] and the references given there. From the theoretical viewpoint, however, the method remains important, since it provides a simple approach to basic structural theorems of LP such as the feasibility theorem, the Farkas Lemma and the Duality Theorem [Dantzig and Eaves 1973, Williams 1986]. An implementation is available e.g. in the linear algebra package of REDUCE. Other useful side-products of the Fourier-Motzkin method are the discovery of equations that are implicitly implied by a system of linear inequalities, and of inequalities that make the system inconsistent [Huynh, Joskowicz, Lassez and Lassez 1990, Lassez 1990].

As a more efficient alternative to Fourier-Motzkin elimination [Huynh et al. 1990, Lassez 1990] propose variable elimination in systems of linear inequalities via extreme point computation. This method eliminates a whole set of variables at once. By forming a quasi-dual of the given system S of linear constraints with respect to the variables to be eliminated, one arrives at a generalized linear optimization problem with a vector valued objective function Φ ranging over a bounded polyhedron P . Then the extreme points of the polytope $\Phi(P)$ determine a finite system of linear inequalities that describe the projection of the solution set of S into the space of the remaining variables.

2.2.6. Elimination by substitution of test points

In this section we describe another elimination method for systems of linear constraints that is both asymptotically and practically much more efficient than the

Fourier-Motzkin method. It was developed in [Weispfenning 1988, Weispfenning 1994, Loos and Weispfenning 1993, Weispfenning 1997c] and implemented in [Burhenne 1990, Kappert 1994], and in the REDLOG-package of REDUCE [Dolzmann and Sturm 1996, Dolzmann and Sturm 1997].

As in the Fourier-Motzkin approach described in the previous section, a given system S of linear constraints of the form $Ax \leq b$ is prepared for the elimination of x_1 by transforming it into an equivalent system S' together with constraints of the form $f_i \leq x_1$ ($1 \leq i \leq k$), $x_1 \leq g_j$ ($k < j \leq m$), where S' is a system of linear constraints for the variables $(x_2, \dots, x_n)$, f_i, g_j are linear polynomials in the remaining variables $(x_2, \dots, x_n)$. For fixed real values of the variables $(x_2, \dots, x_n)$, the latter system determines a closed interval for the possible real values of x_1 . This interval is non-empty iff one of the possible lower endpoints f_i is contained in it, or equivalently iff one of the possible upper endpoints g_j is contained in it. If no lower bound f_i (or no upper bound g_j) is present, then the interval is always non-empty. So if the given system S has a real solution $(x_1, \dots, x_n)$, then for some $1 \leq i \leq k$, $(f_i(x_2, \dots, x_n), x_2, \dots, x_n)$ will also be a solution; similarly for some $k < j \leq m$, $(g_j(x_2, \dots, x_n), x_2, \dots, x_n)$ will also be a solution. In other words, the given system S is feasible iff for some $1 \leq i \leq k$ the system S_i obtained from S by substituting $f_i(x_2, \dots, x_n)$ for x_1 is feasible; similarly for the systems obtained from S by substituting some $g_j(x_2, \dots, x_n)$ for x_1 . If no f_i (g_j) is present one may simply substitute $-\infty$ (∞) for x_1 . Notice that the variable x_1 has been eliminated from each of the new systems S_i . So the same elimination step can now be applied to the variable x_2 in each S_i yielding a new finite set of systems $S_{i,j}$ of linear constraints in $x_3, \dots, x_n$. Iterating this procedure, we arrive at a finite tree of such systems, whose bottom nodes are finite systems of inequalities between real constants, and hence can be evaluated to "true" or "false". The original system S has a real solution iff at least one of the bottom nodes is evaluated to "true".

In case of a linear optimization or feasibility problem, the elimination of a single variable by this method corresponds geometrically to a passage from the polyhedron P described by the linear constraints to the projections along a coordinate axis of some of its facets or the empty set. During the elimination of further variables the different facets produced earlier do not interact, but are treated separately. It is only after the completion of the elimination process that the different values of the objective function at the vertices of P produced by the elimination process are compared. This should be contrasted to the Fourier-Motzkin method, where elimination of a variable corresponds to the projection of all of P along a coordinate axis. So, roughly speaking, the method of substitution of test points is more "local" in character than the "global" Fourier-Motzkin method. This feature offers the additional advantage that the method is well parallelizable. First steps towards parallelization have been quite encouraging [Dolzmann, Gloor and Sturm 1998].

Another feature of the method is the fact that it can handle *arbitrary Boolean combinations of weak or strict linear inequalities* which may in addition include additive parameters: For "and"- "or"-combinations $\varphi(x)$ of weak linear inequalities one proceeds exactly as described above for conjunctive systems. It is not necessary to expand the given "and"- "or"-combination $\varphi(x)$ into disjunctive normal form;

this avoids the possibly exponential size explosion involved in this step. The method yields a finite number of test points t_i , $i \in I$ such that $\varphi(x)$ has a solution iff some t_i satisfies $\varphi(x)$. If $\varphi(x)$ contains additive parameters, the test points t_i , $i \in I$ will depend on these parameters. The characteristic property of the test points will then hold for every real value of the parameters.

If one admits in such an “and”-“or”-combination of constraints also strict linear inequalities, one requires additional test points of the form $f_i \pm \epsilon$, where the f_i are the test points used earlier and ϵ is a positive “*infinitesimal*”. The substitution of a test point of this form into the given constraints can be performed in such a way that the result does no longer contain the symbol ϵ . The same applies to the substitution of test points of the form $\pm\infty$. Hence the method is also referred to as *virtual substitution of test points* (compare Section 2.4.2).

Another extension of the method concerns Boolean combinations of quadratic inequalities (see Section 4.5).

2.3. Disequalities

A *positive linear constraint* is a conjunction

$$Ex = f, Ax \leq b \quad (2.34)$$

of linear equations and inequalities. A *negative linear constraint* is a disjunction of disequalities $c_i^T x \neq \delta_i$, $i = 1, \dots, m$, which will be written as $\neg Cx = d$, with a matrix $C = (c_1, \dots, c_m)^T \in \mathbb{F}^{m \times n}$ and a vector $d = (\delta_1, \dots, \delta_m) \in \mathbb{F}^m$. A system of *generalized linear constraints* [Lassez and McAloon 1992] is a finite conjunction of positive and negative linear constraints. We will use the notation

$$Ex = f, Ax \leq b, \{\neg C_k x = d_k\}_{k=1}^l. \quad (2.35)$$

The key for handling generalized linear constraints is the *independence of negative constraints*. The system (2.35) is feasible if and only if the positive constraints (2.34) together with one negative constraint $\neg C_k x = d_k$ are feasible, for all $k = 1, \dots, l$. Geometrically, this means that a polyhedron is contained in a finite union of affine sets if and only if it is contained in one of them. Suppose the positive constraints (2.34) are feasible and let $\text{Aff}(P)$ denote the affine closure of their solution set P . Lassez and Maher [1992] show that Fourier elimination can be easily modified to compute the affine hull of a polyhedron. The negative constraint $\neg Cx = d$ is consistent with (2.34) if and only if $\text{Aff}(P) \not\subseteq \{x \mid Cx = d\}$. Lassez and McAloon [1992] defined the following notion of *canonical form* for a system of generalized linear constraints:

- the linear equations $Ex = f$ are in *solved form*, i.e. x can be partitioned into (x_E, x_P) such that $Ex = f$ has the form $x_E = \tilde{E}_P x_P + \tilde{f}$
- the linear inequalities $Ax \leq b$ define a full-dimensional polyhedron P in x_P -space. Here, all implicit equalities and redundant inequalities have to be removed such that each inequality in $Ax \leq b$ defines exactly one facet of P .

- the negative constraints $\neg Cx = d$ are *precise*, i.e. for $A = \{x \mid Cx = d\}$ we have $A \cap P \neq \emptyset$, $A = \text{Aff}(A \cap P)$, and the complementary linear equations $Cx = d$ are again in solved form.

They showed that, using linear optimization, the canonical form can be computed in polynomial time. This implies that one can decide in polynomial time whether a system of generalized linear constraints (with rational coefficients over the rational numbers) is feasible, and, if so, compute a rational solution. Koubarakis [1996] extended this result to disjunctions with a finite number of disequalities and at most one inequality.

A lot of work on linear arithmetic constraint solving has been done in the area of constraint logic programming [Colmerauer 1987, Jaffar and Lassez 1987, Dincbas, van Hentenryck, Simonis, Aggoun and Graf 1988, Imbert 1995b]. While in $\text{CLP}(\mathcal{R})$, linear constraints are solved over floating point numbers, the constraint solvers of PROLOG III/IV and CHIP provide infinite precision rational arithmetic, at the expense of efficiency. Holzbaur [1995] has developed a PROLOG-based solver for real and rational linear arithmetic, which has been integrated in the constraint logic programming systems SICStus and Eclipse.

Linear arithmetic constraint solvers in constraint logic programming are based on the Simplex algorithm, which has to be modified in order to accommodate the requirements of incrementality and non-determinism, and the presence of disequalities [Stuckey 1991, Huynh and Marriott 1995]. An in depth description of the $\text{CLP}(\mathcal{R})$ language and system can be found in [Jaffar, Michaylov, Stuckey and Yap 1992], the solved form used in the CHIP system is described in [van Hentenryck and Graf 1992]. Variable elimination in $\text{CLP}(\mathcal{R})$ is discussed in [Huynh, Lassez and Lassez 1992, Jaffar, Maher, Stuckey and Yap 1993] and, with special emphasis, on disequalities in [Imbert 1993, Imbert 1997]. An empirical evaluation of different constraint solvers for $\text{CLP}(\mathcal{R})$ is given in [Burg, Stuckey, Tai and Yap 1995], while Imbert [1995a] compares and unifies several variable elimination methods for systems of linear inequalities that are based on Fourier's approach. Azulay and Pique [1998] study the use of Q-pivots for exact linear solvers.

2.4. Extensions

2.4.1. Parametric linear programming

A linear programming problem given by a system $Ax \leq b$ of linear constraints and an objective function $f(x) = c^T x$ is *parametric* if the data, i.e. the matrix A and the vectors b, c involve real parameters that may take on arbitrary real values. Typical instances are data that depend on a time parameter, or problems where some of the data may vary frequently, and hence should not be fixed in advance. The goal is either to get optimal solutions for all parameter values in some given range, or to perform a *sensitivity analysis*. In the latter case, one is interested in parameter values near a given point p in the parameter space; the goal is then to describe a region of parameter values containing p , where the optimal solution depends smoothly on the parameter values in some explicit way.

The elimination methods described in Section 2.2.5 and 2.2.6 are well-suited for parametric problems, since they are inherently parametric. Call the given linear programming problem *weakly parametric* if parameters occur only in the vector b and *strongly parametric* otherwise. In other words weakly parametric problems involve only additive parameters, while strongly parametric problems contain multiplicative parameters.

Both Fourier-Motzkin elimination and elimination by test points can readily cope with weakly parametric problems. The output of Fourier-Motzkin elimination is a conjunction ψ of linear inequalities involving the parameters and a new variable z representing the “sliding objective function”. For given parameter values the minimum of the objective function under the given constraints equals the minimal real value of z satisfying ψ . The output of elimination by test points is a list of triples consisting of a conjunction ψ of linear inequalities involving the parameters and z , a linear function q of the parameters and of z , and a tuple p of linear functions of the parameters and of z . For given parameter values the minimum value of z can be determined in each first entry of the triple, yielding a corresponding value of the objective function and point coordinates. By comparing the values of the objective function in each triple one finds a minimum value and the coordinates of a minimum point. This output can also easily be adapted to a sensitivity analysis near given parameter values.

As pointed out in Section 2.2.5, Fourier-Motzkin elimination is as a rule too complex to deal with weakly parametric problems in practice, whereas elimination by test points can handle problems of significant size. Both methods can also be adapted to the strongly parametric case by case distinctions on the parameters [Eaves and Rothblum 1992, Weispfenning 1994].

2.4.2. Linear quantifier elimination

The linear theory T_{lin} of real numbers is the first-order theory of the reals as an ordered linear space over the rational numbers. So the language L_{lin} of T has 0 as a constant, scalar multiplication with rational numbers as unary operations, $+$, $-$ as binary operations, and equality $=$ and order $\leq$ as binary relations. L_{lin} -terms can be written in the form of linear polynomials with rational coefficients. Atomic formulas are equations and inequalities between such L -terms. T_{lin} can be construed as the “linear fragment” of the first-order theory of the reals considered in Section 4.4. This relatively small fragment is nevertheless rich enough to express many important application problems, such as feasibility of linear constraints, linear programming with additive parameters, scheduling problems, simulation of electrical networks [Weispfenning 1997c, Sturm 1997].

Explicit algorithmic quantifier elimination procedures for T_{lin} are obtained both from slight extensions of Fourier-Motzkin elimination Section 2.2.5 and elimination by virtual substitution of test points Section 2.2.6.

2.10. THEOREM. T_{lin} admits quantifier elimination in L_{lin} in time $\ell^{(cn)^b}$ for some constant c . The validity of a closed formula can be decided in time $\ell^{(cn)^b}$ for some

constant c . Here ℓ is the length of the prenex input formula φ , n the number of quantified variables in φ and b the number of quantifier-blocks in φ .

The complexity of the decision problem is a direct consequence of the complexity of the quantifier elimination problem [Weispfenning 1988]. By a modified quantifier elimination procedure a sharper result was obtained in [Sontag 1985] for formulas with a bounded number b of quantifier blocks: The decision problem in T_{lin} for prenex formulas with b quantifier blocks beginning with an existential block is in the class Σ_b^P of the polynomial hierarchy. So the decision problem in T_{lin} for existential formulas is in NP; this special case was observed already in [Gathen and Sieveking 1976].

In order to prove quantifier elimination it suffices to eliminate one quantifier at a time. Moreover, since universal quantification can be expressed by existential quantification and negation via the equivalence

$$\forall x(\varphi) \iff \neg \exists x \neg(\varphi), \quad (2.36)$$

it suffices to eliminate one existential quantifier in front of a quantifier-free formula φ . By putting φ into disjunctive normal form and interchanging the existential quantifier with the disjunction we are reduced to the special case of one existential quantifier in front of a conjunction of equations and inequalities. Separating the quantified variable x on the left hand side of the equations and inequalities we arrive at an input formula of the form

$$\exists x \left(\bigwedge_{i=1}^m x \rho_i t_i \right), \quad (2.37)$$

where t_i are terms not containing the variable x , and ρ_i is one of the relations "=", "<", ">". If this formula contains at least one equation, say $x = t_1$, then it is equivalent to

$$\bigwedge_{i=2}^m (t_1 \rho_i t_i). \quad (2.38)$$

Otherwise it is essentially of the form

$$\exists x \left(\bigwedge_{i=1}^k x > t_i \wedge \bigwedge_{j=k+1}^m x < t_j \right). \quad (2.39)$$

If $k = 0$ or $m = k$ then this formula is obviously true. Otherwise, quantifier elimination via the Fourier-Motzkin method produces the equivalent formula

$$\bigwedge_{i=1}^k \bigwedge_{j=k+1}^m t_i < t_j. \quad (2.40)$$

In contrast, elimination via substitution of test points produces the equivalent formula

$$\bigvee_{r=1}^k \bigvee_{s=k+1}^m \bigwedge_{i=1}^k \frac{1}{2}(t_r + t_s) > t_i \wedge \bigwedge_{j=k+1}^m \frac{1}{2}(t_r + t_s) < t_j. \quad (2.41)$$

The advantage of the second method is the fact that subsequent further existential quantifiers can be interchanged with the disjunction produced in this elimination, whereas the first method yields one large conjunction. Under iteration the second method thus produces the indicated complexity behaviour, whereas the first is much more complex [Weispfenning 1988, Weispfenning 1994, Weispfenning 1997c]. In fact the second method can be improved considerably in practical complexity, when one admits virtual substitution of test points (compare Section 2.2.6). Then the output will be the shorter of the following two expressions:

$$\bigvee_{r=1}^k \bigwedge_{i=1}^k t_r + \epsilon > t_i \wedge \bigwedge_{j=k+1}^m t_r + \epsilon < t_j. \quad (2.42)$$

$$\bigvee_{s=k+1}^m \bigwedge_{i=1}^k t_s - \epsilon > t_i \wedge \bigwedge_{j=k+1}^m t_s - \epsilon < t_j. \quad (2.43)$$

Here ϵ denotes a positive infinitesimal number that can be eliminated easily from these expressions yielding again quantifier-free formulas [Loos and Weispfenning 1993].

A side-effect of elimination by test points is the fact that this method yields sample answers for existentially quantified variables. This feature is of importance in many applications, especially in optimization problems [Weispfenning 1994, Weispfenning 1997c].

As a by-product an analysis of the properties required in the course of these quantifier elimination procedures shows that only the axioms of ordered linear spaces over the field of rational numbers are required. So quantifier elimination and the induced decision procedure are valid in all such spaces in particular in all subfields of the reals:

2.11. COROLLARY. *T_{lin} is the common first-order theory of all ordered linear spaces over the field of rational numbers in L_{lin} .*

The upper bounds for the quantifier elimination and decision problem in T_{lin} have closely matching worst-case lower bounds. The quantifier elimination problem for T_{lin} is inherently doubly exponential in the worst-case [Weispfenning 1988]. More precisely: There is a series $\{\varphi_n\}$ of L -formulas of length linear in n such that for any T_{lin} -equivalent series of quantifier-free formulas $\{\psi_n\}$ the length of ψ_n grows doubly exponentially with n . For formulas with a bounded number b of quantifier blocks there is a corresponding refined singly exponential lower bound, where the second exponent equals b . The decision problem in T_{lin} for prenex closed formulas with a bounded number b of quantifier blocks starting with a block of existential quantifiers is complete for the class Σ_b^P of the polynomial hierarchy; in particular the decision problem in T_{lin} for closed existential formulas is NP-complete [Sontag 1985, Fürer 1982]. As a consequence the decision problem in T_{lin} for closed universal formulas is coNP-complete. Using conjunctive normal forms any closed universal formula can be equivalently rewritten as a conjunction of universal

formulas asserting the fact that a polyhedron P is contained in a union of polyhedra $P_1, \dots, P_n$. So this problem class is also coNP-complete [Srivastava 1993].

3. Linear diophantine constraints

In this section, we study linear arithmetic constraints over the integer numbers. While linear equations over $\mathbb{Z}$ are still solvable in polynomial time, see Section 3.1, linear inequalities over $\mathbb{Z}$ (and also linear equations over $\mathbb{N}$) are NP-hard, see Sections 3.2 to 3.7. A lot of work on linear diophantine constraints has been done in integer linear programming. Standard references are [Schrijver 1986, Nemhauser and Wolsey 1988, Wolsey 1998, Hooker 2000].

3.1. Equations over $\mathbb{Z}$

3.1.1. One linear equation over $\mathbb{Z}$

A linear diophantine equation

$$\alpha_1 x_1 + \dots + \alpha_n x_n = \beta, \quad (3.1)$$

with $\alpha_1, \dots, \alpha_n, \beta \in \mathbb{Z}$ has an integer solution if and only if the greatest common divisor $\gcd(\alpha_1, \dots, \alpha_n)$ of the coefficients on the left-hand side divides the right-hand side β . Using the Euclidean algorithm, we can decide in polynomial time if a solution exists, and if so, find one. We start by computing $\alpha, \gamma, \delta \in \mathbb{Z}$ such that

$$\alpha' = \gcd(\alpha_1, \alpha_2) = \alpha_1 \gamma + \alpha_2 \delta. \quad (3.2)$$

Then we solve recursively the linear diophantine equation in $n - 1$ variables

$$\alpha' x' + \alpha_3 x_3 + \dots + \alpha_n x_n = \beta. \quad (3.3)$$

If (3.3) has no solution, then (3.1) has no solution. If (3.3) has the solution $x', x_3, \dots, x_n$, then $x_1 = \gamma x', x_2 = \delta x', x_3, \dots, x_n$ defines an integral solution to (3.1).

3.1.2. Systems of linear diophantine equations over $\mathbb{Z}$

Next we consider systems of linear equations over $\mathbb{Z}$. There are the following basic results.

3.1. PROPOSITION (Solvability). *Given a rational matrix $A \in \mathbb{Q}^{m \times n}$ and a rational vector $b \in \mathbb{Q}^m$, the system $Ax = b$ has an integral solution $x \in \mathbb{Z}^n$ if and only if $y^T b$ is an integer, for each vector $y \in \mathbb{Q}^m$ for which $y^T A$ is integral.*

3.2. THEOREM. *Given a system $Ax = b$ with $A \in \mathbb{Q}^{n \times n}$, $b \in \mathbb{Q}^n$ we can decide in polynomial time if it has an integral solution $x \in \mathbb{Z}^n$, and if so, find*

one. If the system is solvable, we can also find in polynomial time integral vectors $x^0, x^1, \dots, x^k \in \mathbb{Z}^n$ such that

$$\text{Sol}_{\mathbb{Z}}(Ax = b) = x^0 + \mathbb{Z} \cdot x^1 + \dots + \mathbb{Z} \cdot x^k, \quad (3.4)$$

with $x^1, \dots, x^k$ linearly independent.

Theorem 3.2 can be proved by computing the Hermite normal form of the matrix A . We say that a rational matrix A of full row rank is in *Hermite normal form* [Hermite 1851] if it has the form $(B \ 0)$, where B is a non-singular, lower triangular, non-negative matrix in which each row has a unique maximal entry, and this entry lies on the maximal diagonal of B . Hermite mentioned that every rational matrix A of full row rank can be brought into Hermite normal form by a series of *elementary integer column operations* of the form

- exchanging two columns
- multiplying a column by -1
- adding an integral multiple of one column to another column.

The Hermite normal form of A is uniquely determined by A . The classical Hermite normal form algorithm is an integer variant of Gaussian elimination. Although the encoding length of the Hermite normal form $(B \ 0)$ of the rational matrix A (of full row rank) is bounded by a polynomial in $\langle A \rangle$, and the number of elementary operations is also polynomially bounded, the intermediate numbers calculated during the execution can become enormously large. Hafner and McCurley [1991] give an example of a 20×20 integer matrix with entries only between 0 and 10, but which needs integers of up to 1500 decimal digits in a standard Hermite normal form algorithm. Fang and Havas [1997] proved an exponential lower bound on the encoding length of the intermediate entries in a well-defined variant of integer Gaussian elimination for Hermite normal form computation.

The first polynomial algorithm for computing the Hermite normal form was found by Frumkin [1976], who uses modular arithmetic to control the size of the entries that arise during the transformation. Others that have used modular arithmetic in various forms include [Schrijver 1986, Dormich, Kannan and Trotter 1987, Iliopoulos 1989a, Iliopoulos 1989b, Hafner and McCurley 1991, Storjohann and Labahn 1996]. Let $A \in \mathbb{Z}^{m \times n}$ be an integer matrix of rank m . Let $\delta = |\det(A)|$ be the absolute value of the determinant of some submatrix of A of rank m and consider the $m \times (n+m)$ -matrix $A' = (A \mid D)$ obtained from A by extending it with the $m \times m$ diagonal matrix $D = \text{diag}(\delta, \dots, \delta)$. One easily checks that the Hermite normal forms of A and A' are the same except for the last m columns of the Hermite normal form of A' , which are all zero. Therefore, we may work with A' instead of A . The basic idea to prevent explosion of the intermediate entries is then to perform further elementary operations using the m additional columns in A' to bring all entries in the original n columns between 0 and δ .

Kannan and Bachem [1979] used a rearrangement in the order of operations of the classical Hermite normal form algorithm and proved a polynomial time and space bound. In contrast to previous approaches that built the Hermite normal

form row by row, their algorithm successively puts the submatrices consisting of the first i row and columns into Hermite normal form. Chou and Collins [1982] improved Kannan and Bachem's procedure and gave a better space bound. Havas, Holt and Rees [1993] and Havas and Majewski [1994] developed a heuristic pivot selection strategy to reduce the intermediate entry growth in Gaussian elimination methods, for which they obtain good empirical results.

Smith [1861] proved that any integer matrix A can be transformed by elementary integer row and column operations into the *Smith normal form*

$$\begin{pmatrix} D & 0 \\ 0 & 0 \end{pmatrix}, \quad (3.5)$$

where $D = \text{diag}(\delta_1, \dots, \delta_k)$ and $\delta_1, \dots, \delta_k$ are positive integers such that δ_i divides δ_j , for all $1 \leq i \leq j \leq k$. Computing the Smith normal form is studied, for example, in [Storjohann 1997, Havas and Majewski 1997].

3.1.3. Lattice reduction

A *lattice* in $\mathbb{R}^m$ is any set of the form

$$L = L(a_1, \dots, a_n) = \mathbb{Z}a_1 + \dots + \mathbb{Z}a_n, \quad (3.6)$$

with $a_1, \dots, a_n \in \mathbb{R}^m$. A system of linear diophantine equations $Ax = b$ is solvable over $\mathbb{Z}^n$ iff the right-hand side b belongs to the lattice in $\mathbb{R}^m$ that is generated by the columns of A . The solutions of a homogeneous system $Ax = 0, x \in \mathbb{Z}^n$ form a lattice in the space $\mathbb{R}^n$.

If the generators $a_1, \dots, a_n$ of a lattice L are linearly independent, then $(a_1, \dots, a_n)$ is called a *basis* of the lattice. Clearly, a lattice L may have several bases. Among the many possible bases, one would like to select one with some nice properties. The famous *LLL algorithm* [Lenstra, Lenstra and Lovász 1982] computes for given $a_1, \dots, a_n \in \mathbb{Q}^n$ in polynomial time a so-called *reduced* basis of the lattice $L(a_1, \dots, a_n)$. The purpose of a reduced basis is to have short vectors that are nearly orthogonal. This algorithm has many important applications, e.g. in diophantine approximation or in cryptography. It can also be used to compute the Hermite normal form of an integer matrix [Havas, Majewski and Matthwes 1998].

The LLL algorithm computes in polynomial time an approximation of a shortest vector in the lattice. The problem of finding a nonzero lattice vector $v \in L(a_1, \dots, a_n)$ such that the norm $\|v\|$ is minimal is called the *shortest lattice vector problem*. For the maximum norm $\|x\|_\infty = \max_{i=1, \dots, m} |x_i|$, this problem is known to be NP-hard [van Emde Boas 1981]. The question whether this problem is NP-hard also for the Euclidean norm $\|x\|_2 = \sqrt{x^T x}$ has been open for almost two decades. For randomized reductions, it was recently solved in the affirmative by Ajtai [1998]. A related, but essentially different problem is the *nearest lattice vector problem*. It consists in finding for a vector $b \in \mathbb{Q}^m$ a vector $v \in L(a_1, \dots, a_n)$ such that $\|b - v\|$ is minimal. This problem is known to be NP-hard for any norm [van Emde Boas 1981]. Henk [1997] showed that the shortest lattice vector problem is

polynomially reducible to the nearest lattice vector problem. Various authors have studied recently the complexity of approximating shortest or nearest lattice vectors, see e.g. [Arora, Babai, Stern and Sweedyk 1993, Blömer and Seifert 1999]. The *minimum basis problem* of finding a basis $(b_1, \dots, b_n)$ such that $\|b_1\| \cdot \dots \cdot \|b_n\|$ is minimal, is also NP-hard [Grötschel et al. 1988].

3.2. Equations over $\mathbb{N}$, inequalities over $\mathbb{Z}$

All constraint problems that we have considered so far can be solved in polynomial time. In this section, we consider systems of linear inequalities over the integer numbers

$$Ax \leq b, x \in \mathbb{Z}^n, \quad (3.7)$$

and, as a special case, systems of linear equations over the non-negative integer numbers

$$Ax = b, x \in \mathbb{N}^n. \quad (3.8)$$

Any set of *propositional clauses* of the form

$$x_1 \vee \dots \vee x_m \vee \bar{y}_1 \vee \dots \vee \bar{y}_k \quad (3.9)$$

can be translated into a system of *clausal inequalities* of the form

$$\begin{array}{rccccccc} x_1 & + \dots + & x_m & + & (1 - y_1) & + \dots + & (1 - y_k) & \geq & 1 & \text{or} \\ x_1 & + \dots + & x_m & & -y_1 & - & \dots & -y_k & \geq & 1 - k. \end{array} \quad (3.10)$$

The clause set is satisfiable if and only if the corresponding system of clausal inequalities together with the bounds $0 \leq x_i, y_j \leq 1$ has an integer solution. This implies immediately that deciding the satisfiability of a system of linear diophantine inequalities is NP-hard.

The solution set of (3.7) may be infinite. However, there is always a finite parametric description.

3.3. THEOREM. *Let $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$, with $A \in \mathbb{Z}^{m \times n}$ and $b \in \mathbb{Z}^m$, be a rational polyhedron and let $S = P \cap \mathbb{Z}^n$ be the set of integer points contained in P . Then there exist finitely many points $p^1, \dots, p^k$ in S and a finitely many rays of P $r^1, \dots, r^l \in \mathbb{Z}^n$ such that*

$$S = \left\{ \sum_{i=1}^k \lambda_i p^i + \sum_{j=1}^l \mu_j r^j \mid \lambda_i, \mu_j \in \mathbb{N}, \sum_{i=1}^k \lambda_i = 1 \right\}. \quad (3.11)$$

The absolute value of all p^i and r^j can be bounded by $(n+1)\Delta$, where Δ is the maximum absolute value of the subdeterminants of the matrix $(A \mid b)$.

In particular, this theorem implies that if a system of linear inequalities has an integral solution, then it has one with encoding length polynomially bounded by $\langle (A \mid b) \rangle$. Therefore, we can conclude:

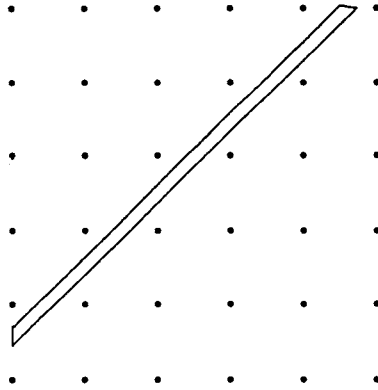


Figure 6: Polytope with empty integer hull

3.4. THEOREM. *The problem of deciding for a rational matrix $A \in \mathbb{Q}^{m \times n}$ and a rational vector $b \in \mathbb{Q}^m$ whether the system of linear inequalities $Ax \leq b$ has an integral solution $x \in \mathbb{Z}^n$ is NP-complete.*

In contrast to the polynomial solvability of 2SAT, the problem stays NP-complete if we admit only two variables in each constraint [Lagarias 1985]. The following problems are also NP-complete:

- Given $A \in \mathbb{Q}^{m \times n}, b \in \mathbb{Q}^m$, does $Ax = b$ have a *nonnegative* integral solution $x \in \mathbb{N}^n$?
- Given $A \in \mathbb{Q}^{m \times n}, b \in \mathbb{Q}^m$, does $Ax = b$ have a 0-1 solution $x \in \{0, 1\}^n$?
- Given $a \in \mathbb{Q}^n, \beta \in \mathbb{Q}$, does $a^T x = \beta$ have a *nonnegative* integral solution $x \in \mathbb{N}^n$?
- Given $a \in \mathbb{Q}^n, \beta \in \mathbb{Q}$, does $a^T x = \beta$ have a 0-1 solution $x \in \{0, 1\}^n$?
- Knapsack problem: Given $a, c \in \mathbb{Q}^n$ and $\beta, \delta \in \mathbb{Q}$, is there a 0-1 vector $x \in \{0, 1\}^n$ with $a^T x \leq \beta$ and $c^T x \geq \delta$?

While solving linear diophantine inequalities in general is NP-hard, a celebrated result of Lenstra [1983] states that this can be done in polynomial time, if the dimension n is fixed. Before Lenstra's theorem was obtained, this was known only for the cases $n = 1, 2$.

3.5. THEOREM (Lenstra 83). *For each fixed natural number n , there exists a polynomial algorithm which finds an integral solution $x \in \mathbb{Z}^n$ of a given system $Ax \leq b, A \in \mathbb{Q}^{m \times n}, b \in \mathbb{Q}^m$ in n variables, or decides that no such solution exists.*

The geometric intuition underlying Lenstra's algorithm can be described as follows [Aardal, Weismantel and Wolsey 1999, Aardal, Hurkens and Lenstra 1998a]. Consider a bounded polyhedron P like the one in Fig. 6, which may extend arbitrarily far in one direction. Depending on the length of P , a standard branch-and-bound

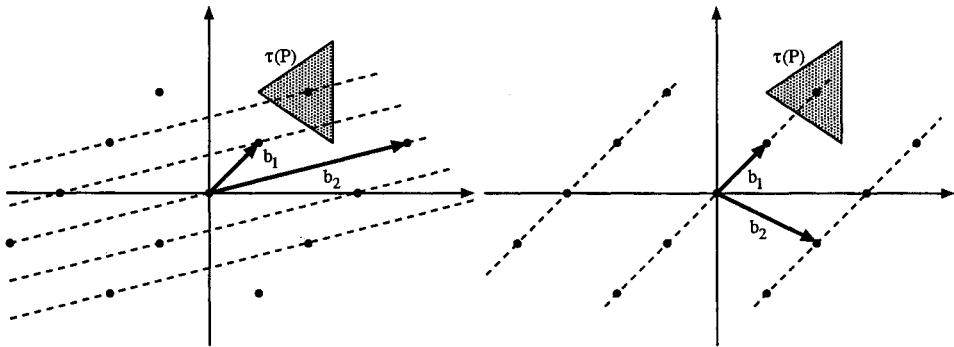


Figure 7: Non-orthogonal vs. nearly orthogonal lattice bases

algorithm (see Section 3.4) may require arbitrarily many iterations before concluding that no feasible solution exists. Suppose the polytope P is full-dimensional and consider the lattice $\mathbb{Z}^n$ with the lattice basis $e_1, \dots, e_n$. Our problem is to decide whether there exists a point $x \in P \cap \mathbb{Z}^n$. To avoid working with the thin polytope P , we may apply a linear transformation τ to make it appear more regular. Then we have to decide whether there exists a point $y \in \tau(P) \cap \tau(\mathbb{Z}^n)$. The new polytope $\tau(P)$ has a regular shape, but the basis vectors $\tau(e_j)$ are not necessarily orthogonal any longer, so the difficulty in branching is still present. To overcome this situation, we may use lattice reduction, see Section 3.1.3, to obtain a new basis $(b_1, \dots, b_n)$ of the lattice $\tau(\mathbb{Z}^n)$ having short and nearly-orthogonal vectors. In particular, one can show that the distance between two consecutive hyperplanes $H + kb_n, H + (k+1)b_n$, where $H = \mathbb{R}b_1 + \dots + \mathbb{R}b_{n-1}$ and $k \in \mathbb{Z}$, is not too short, which means that if we branch on these hyperplanes, then there cannot be too many of them, see Fig. 7.

Lovász and Scarf [1992] developed a *generalized basis reduction algorithm* for integer linear optimization. Like Lenstra's approach, this algorithm uses branching on hyperplanes. However, instead of using the transformation τ to transform the polytope and the initial basis, and then applying the standard basis reduction, their algorithm measures the width of the polytope in different independent directions to produce a so-called *Lovász-Scarf reduced basis*. Cook, Rutherford, Scarf and Shallcross [1993] describe a successful implementation of this algorithm.

Aardal et al. [1998a, 1998b] develop an algorithm for solving systems of linear diophantine equations with lower and upper bounds on the variables that is based on lattice basis reduction. This algorithm first finds a short vector satisfying the system of linear diophantine equations, and a set of vectors solving the corresponding homogeneous system. Due to basis reduction, all these vectors are relatively short. The next step is to branch on linear combinations of the solutions of the homogeneous system, which either yields a vector that satisfies the bound constraints or provides a proof that no such vector exists.

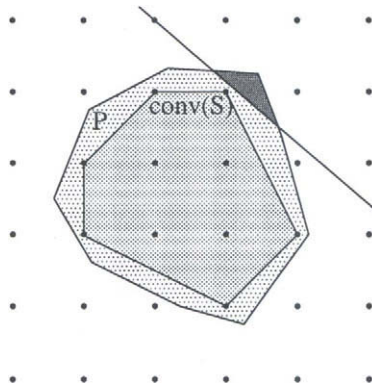


Figure 8: Cutting plane

3.3. Cutting planes

Consider the *integer linear optimization* or *integer linear programming* problem

$$\max\{c^T x \mid Ax \leq b, x \in \mathbb{Z}^n\}, \quad A \in \mathbb{Z}^{m \times n}, b \in \mathbb{Z}^m. \quad (3.12)$$

An important consequence of Theorem (3.3) is that for a rational polyhedron $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$, the convex hull of the set of integer points $S = P \cap \mathbb{Z}^n$ in P is again a rational polyhedron. Therefore, in principle, the integer linear optimization problem (3.12) can be reduced to an ordinary linear optimization problem

$$\max\{c^T x \mid x \in \text{conv}(S) \subseteq \mathbb{F}^n\} \quad (3.13)$$

over the rational polyhedron $\text{conv}(S)$. If (3.13) has a bounded optimal value, then it has an optimal solution, namely an extreme point of $\text{conv}(S)$, which is an optimal solution of (3.12). The objective value of (3.12) is unbounded if and only if the objective value of (3.13) is unbounded.

In order to solve the linear optimization problem (3.13), we need a linear inequality system $Cx \leq d$ defining the polyhedron $\text{conv}(S)$. In general, the polyhedron $\text{conv}(S)$ will have exponentially many facets. Therefore, such a description cannot be obtained in polynomial time. However, in order to find an optimal solution of (3.12), an approximation of $\text{conv}(S)$ might be sufficient. This idea leads to *cutting plane algorithms* for solving the integer linear optimization problem. We start with the *linear programming relaxation* $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ of the original problem. Then we add new inequalities which are satisfied by all points in S but which cut off at least one fractional vertex of P . These are called *cutting planes* (Fig. 8). This is repeated until an integer vertex is obtained, which belongs to the integer solution set S and maximizes $c^T x$.

3.3.1. Gomory-Chvátal cutting planes

One basic inference rule for deriving cutting planes is the Gomory-Chvátal rounding principle:

$$\text{Rounding: } \frac{a^T x \leq \beta}{a^T x \leq \lfloor \beta \rfloor} \text{ if } a \in \mathbb{Z}^n \quad (3.14)$$

If $a \in \mathbb{Z}^n$ is integer, then for all integer vectors $x \in \mathbb{Z}^n$, the left-hand side of the inequality $a^T x \leq \beta$ is integer. Therefore, we do not lose any integer solution if we round down the right-hand side β to the next integer number $\lfloor \beta \rfloor$. Geometrically, this means the following. Assume that the entries of a are coprime integer numbers and that β is fractional. Then rounding β means that we translate the hyperplane $a^T x = \beta$ until it meets an integer point. Given a system of linear inequalities $Ax \leq b$, a *Gomory-Chvátal cutting plane* is obtained by combining the rules `Nonneg.lin.com` and `Rounding`:

$$\text{GC_cp: } \frac{Ax \leq b}{u^T Ax \leq \lfloor u^T b \rfloor} \text{ if } \begin{cases} u \geq 0, \\ u^T A \in \mathbb{Z}^n \end{cases} \quad (3.15)$$

A *cutting plane proof* of length l of an inequality $c^T x \leq \delta$ from $Ax \leq b$ is a sequence of inequalities $c_1^T x \leq \delta_1, \dots, c_l^T x \leq \delta_l$ such that

$$\text{CG_cp: } \frac{Ax \leq b, c_1^T x \leq \delta_1, \dots, c_{i-1}^T x \leq \delta_{i-1}}{c_i^T x \leq \delta_i} \quad (3.16)$$

for each $i = 1, \dots, l$, and

$$\text{weak_rhs: } \frac{c_i^T x \leq \delta_i}{c^T x \leq \delta}. \quad (3.17)$$

Gomory-Chvátal cutting planes provide a complete inference system for linear diophantine inequalities.

3.6. THEOREM (Chvátal 73, Schrijver 80). *Let $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ be a non-empty polyhedron that is rational or bounded and let $S = P \cap \mathbb{Z}^n$.*

- *If $S \neq \emptyset$ and $c^T x \leq \delta$ is valid for S , then there is a cutting plane proof of $c^T x \leq \delta$ from $Ax \leq b$.*
- *If $S = \emptyset$, then there is a cutting plane proof starting from $Ax \leq b$ yielding the contradiction $0 \leq -1$.*

In the given form, the Gomory-Chvátal procedure is not effective. It is not clear how to choose the weights in the non-negative linear combination of the given constraint matrix. However, several well-known inference rules are subsumed by the Gomory-Chvátal principle, see Section 3.7. The first cutting plane procedure for solving integer linear optimization problems was proposed by Gomory in 1958. Here, the cutting planes are generated from a basic feasible solution in the Simplex algorithm. If v is a vertex of the polyhedron P with some fractional component, then by a simple rounding operation one can obtain a valid inequality for $P \cap \mathbb{Z}^n$ that cuts off v . Gomory showed that by systematically adding these cuts, and using the dual Simplex method with appropriate anticycling rules, one can obtain a finitely

terminating cutting plane algorithm for general integer linear optimization problems. The practical performance of Gomory's original cutting plane algorithm is not good. However, when used within a branch-and-cut framework (see Section 3.4.3), Gomory's cutting planes turned to be very useful [Balas, Ceria, Cornuéjols and Natraj 1996]. As a consequence, they are currently used in most state-of-the-art integer programming software packages.

The set of vectors P' satisfying all cutting planes for P is called the *elementary closure* of P . Let $P^{(0)} = P$ and $P^{(i+1)} = (P^{(i)})'$, for $i \geq 0$. The *Chvátal rank* of P is the smallest number t such that $P^{(t)} = P_I$, where $P_I = \text{conv}(P \cap \mathbb{Z}^n)$ is the *integer hull* of P . It indicates the "degree of discreteness" of the underlying integer linear optimization problem. Chvátal [1973] showed that every bounded polyhedron $P \subseteq \mathbb{R}^n$ has finite rank. Schrijver [1980] extended this result to possibly unbounded, but rational polyhedra $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$, where $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$. Cook, Gerards, Schrijver and Tardos [1986] and Gerards [1990] proved that for every matrix $A \in \mathbb{Z}^{m \times n}$ there exists $t \in \mathbb{N}$ such that for all right-hand sides $b \in \mathbb{Z}^m$, the Chvátal rank of $P_b = \{x \in \mathbb{F}^n \mid Ax \leq b\}$ is bounded by t . Already in dimension 2, there exist rational polyhedra of arbitrarily large Chvátal rank [Chvátal 1973]. Bockmayr, Eisenbrand, Hartmann and Schulz [1999] and Eisenbrand and Schulz [1999] studied the Chvátal rank of polytopes contained in the n -dimensional 0/1 cube $[0, 1]^n$. They showed that the Chvátal rank of a polytope $P \subseteq [0, 1]^n$ is $O(n^2 \log n)$ and prove the linear upper and lower bound n for the case $P \cap \mathbb{Z}^n = \emptyset$. Eisenbrand [1999] proved that the problem

Given $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$ and a rational vector $\hat{x} \in \mathbb{Q}^n$, decide whether $\hat{x}$ is outside the elementary closure P' of the polyhedron $P = \{x \in \mathbb{F}^n \mid Ax \leq b\}$.

is NP-complete. In other words, it is NP-complete to decide whether there is an inference using rule CG_cp that separates the point $\hat{x}$ from P' .

Caprara, Fischetti and Letchford [2000] apply Gaussian elimination in the field $\mathbb{Z}/k\mathbb{Z}$, k a prime number, to compute for a given *amount of violation* $\nu \in \{0, \dots, k-1\}$ a Gomory-Chvátal cutting plane $u^T Ax \leq \lfloor u^T b \rfloor$, with $u^T A \in k\mathbb{Z}$ and $u^T b - \lfloor u^T b \rfloor = \nu/k$. Bockmayr and Eisenbrand [1999] use Hermite normal form computation in the residue ring $\mathbb{Z}/d\mathbb{Z}$, where $d = |\det(A)|$, to compute in polynomial time a Gomory-Chvátal cutting plane with maximal amount of violation. They also derive, in fixed dimension, a polynomial upper bound on the number of inequalities needed to describe the elementary closure.

3.3.2. Disjunctive cutting planes

A second principle to derive cutting planes is based on disjunctive programming [Balas 1979, Balas 1998, Sherali and Shetty 1980]. A *disjunctive optimization problem* is of the form

$$\max\{c^T x \mid \bigvee_{h \in H} A^h x \leq b^h, x \geq 0\}, \quad (3.18)$$

for H finite, $A^h \in \mathbb{F}^{m_h \times n}$, and $b^h \in \mathbb{F}^{m_h}$. The feasible set of the disjunctive optimization problem (3.18) is the union of the polyhedra $P^h = \{x \in \mathbb{F}^n \mid A^h x \leq$

$b^h, x \geq 0\}, h \in H$. *Disjunctive cutting planes* are obtained by applying the rule

$$\text{disj_cp: } \frac{\bigvee_{h \in H} A^h x \leq b^h, x \geq 0}{a^T x \leq \beta} \text{ if } \begin{cases} u_h \in \mathbb{F}^{m_h}, u_h \geq 0, \\ a^T \leq u_h^T A^h, \\ u_h^T b^h \leq \beta, \\ \text{for all } h \in H. \end{cases} \quad (3.19)$$

We first take a non-negative linear combination of each disjunct and then weaken the left-hand and the right-hand side of all the inequalities obtained. The constraint $x \geq 0$ allows us to weaken also the left-hand side. The completeness of the disjunctive cutting plane principle was studied by several authors, see e.g. [Blair and Jeroslow 1978, Jeroslow 1977]. To apply the inference rule *disj_cp* to integer programming problems, one may, e.g., consider disjunctions of the form

$$\begin{array}{c} Ax \leq b, x \geq 0 \\ x_j \leq d \end{array} \quad \bigvee \quad \begin{array}{c} Ax \leq b, x \geq 0 \\ x_j \geq d + 1, \end{array} \quad (3.20)$$

for some $j \in \{1, \dots, n\}$ and $d \in \mathbb{Z}$. A successful branch-and-cut algorithm for general linear 0-1 optimization, which is based on disjunctive cutting planes, is the *lift-and-project method* developed by Balas, Ceria and Cornuéjols [1993, 1996]. Recently, some interesting connections between disjunctive cuts and Gomory's mixed integer cut have been discovered [Balas and Perregaard 2000].

3.4. Branch-and-infer

Although there exist complete inference systems for solving linear diophantine inequalities, like the Gomory-Chvátal rounding procedure, pure cutting plane algorithms do not work well in practice. From the practical point of view, it is much more efficient to combine inference with some kind of search. We next develop a general framework, *branch-and-infer* [Bockmayr and Kasper 1998, Kasper 1998], which allows us to describe various approaches for solving constraint problems over the integers in a uniform way. In particular, this framework unifies classical *integer linear programming* [Nemhauser and Wolsey 1988, Wolsey 1998, Johnson, Nemhauser and Savelsbergh 2000] and *finite domain constraint programming* [van Hentenryck 1989, Jaffar and Maher 1994, van Hentenryck and Saraswat 1996, Wallace 1996, Marriott and Stuckey 1998].

Branch-and-infer is based on a distinction between *primitive* and *non-primitive constraints*. Intuitively, the primitive constraints are those constraints that can be solved easily. The non-primitive constraints are the difficult constraints, which make the problem hard. In integer linear programming (ILP), the primitive constraints are linear equations and inequalities when solved over the real (or rational) numbers. The only non-primitive constraint is *integer*(x), i.e. the condition that certain variables should take integer values:

$$\begin{aligned} \text{Prim}(\text{ILP}) &= \{a^T x \diamond \beta \mid a \in \mathbb{Z}^n, \beta \in \mathbb{Z}, \diamond \in \{\leq, \geq, =\}\} \\ \text{NPrim}(\text{ILP}) &= \{\text{integer}(x)\} \end{aligned} \quad (3.21)$$

In finite domain constraint programming (CP(FD)), the primitive constraints either restrict the domain of one variable, or express equations between two variables. All other constraints are non-primitive. This includes linear equations, inequalities or disequalities in several variables, and symbolic constraints like `alldifferent(x)`, which states that certain variables should take pairwise different values:

$$\begin{aligned} \text{Prim}(FD) &= \{x_i \diamond \alpha, x_i = x_j \mid \alpha \in \mathbb{Z}, \diamond \in \{\leq, \geq, =, \neq, >, <\}\} \\ &\quad \cup \{\text{integer}(x)\} \\ \text{NPrim}(FD) &= \{a^T x \diamond \beta \mid a \in \mathbb{Z}^n, \beta \in \mathbb{Z}, \diamond \in \{\leq, \geq, =, \neq, >, <\}\} \\ &\quad \cup \{\text{alldifferent}(x), \dots\} \end{aligned} \quad (3.22)$$

The primitive constraints are collected in the *constraint store* [Saraswat 1993]. They define a *relaxation* of the problem for which an efficient solution method is available. The non-primitive constraints are handled locally by an *inference agent* that derives from a given non-primitive constraint and the current relaxation new primitive constraints that tighten this relaxation. Since, in general, a problem cannot be solved using the relaxation alone, inference has to be combined with search, which together provide a complete solution method. To describe branch-and-infer in a formal way, we use transition rules of the form

$$- : \frac{\langle P, S \rangle}{\langle P', S' \rangle} \text{ if } \textit{Cond} \quad (3.23)$$

saying that from a computation state $\langle P, S \rangle$ we may proceed to a computation state $\langle P', S' \rangle$ if the conditions in *Cond* are satisfied. Here $P = \{C_1, \dots, C_m\}$ (resp. P') denotes a set of constraint (sub-)problems $C_1, \dots, C_m$, which logically corresponds to the disjunction $C_1 \vee \dots \vee C_m$. The set S (resp. S') denotes a set of feasible solutions. For a set T and an element t , we will write $t.T$ instead of $\{t\} \cup T$. When solving a constraint problem C , the initial state is $\langle \{C\}, \emptyset \rangle$, and the final state is $\langle \emptyset, \{S\} \rangle$. If $S = \emptyset$, then the problem is infeasible.

The operational behaviour of the non-primitive constraints is formalized by the rule

$$\text{bi_infer} : \frac{\langle (c \wedge C).P, S \rangle}{\langle (p \wedge (c \wedge C)).P, S \rangle} \text{ if } \begin{cases} c \text{ is non-primitive,} \\ p \text{ is primitive,} \\ \text{Prim}(C) \wedge c \rightarrow p, \\ \text{Prim}(C) \not\vdash p. \end{cases} \quad (3.24)$$

In integer linear programming, the relaxation obtained from the primitive constraints is the standard linear programming relaxation. Inferring a new primitive constraint corresponds to the generation of a cutting plane that cuts off some part of this relaxation. In finite domain constraint programming, the basic inference principle is *domain reduction*. The corresponding *local consistency* techniques have been studied in artificial intelligence for a long time, see e.g. [Mackworth 1977, Tsang 1993]. For each non-primitive constraint, so-called *propagation* algorithms try to remove inconsistent values from the domain of the variables occurring in the constraint. From a logical point of view, this corresponds to the

generation of a new primitive constraint of the form $x \leq u, x \geq l$, which is called *bound reasoning*, or $x \neq v$, which is called *domain reasoning*. Whenever the domain of a variable changes, all propagation algorithms of the constraints in which this variable occurs may become active and further reduce the domains of their variables.

In general, inferring primitive constraints alone will be not sufficient to solve the given problem, either because the primitive constraints are not expressive enough, like in CP(FD), or because exponentially many of them would be necessary, like in ILP. Therefore, we need a second principle that allows us to tighten the relaxation further if the inference process has been suspended. The idea is to split the problem into subproblems and to process each subproblem independently of the others. The subproblems are obtained by setting up branching constraints and adding to each of them one copy of the problem under consideration. If the branching constraints are chosen in the right way, the relaxation of a subproblem will be strictly stronger than the relaxation of the father problem. Therefore, the inference agents associated with the non-primitive constraints may become active again and derive new primitive constraints. The branching operation is described by the rule

$$\text{bi_branch} : \frac{\langle C.P, S \rangle}{\langle \{c_1 \wedge C, \dots, c_k \wedge C\} \cup P, S \rangle} \text{ if } \begin{cases} C \equiv C \wedge (\bigvee_{i=1}^k c_i), \\ c_i \text{ is primitive,} \\ \text{Prim}(C) \not\vdash c_i, \\ i = 1, \dots, k. \end{cases} \quad (3.25)$$

The constraints $c_1, \dots, c_k$ are called the *branching constraints*, the problems $\{c_1 \wedge C, \dots, c_k \wedge C\}$ are the new *subproblems*. In many applications, we will have a binary branching of the form $c_1 \equiv c, c_2 \equiv \neg c$. By repeated application of the branching rule, we build up a *search tree*. Theoretically, we can get a complete enumeration of all the solutions in $\text{Sol}_Z(C)$. In practice, generating new subproblems has to be avoided as much as possible. Branching can be avoided if the (sub-)problem is infeasible. Since deciding the satisfiability of the whole problem is intractable, we test feasibility only on the primitive constraints, i.e. the relaxation. The next rule describes pruning by the infeasibility of the relaxation, which is denoted by $\perp$.

$$\text{bi_clash} : \frac{\langle C.P, S \rangle}{\langle P, S \rangle} \text{ if } \text{Prim}(C) \rightarrow \perp \quad (3.26)$$

The transition rules *bi_infer*, *bi_branch* and *bi_clash* are the basic rules in the branch-and-infer framework. What is still missing are rules that describe when a solution has been obtained. This depends on the type of problem to be solved, i.e. whether we want to find feasible or optimal solutions.

Solving a *constraint satisfaction* problem means deciding whether the problem is satisfiable and if so computing one or more feasible solutions. We hide the concrete method of computing feasible solutions from the relaxation and the way they are represented in a function *extract*. This function has to be chosen properly according to whether one wants to compute only one solution or more. This yields our

next rule:

$$\text{bi_sol} : \frac{\langle C.P, S \rangle}{\langle P, S \cup S^* \rangle} \text{ if } \begin{cases} S^* = \text{extract}(\text{Prim}(C)) \\ S^* \rightarrow C. \end{cases} \quad (3.27)$$

If one wants to compute more or all solutions, then repeated application of this rule to the different subproblems will collect the different solution families. Thus the task of finding more solutions is left to the control strategy for the application of the different transition rules.

In *constrained optimization*, one would like to compute a feasible solution that is optimal with respect to some objective function. For a maximization problem $\max\{f(x) \mid x \in \text{Sol}_Z(C)\}$, feasible solutions in $\text{Sol}_Z(C)$ yield lower bounds for the maximum value of f . In order to find an optimal solution, there exist two general methods, branch-and-bound, as it is used in finite domain constraint programming, and branch-and-relax resp. branch-and-cut, which are standard techniques in integer linear programming. Note that we follow here the terminology of constraint programming, where branch-and-relax corresponds to what is usually called branch-and-bound in integer linear programming. We now formalize the different approaches in our framework.

3.4.1. Branch-and-bound

The *branch-and-bound* method is characterized by using only lower bounds to find an optimal solution. Thus we solve a sequence of satisfiability problems leading successively to better solutions. More precisely, we repeatedly compute a feasible solution $s^* \in \text{Sol}_Z(C)$ and then add the constraint $f(x) \geq f(s^*) + 1$ to all the subproblems of our search tree, which restricts the set of feasible solutions to those that yield better objective function values. The constraint $f(x) \geq f(s^*) + 1$ is called a *lower bounding constraint*. If after adding a lower bounding constraint, all the subproblems become infeasible, then the last feasible solution is optimal. We require that f takes always integral values if x is integral since otherwise feasible solutions may be lost and the global optimum cannot be found. To describe branch-and-bound in our framework, we extend the transition system consisting of the rules *bi.branch*, *bi.clash* by the rule

$$\text{bi_climb} : \frac{\langle \{C_1, \dots, C_n\}, \{s\} \rangle}{\langle \{c \wedge C_1, \dots, c \wedge C_n\}, \{s^*\} \rangle} \text{ if } \begin{cases} s^* = \text{extract}(\text{Prim}(C_1)) \\ f(s^*) > f(s) \\ c \equiv (f(x) \geq f(s^*) + 1). \end{cases} \quad (3.28)$$

Here, the function *extract* is again responsible for computing a feasible solution of the relaxation. If no feasible solution is known, we assume $f(s) = -\infty$.

3.4.2. Branch-and-relax

In contrast to branch-and-bound, which uses only lower bounds, *branch-and-relax* works with two bounds. In addition to the global lower bound *glb* obtained from a feasible solution, we compute for each subproblem a local upper bound *lub*. For example, this can be done by optimizing the objective function subject to the

relaxation of a subproblem, i.e. the primitive constraints in the constraint store. We describe branch-and-relax again by an extension of the transition system given by the rules `bi_clash`, `bi_branch`. The local upper bounds allow us to introduce a new rule to prune the search tree. If a local upper bound is smaller than the best known global lower bound, then the corresponding subproblem cannot lead to a better solution and therefore can be discarded.

$$\text{bi_bound} : \frac{\langle C.P, \{s\} \rangle}{\langle P, \{s\} \rangle} \text{ if } \max\{f(x) \mid x \in \text{Solz}(C)\} \leq \text{lub} \leq f(s) \quad (3.29)$$

Furthermore, when computing a local upper bound, we may find an optimal solution of a subproblem that yields a better feasible solution of the whole problem.

$$\text{bi_opt} : \frac{\langle C.P, \{s\} \rangle}{\langle P, \{s^*\} \rangle} \text{ if } \begin{cases} \max\{f(x) \mid x \in \text{Solz}(\text{Prim}(C))\} = f(s^*) \\ s^* \in \text{Solz}(C) \\ f(s^*) > f(s) \end{cases} \quad (3.30)$$

Branch-and-relax is obtained from branch-and-bound by replacing the rule `bi_climb` with the two rules `bi_bound` and `bi_opt`.

3.4.3. Branch-and-cut

To apply branch-and-relax in practice, we must be able to compute local upper bounds in a computationally feasible way. For example, this is possible in integer linear programming, where we can obtain an upper bound by solving the linear programming relaxation. We may even find a feasible solution of the whole problem, so that the rule `bi_opt` can be applied. In order to obtain good lower bounds, we need *tight* linear relaxations. Given two systems $Ax \leq b$ and $A'x \leq b'$ defining the same set of feasible solutions, we say that $Ax \leq b$ is *tighter* than $A'x \leq b'$ if $\{x \in \mathbb{F}^n \mid Ax \leq b\} \subseteq \{x \in \mathbb{F}^n \mid A'x \leq b'\}$. Tight descriptions can be obtained statically by adding redundant inequalities to the problem formulation or dynamically by computing cutting planes that cut off some part of the linear relaxation without eliminating feasible solutions. Combining branch-and-bound with cutting plane generation, which is called *branch-and-cut*, is one of the most powerful techniques in ILP [Johnson et al. 2000]. In our framework, branch-and-cut is captured by the rules `bi_infer`, `bi_branch`, `bi_clash`, `bi_bound`, and `bi_opt`. Here, the last four rules describe branch-and-relax, while the first rule `bi_infer` allows for the generation of cutting planes.

3.5. Gröbner bases in integer programming

Another approach to solve integer linear optimization problems that recently has received a lot of interest is based on Gröbner bases [Conti and Traverso 1991, Thomas 1995, Thomas 1998, Ziegler 1999]. Consider an integer linear program of the form

$$\min\{c^T u \mid Au = b, u \in \mathbb{N}^n\}, \quad (3.31)$$

with $A \in \mathbb{Z}^{m \times n}$ of rank m , $b \in \mathbb{Z}^m$ and $c \in \mathbb{Z}^n$. A vector $u^* \in \mathbb{N}^n$ is a feasible solution of (3.31) iff $Au^* = b$. Consider the congruence class $\equiv_A$ on the monoid $\mathbb{N}^n$ defined by

$$u \equiv_A v \text{ iff } Au = Av. \quad (3.32)$$

Then the set of all feasible solutions of (3.31) corresponds to the congruence class $\{u \in \mathbb{N}^n \mid u \equiv_A u^*\}$. We are looking for an element of this class with minimal cost.

The starting point of the Gröbner basis approach is to identify a vector $u = (u_1, \dots, u_n) \in \mathbb{N}^n$ with a *monomial* $x^u = x_1^{u_1} \cdot \dots \cdot x_n^{u_n}$ in the polynomial ring $\mathbb{Q}[x_1, \dots, x_n]$. Then one considers the *toric ideal*

$$I_A = \langle x^u - x^v \mid Au = Av, u, v \in \mathbb{N}^n \rangle \quad (3.33)$$

generated by all *binomials*, i.e. monomial differences $x^u - x^v$ for which $u \equiv_A v$. A Gröbner basis $G_{A, \succ}$ of the ideal I_A , for some term order $\succ$, defines a confluent and terminating reduction relation on $\mathbb{Q}[x_1, \dots, x_n]$ that solves the word problem for the congruence relation generated by I_A . For any binomial p , we can compute a normal form $NF_{G_{A, \succ}}(p)$, which will again be a binomial. We have

$$u \equiv_A v \text{ iff } x^u - x^v \in I_A \text{ iff } NF_{G_{A, \succ}}(x^u - x^v) = 0. \quad (3.34)$$

If we choose a term order $\succ_c$ that is compatible with the objective function, i.e. $x^u \succ_c x^v$ for $c^T u > c^T v$, then we can obtain an optimal solution u^* from a feasible solution u° simply by normalizing x^{u° with $G_{A, \succ_c}$. This yields the following algorithm

1. Compute the reduced Gröbner basis $G_{A, c}$ of I_A w.r.t. $\succ_c$.
2. Compute the normal form x^{u^*} of x^{u° w.r.t. $G_{A, c}$. Then u^* is an optimal solution of (3.31).

Note that the Gröbner basis $G_{A, c}$ does not depend on the right-hand side b . In some sense, we are solving problem (3.31) for all right-hand sides. A key technical problem is to obtain a good starting basis of I_A that is needed to compute the Gröbner basis with Buchberger's algorithm.

3.6. Computing Hilbert bases

Next we present methods for computing *all* solutions of a system of linear diophantine constraints over the nonnegative integers. This problem is at the heart of various algorithms for equational unification, e.g. in associative, associative-commutative or distributive theories. Properties like liveness and reachability of Petri nets can also be verified by completely solving a system of linear equations over $\mathbb{N}$. The first complete method for solving a single homogeneous diophantine equation was proposed by Elliott [1903] and extended by MacMahon [1916] to the case of systems of homogeneous equations. The importance of this problem for unification theory and automated deduction, see e.g. [Bürckert, Herold, Kapur, Siekmann, Stickel, Tepp and Zhang 1988, Adi and Kirchner 1992], has stimulated a

lot of research during the last 20 years. Various new algorithms have been proposed for solving

- one linear diophantine equation [Huet and Lang 1978, Fortenbacher 1993, Guckenbiehl and Herold 1985, Clausen and Fortenbacher 1989, Lankford 1989, Filgueiras and Tomás 1991, Filgueiras and Tomás 1993],
- systems of linear diophantine equations [Romeuf 1990, Abdulrab and Pécuchet 1989, Pottier 1991a, Domenjoud 1991, Contejean and Devie 1994, Tomás and Filgueiras 1997, Pasechnik 1998],
- systems of linear diophantine equations, inequalities, and possibly disequalities [Abdulrab and Maksimenko 1995, Domenjoud and Tomás 1995, Boudet and Comon 1996, Ajili and Contejean 1997, Ajili and Lock 1998].

Our presentation is based on [Tomás 1997, Ajili 1998], which also contain overviews of the existing algorithms and a more detailed description of some of them.

3.6.1. Representing the solution set.

It has been known for a long time [Gordan 1873, Hilbert 1890] that the set of non-negative integer solutions of a system of homogeneous linear diophantine equations

$$Ax = 0, x \in \mathbb{N}^n, \quad (3.35)$$

with $A \in \mathbb{Z}^{m \times n}$, is a finitely generated submonoid $\mathcal{M}$ of $\mathbb{N}^n$. There exists a unique finite subset $\mathcal{H}(\mathcal{M})$, which is called the *Hilbert basis* of $\mathcal{M}$, such that each solution of (3.35) is a non-negative integer linear combination of elements in $\mathcal{H}(\mathcal{M})$. In contrast to Carathéodory's theorem in the real case, more than n elements of the Hilbert basis might be necessary to represent a given solution $x \in \mathcal{M}$ [Bruns, Gubeladze, Henk, Martin and Weismantel 1999, Sebő 1990]. The solutions in $\mathcal{H}(\mathcal{M})$ are the *minimal* elements of $\mathcal{M} \setminus \{0\}$ with respect to the partial ordering $\leq$ on $\mathbb{N}^n$ defined by

$$(a_1, \dots, a_n) \leq (b_1, \dots, b_n) \text{ iff } a_i \leq b_i, \text{ for all } i = 1, \dots, n. \quad (3.36)$$

Alternatively, $\mathcal{H}(\mathcal{M})$ contains precisely the *non-decomposable* elements of $\mathcal{M}$, i.e. those elements in $\mathcal{M} \setminus \{0\}$ that cannot be written as the sum of two other elements of $\mathcal{M} \setminus \{0\}$. The solution set of a system of homogeneous linear diophantine *inequalities*

$$Ax \leq 0, x \in \mathbb{N}^n \quad (3.37)$$

defines again a submonoid of $\mathbb{N}^n$. This monoid is finitely generated by its non-decomposable elements, but not by its minimal elements. For example, the solution set of $x - y \leq 0, x, y \in \mathbb{N}$, is generated by the non-decomposable elements $(0, 1)$ and $(1, 1)$, but not by the minimal solution $(0, 1)$. Finally, any solution of the *inhomogeneous* system of linear diophantine *inequalities*

$$Ax \leq b, x \in \mathbb{N}^n \quad (3.38)$$

with $A \in \mathbb{Z}^{m \times n}, b \in \mathbb{Z}^m$, can be obtained as the sum of a non-decomposable solution of the system $Ax - bz \leq 0, (x, z) \in \mathbb{N}^{n+1}$ with $z = 1$, and a non-negative integer linear combination of non-decomposable solutions of the homogeneous system (3.37) [Ajili 1998].

3.6.2. Algorithms.

Elliott [1903] was probably the first to develop a method for computing all minimal solutions $x \in \mathbb{N}^n$ of a linear diophantine equation

$$a_1x_1 + \dots + a_nx_n = 0, \quad a_1, \dots, a_n \in \mathbb{Z}. \quad (3.39)$$

He considered the *generating function*

$$\prod_{i=1}^n (1 + y_i\lambda^{a_i} + y_i^2\lambda^{2a_i} + \dots) = \prod_{i=1}^n \frac{1}{1 - y_i\lambda^{a_i}}, \quad (3.40)$$

where $y_1, \dots, y_n, \lambda$ are formal indeterminates. Expanding this function yields the sum of all terms of the form $y^u \lambda^\beta$, where $u \in \mathbb{N}^n, \beta \in \mathbb{Z}$ and $a_1u_1 + \dots + a_nu_n = \beta$. It follows that $u \in \mathbb{N}^n$ is a solution of (3.39) iff the expansion contains the term $y^u \lambda^0 = y^u$. The idea of Elliott is to rewrite this generating function until factors appear that do not contain λ . MacMahon [1916] extended this algorithm to the case of systems of homogeneous diophantine equations. Domenjoud and Tomás [1995] reformulated Elliott-MacMahon's approach as a transformation process on constrained parametric expressions and derived an algorithm for solving arbitrary linear diophantine constraints involving equations, inequalities, and disequalities. The algorithm computes a decomposition of the solution set into possibly overlapping subsets, obtaining a parametric representation of each subset from which minimal solutions can be extracted immediately. Another algorithm that computes a parametric representation was proposed by Abdulrab and Maksimenko [1995] based on a method of Makanin [1979], which applied only to systems of equations. In contrast to Domenjoud and Tomás' algorithm, the non-decomposable solutions may not occur explicitly in the parametric representation found. But, the subsets in the decomposition are pairwise disjoint, such that each solution is given in a unique way. A variant of the Elliott-MacMahon algorithm to compute the Hilbert Basis of a system of homogeneous linear diophantine equations is also presented in Pasechnik [1998].

Stanley [1973, 1986] found an explicit formula for the generating function and described a method to compute it. He analyzed the geometric structure of the polyhedral cone C defined by the non-negative real solutions of the system $Ax = 0$. The cone C can be represented as a finite union of simplicial cones. Domenjoud [1991] proposed an algorithm that looks for solutions in all simplicial subcones whose extreme rays are extreme rays of the solution cone C . As noted in [Tomás and Filgueiras 1997], Stanley's results imply that it is enough to consider a triangulation of the solution cone, which results in a much smaller number of subcones. The geometric structure of the solution cone was also used by [Tomás and Filgueiras 1997, Tomás 1997] to extend their *slopes algorithm*, which was originally developed for solving a single linear diophantine equation, to systems of equations. Another algorithm that they proposed is the *rectangles algorithm* [Filgueiras and Tomás 1993].

A second family of algorithms has been obtained by representing the partially ordered set $(\mathbb{N}^n, \leq)$ as a directed acyclic graph and by searching for minimal so-

lutions in that graph. These methods are quite efficient when the elements of the Hilbert basis have a small norm. A first algorithm of this type was proposed by Huet and Lang [1978] for solving the homogeneous linear diophantine equation

$$\sum_{i=1}^n a_i x_i = \sum_{j=1}^m b_j y_j, \quad x_i, y_j \in \mathbb{N}, \quad (3.41)$$

where $a_i, b_j \in \mathbb{N}$. The basic idea is to perform a lexicographic enumeration of tuples in $\mathbb{N}^{n+m}$ using the following bound on the components of minimal solutions

$$x_i \leq \max_{j=1, \dots, m} b_j \quad \text{and} \quad y_j \leq \max_{i=1, \dots, n} a_i. \quad (3.42)$$

Lambert [1987] substantially improved Huet's bound by showing that

$$\sum_{i=1}^n x_i \leq \max_{j=1, \dots, m} b_j \quad \text{and} \quad \sum_{j=1}^m y_j \leq \max_{i=1, \dots, n} a_i, \quad (3.43)$$

see also [Henk and Weismantel 2000] for a recent generalization.

Fortenbacher [1993] proposed to direct the search for a solution by visiting a successor node only if its residue is closer to zero than that of the current node. More precisely, let $v \in \mathbb{N}^{n+m}$ be some tuple found by the algorithm that is not greater than or equal to some solution that has already been obtained. Then

$$v + e_k \text{ is a successor of } v \text{ iff } \text{res}(v) \text{res}(e_k) < 0, \quad (3.44)$$

where the starting nodes $e_1, \dots, e_{n+m}$ form the canonical basis of $\mathbb{R}^{m+n}$, and the residue is given by $\text{res}(v) = \sum_{i=1}^n a_i v_i - \sum_{j=1}^m b_j v_{n+j}$. Guckenbiehl and Herold [1985] extended this algorithm to solve a single non-homogeneous linear diophantine equation. An improved version of both algorithms was developed in [Clausen and Fortenbacher 1989].

Contejean and Devie [1994] generalized Fortenbacher's algorithm to apply to a general system of homogeneous linear diophantine equations $Ax = 0, x \in \mathbb{N}^n$ by replacing (3.44) with

$$v + e_k \text{ is successor of } v \text{ iff } (Av)^T A e_k < 0, \quad (3.45)$$

The geometric interpretation of this condition is that $A(v + e_k)$ should lie in the halfspace orthogonal to Av containing the origin, which generalizes (3.44) to the m -dimensional case, where m denotes the number of equations to be solved (Fig. 9). While the correctness of this condition can be checked easily, the termination proof is technically involved and based on topological arguments [Contejean and Devie 1994].

Ajili [1994] proposed an algorithm for finding a complete representation of the solution set of a linear homogeneous inequality based on Fortenbacher's algorithm. Rather than using a slack variable to reduce the problem to that of solving a linear homogeneous equation, the algorithm solves the inequality directly. Based on

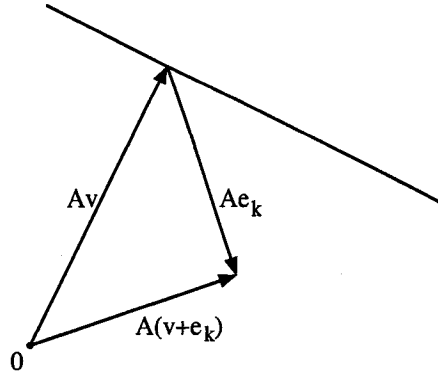


Figure 9: Criterion for determining successor nodes

this idea, Ajili and Contejean [1997] adapted the algorithm of Contejean and Devie [1994] to handle systems of linear equations and inequalities without introducing slack variables that would blow up the search space. This extension requires bounds on minimal solutions to ensure termination. [Ajili and Lock 1998] present an extended version of this algorithm that uses constraint propagation techniques to prune the search space.

The Gröbner basis approach for integer programming (see Section 3.5) can also be used to compute a Hilbert basis for a system $Au = 0, u \in \mathbb{N}^n$ [Pottier 1991a, Pottier 1991b, Sturmfels 1993]. Let G be the reduced Gröbner basis of the ideal

$$J_A = \langle x_1 - y_1 t_1^{a_{11}} \dots t_m^{a_{m1}}, \dots, x_n - y_n t_1^{a_{1n}} \dots t_m^{a_{mn}} \rangle \quad (3.46)$$

in the ring $\mathbb{Q}[x_1, \dots, x_n, y_1, \dots, y_n, t_1^{\pm 1}, \dots, t_m^{\pm 1}] = \mathbb{Q}[x_1, \dots, x_n, y_1, \dots, y_n, t_1, \dots, t_m, t_1^{-1}, \dots, t_m^{-1}] / (t_1 t_1^{-1} - 1, \dots, t_m t_m^{-1} - 1)$ with respect to a term order $\succ$ such that $t_i \succ x_j \succ y_k$. Then the Hilbert basis of $Au = 0, u \in \mathbb{N}^n$ can be obtained as $G \cap \{x^u - y^u \mid u \in \mathbb{N}^n\}$.

Boudet and Comon [1996] show how to compute efficiently an automaton that accepts the set of solutions of a linear diophantine equation, and more generally of a system of linear diophantine equations, inequalities and disequalities. Applying Büchi's techniques for Presburger arithmetic, they obtain a decision procedure for Presburger arithmetic, which has nearly optimal worst case complexity, both for the existential fragment and for the full first-order theory (compare Section 3.8 for more precise worst-case bounds).

3.6.3. Bounds on the size and number of minimal solutions.

Many authors have deduced bounds on the size of minimal solutions of $Ax = 0, x \in \mathbb{N}^n$ in terms of the matrix A . Suppose $A \in \mathbb{Z}^{m \times n}$ has full row rank m and denote by Δ (resp. Δ_m) the maximum absolute value of a subdeterminant (resp. $m \times m$ subdeterminant) of A . Lambert [1987] gave for each minimal non-negative solution

of $Ax = 0$ the bound $\|x\|_\infty \leq n\Delta$. Pottier [1991a] and Domenjoud [1991] improved this independently to

$$\|x\|_\infty \leq (n - m)\Delta_m. \quad (3.47)$$

Other bounds in terms of the norm of A are, e.g. [Domenjoud 1991, Pottier 1991a]

$$\|x\|_\infty \leq (n - m) \left(\frac{\|A\|_1}{m} \right)^m \quad \text{and} \quad \|x\|_\infty \leq (n - m) m^{\frac{m}{2}} \|A\|_\infty^m, \quad (3.48)$$

where $\|A\|_1 = \sum_{i,j} |a_{ij}|$ and $\|A\|_\infty = \max_{i,j} |a_{ij}|$.

Henk and Weismantel [1997] derive an upper bound on the height of minimal Hilbert bases. Given a pointed polyhedral cone $C = \text{cone}(a^1, \dots, a^k)$, with $a^i \in \mathbb{Z}^n$, and an element b in the minimal Hilbert basis of C , the *height* of b is defined as

$$h_C(b) = \min \left\{ \sum_{i=1}^k \lambda_i \mid b = \sum_{i=1}^k \lambda_i a^i, \lambda_i \geq 0 \right\}. \quad (3.49)$$

Lankford [1989] gave an exponential lower bound for the number of minimal solutions of the linear diophantine equation $nx_{n+1} = x_1 + 2x_2 + \dots + nx_n$. Domenjoud [1992] derives an explicit formula for the number of minimal solutions of the equation

$$a \cdot \sum_{i=1}^n x_i = b \cdot \sum_{j=1}^m y_j, \quad a, b \in \mathbb{N}. \quad (3.50)$$

The complexity of recognizing and counting Hilbert bases is studied in [Durand, Hermann and Juban 1999] and [Hermann, Juban and Kolaitis 1999].

3.7. Linear 0-1 inequalities

Linear 0-1 inequalities

$$Ax \geq b, x \in \{0, 1\}^n \quad (3.51)$$

with $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}$, are special linear diophantine constraints, which are particularly interesting from a logical point of view. There are many remarkable parallels between theorem proving and mathematical programming. The logical concepts of resolution, extended resolution, input and unit refutation, the Davis-Putnam-Procedure, and drawing of inferences pertinent to a given topic are closely related to the mathematical concepts of cutting planes, Chvátal's method, elementary closure, branch and bound, and projection of a polytope, respectively [Hooker 1988b, Chandru and Hooker 1999, Hooker 2000].

For example, it is easy to see that any resolvent of a set of propositional clauses can also be obtained as a Gomory-Chvátal cutting plane from the corresponding clausal inequalities and the bound inequalities $0 \leq x_i, y_j \leq 1$. But, Gomory-Chvátal cutting planes are strictly more powerful. For example, the unsatisfiability of the famous *pigeon-hole problem* (fit n pigeons into $n - 1$ holes) can be proved by cutting planes in $O(n^3)$ steps [Cook, Coullard and Turán 1987], whereas any resolution

proof requires exponentially many steps [Haken 1985]. A long-standing open problem was to prove a superpolynomial lower bound for the cutting plane proof system. This was finally solved by [Pudlák 1997], see also [Haken and Cook 1999], who found classes of propositional tautologies that admit no polynomial cutting plane proof. There exist several other proof systems for linear 0-1 inequalities whose complexity has been investigated, see [Pudlák 1999].

Hooker [1988a] and [1992] generalized resolution from propositional clauses to arbitrary linear 0-1 inequalities. We sketch his method for the case of extended clauses. An *extended clause* is a linear 0-1 inequality of the form

$$L_1 + \cdots + L_m \geq d, \quad (3.52)$$

where $d \geq 1$ is a positive integer number and $L_i, i = 1, \dots, m$, is either a *positive literal* x_i or a *negative literal* $\bar{x}_i = 1 - x_i$. The intuitive meaning of (3.52) is that at least d out of the m literals L_i have to be true. Classical clauses or clausal inequalities correspond to the case $d = 1$. If $x_i, i = 1, \dots, l$, are the positive literals and $\bar{y}_j, j = 1, \dots, k$, are the negative literals, then the extended clause (3.52) can also be written in the form

$$x_1 + \cdots + x_l - y_1 - \cdots - y_k \geq d - k. \quad (3.53)$$

In many cases, extended clauses give a more compact representation of a set $S \subseteq \{0, 1\}^n$ than classical clauses. For example, the extended clause $L_1 + \cdots + L_m \geq d$ is equivalent to the conjunction $\bigwedge_{I \subseteq \{1, \dots, m\}; |I|=m-d+1} \sum_{i \in I} L_i \geq 1$ of $\binom{m}{m-d+1}$ classical clauses.

A classical clause $L_1 + \cdots + L_m \geq 1$ implies another clause $L'_1 + \cdots + L'_k \geq 1$ if and only if $L = \{L_1, \dots, L_m\} \subseteq L' = \{L'_1, \dots, L'_k\}$. A similar result holds for extended clauses. Abbreviate an extended clause $L_1 + \cdots + L_m \geq d$ by $L \geq d$ and view L as a set of literals $\{L_1, \dots, L_m\}$ of cardinality $|L| = m$. Then $L \geq d$ implies $L' \geq d'$ if and only if $|L \setminus L'| \leq d - d'$. This means that the entailment problem for extended clauses is easy, while for arbitrary linear 0-1 inequalities it is coNP-complete. *Generalized resolution* is described by the inference rules:

$$\text{Impl_Ext_Cl: } \frac{L \geq d}{L' \geq d'} \quad \text{if } |L \setminus L'| \leq d - d' \quad (3.54)$$

$$\text{Resolution: } \frac{L_i + L \geq 1, \bar{L}_i + L' \geq 1}{L + L' \geq 1}$$

$$\begin{array}{rcl} \square & L_2 & + \cdots + L_{m-1} + L_m \geq d \\ L_1 & \square & + \cdots + L_{m-1} + L_m \geq d \\ \text{Diagonal_Sum:} & \vdots & \\ L_1 & + L_2 & + \cdots + L_{m-1} \square \geq d \\ \hline L_1 & + L_2 & + \cdots + L_{m-1} + L_m \geq d + 1 \end{array} \quad (3.55)$$

Generalizing a classical result from Quine, Hooker [1992] showed that by applying these rules one can compute all prime extended clauses for a set of extended clauses C . An extended clause $L \geq d$ is *prime* for C if $L \geq d$ is implied by C and if there is no other extended clause implied by C that implies $L \geq d$. A set of extended clauses is unsatisfiable if and only one can derive by generalized resolution the empty clause $0 \geq 1$. Barth [1996] introduced *simplifying* resolvents and diagonal sums and showed how these can be computed efficiently. He also devised an algorithm to compute for an arbitrary linear 0-1 inequality an equivalent set of extended clauses. Bockmayr [1993, 1995] and Barth and Bockmayr [1995, 1998] study logic-based and polyhedral 0-1 constraint solving in the context of constraint logic programming.

3.8. Presburger arithmetic

Presburger arithmetic PA is the first-order theory of the integers in the language L having 0, 1 as constants, +, − as binary operations, and equality =, order $\leq$ and congruences $\equiv_n$ modulo all integers $n \geq 1$ as binary relations. So L -terms in PA can be written in the form of linear polynomials with integer coefficients. Atomic formulas are equations, inequalities or congruences between such L -terms. Formally, integer coefficients of variables and the constant 1 as well as moduli of congruences are written in unary notation. PA can be construed as the “linear fragment” of number theory. This relatively small fragment is nevertheless rich enough to express many important application problems, such as solvability of (parametric) systems of linear Diophantine equations, integer feasibility of systems of (parametric) linear constraints, integer programming, and certain problems in program description and verification [Gurari 1985, Suzuki and Jefferson 1980].

The fundamental paper of Presburger [1929] established an effective quantifier elimination procedure and decision procedure for PA. The strength of his results could be appreciated only by 1931, when Gödel’s incompleteness theorem showed that full first-order number theory (with multiplication), and hence even the “quadratic fragment” of first-order number theory is algorithmically undecidable [Gödel 1931, Gurari and Ibarra 1979]. As a by-product an analysis of the properties required in the course of a quantifier elimination procedure provides easily explicit axioms for PA: Presburger arithmetic can be axiomatized by first-order axioms saying that $\mathbb{Z}$ is a discretely ordered Abelian group with smallest positive element 1 and that $|\mathbb{Z}/n\mathbb{Z}| = n$ for all natural numbers n . As a consequence, any lexicographical product $G = H \times \mathbb{Z}$ of a divisible ordered Abelian group H with $\mathbb{Z}$ is a model of PA.

The complexity of the decision problem for fragments and extensions of PA has been studied extensively [Berman 1980, Cooper 1972, Ferrante and Rackoff 1979, Fischer and Rabin 1974, Fürer 1982, Gathen and Sieveking 1978, Grädel 1987, Gurari and Ibarra 1979, Gurari and Ibarra 1981, Lipshitz 1978, Oppen 1973, Reddy and Loveland 1978, Scarpellini 1984, Shostak 1979, Volger 1983]. Roughly speaking, the central results are the following: The validity of a prenex closed L -formula φ in the domain of integers can be decided in deterministic space bounded by $\ell^{n^{cb}}$,

for some constant c , where ℓ is the length of the input formula φ . Here n is the number of quantified variables in φ and b is the number of quantifier-blocks in the prefix of φ . More precisely, this decision can be made in time bounded by $\ell^{n^{cb}}$ by an alternating Turing machine with b alternations of quantifiers. A corresponding lower bound due to Fischer and Rabin [1974] asserts that there is a series $\{\varphi_n\}$ of closed formulas of length linear in n such that the validity of φ_n cannot be algorithmically decided in non-deterministic time 2^{2^n} . This lower bound is achieved by the powerful technique of coding the multiplication of a large initial segment of the positive integers by a short PA-formula. This enables one to transform the undecidability of the elementary theory of integers with addition and multiplication into a hardness result for PA.

An upper bound for the worst-case complexity of the quantifier elimination problem in PA was obtained in [Weispfenning 1990b]: Let ℓ be the length of the input formula φ , n the number of quantified variables in φ and b the number of quantifier-blocks in φ . Then the bound is of the form $\exp(\ell^{(cn)^b})$ for some constant c , where $\exp(x) = 2^x$. In [Weispfenning 1997a] it was shown that this upper bound is essentially tight; so the quantifier elimination problem for PA is inherently triply exponential. The lower bound uses the Fischer-Rabin technique [Fischer and Rabin 1974] together with the construction of a series of PA-formulas in one parameter that describe large finite sets in a short way.

An efficient quantifier elimination procedure for PA [Weispfenning 1990b] goes along the lines of elimination by test points sketched in Section 2.4.2. The fact that elements of $\mathbb{Z}$ may not be divisible by a natural number n account for the fact that new congruences have to be introduced during quantifier elimination; moreover the number of test points required is much higher than for linear elimination in the reals.

This result seems to preclude any practical application of quantifier elimination in PA. However, the situation can be improved by exactly one exponential in exchange for a little sacrifice that is comparatively harmless in applications, namely an NP-search problem. The clue for this improvement is to abandon complete quantifier elimination in favour of bounded quantifier elimination, where one admits *bounded quantifiers* of the form $(\exists x, |x| \leq t), (\forall x, |x| \leq t)$, for some constant term t , in the output formulas.

This shift of viewpoint offers an extra advantage that is missing from complete quantifier elimination, namely *uniformity in the coefficients* of PA-terms and PA-formulas: Consider *Uniform Presburger arithmetic* UPA as the extension of PA, where we allow unquantified parameters in the coefficients of the integer variables. Formally this is achieved by introducing scalar terms of a new “scalar” sort that are built up from scalar parameters by certain scalar operations. Then the following results can be shown [Weispfenning 1997a]:

1. Complete quantifier elimination is impossible in UPA; in other words any quantifier elimination procedure for PA is necessarily non-uniform in the coefficients of the input. (This is in sharp contrast to a result of Dries and Holly [1992] that proves quantifier elimination for UPA *without order*.)

2. Bounded quantifier elimination for UPA has upper and lower worst-case complexity bounds that are exactly one exponential lower than those for quantifier elimination in PA.

Efficient bounded quantifier elimination for UPA can be used successfully for mixed integer optimization problems with linear constraints and linear or quadratic objective function, and problems of automatic loop parallelization in programs [Lengauer 1993].

Another extension of quantifier elimination for PA appears in [Weispfenning 1999]. It concerns mixed real-integer linear elimination. Let MRL be the language PA of Presburger arithmetic extended by the unary integer-part operation $[\cdot]$. Interpret this language over the real numbers with congruences being defined by $a \equiv_n b \iff b - a \in n\mathbb{Z}$. Then the elementary theory of reals in MRL admits effective quantifier elimination and hence is decidable. Upper complexity bounds for the quantifier elimination and decision problem are similar to those for Presburger arithmetic. So these results are a common generalization of corresponding results in Section 2.4.2 and 3.8. Further extensions to decision problems involving a transcendental function appear in [Anai and Weispfenning 2000, Weispfenning 2000].

4. Non-linear constraints over continuous domains

4.1. Zero-sets of univariate polynomials

In this section we describe exact symbolic algorithms that determine as closely as possible the structure of the set of common real or complex zeros of a finite set of univariate polynomials with real or complex coefficients, respectively. In particular these algorithms will determine the exact number of zeros and their location to any prescribed accuracy.

Consider a set F of univariate polynomials in $\mathbb{F}[x]$ and a univariate polynomial $G \in \mathbb{F}[x]$. Then $F = 0$ is the system of equations $\{f(x) = 0 \mid f \in F\}$. We denote the *greatest common divisor* of the polynomials in F by $\gcd(F)$. For $g \in \mathbb{F}[x]$ we let g^* denote the *square-free part* of g , i.e. the product of all different irreducible factors of g . We let $V_{\mathbb{K}}(F)$ denote the *variety* of F in some extension field $\mathbb{K}$ of $\mathbb{F}$, i.e. the set of all common zeros of F in $\mathbb{K}$. For a single polynomial g we put $V_{\mathbb{K}}(g) = V_{\mathbb{K}}(\{g\})$. We say the equation $g = 0$ is *implied by* the system $F = 0$ over $\mathbb{K}$ if $V_{\mathbb{K}}(F) \subseteq V_{\mathbb{K}}(g)$.

4.1. PROPOSITION (Solvability and entailment). *Let $\mathbb{K}$ be an extension field of $\mathbb{F}$; let $g = \gcd(F)$ and let g^* be the square-free part of g . Then the following hold:*

1. $V_{\mathbb{K}}(F) = V_{\mathbb{K}}(g) = V_{\mathbb{K}}(g^*)$. So the system $F = 0$ is solvable in $\mathbb{K}$ iff g^* has a zero in $\mathbb{K}$. For an algebraically closed field $\mathbb{K}$, this is the case iff g^* is not in $\mathbb{F}$. Moreover the number of different solutions of this system equals the degree of g^* .
2. Let h be a further univariate polynomial in $\mathbb{F}[x]$. Then the equation $h = 0$ is implied by $F = 0$ if g^* divides h in the polynomial ring $\mathbb{F}[x]$. For an algebraically

closed field $\mathbb{K}$, the converse is also valid.

This is proved as follows: We let $\text{Id}(F)$ denote the *ideal* generated by F , i.e. the set of all linear combinations of polynomials in F using arbitrary polynomials from $\mathbb{F}[x]$ as factors. Then $\text{Id}(F) = \text{Id}(g)$, and so $V_{\mathbb{K}}(F) = V_{\mathbb{K}}(g)$. The equation $V_{\mathbb{K}}(g) = V_{\mathbb{K}}(g^*)$ follows directly from the definition of the square-free part. Since g^* splits into pairwise different linear factors over an algebraically closed field $\mathbb{K}$, the number of different zeros of g^* in $\mathbb{K}$ equals the degree of g^* . If g^* divides h then clearly $V_{\mathbb{K}}(g^*) \subseteq V_{\mathbb{K}}(h)$. If $\mathbb{K}$ is an algebraically closed field and $V_{\mathbb{K}}(g^*) \subseteq V_{\mathbb{K}}(h)$, then every linear factor of g^* in $\mathbb{K}[x]$ must also occur as linear factor in h . Since the linear factors of g^* are pairwise different, this means that g^* divides h .

Notice that for an algebraically closed field $\mathbb{K}$ these criteria reduce the geometric questions whether $V_{\mathbb{K}}(F) = \emptyset$ or whether $V_{\mathbb{K}}(F) \subseteq V_{\mathbb{K}}(g)$ to purely arithmetic questions in the ring $\mathbb{F}[x]$. In order to solve these arithmetic questions algorithmically we use the *Euclidean algorithm for univariate polynomials* as a tool to compute $g = \gcd(F)$ and g^* : The basic step of this algorithm is the reduction of a polynomial f by a polynomial g with $\deg(f) \geq \deg(g) \geq 0$: If f has highest monomial ax^m and g has highest monomial bx^n , then

$$f \xrightarrow{g} f_1 = f - a/bx^{m-n}g \quad (4.1)$$

and $\deg(f_1) < \deg(g)$ or $f_1 = 0$. This simple reduction step can be applied iteratively to a finite non-empty set F of non-zero polynomials in $\mathbb{F}[x]$ as follows: Choose $g \in F$ of minimal degree; replace all other polynomials $f \in F$ by their reducts f_1 w.r.t. g and discard all zero polynomials from the result. Iterate this procedure, until F contains only one polynomial g . Notice that in each iteration the maximal degree of all polynomials in F will decrease; so termination is guaranteed. Moreover, after each iteration the ideal generated by F , $\text{Id}(F)$, remains the same. So at the end we have $\text{Id}(F) = \text{Id}(g)$, which means that g is the greatest common divisor of all polynomials in F .

Next we can use another gcd computation, viz. $g^* = g / \gcd(g, g')$, where g' is the derivative of g , to get the *square-free part* g^* of g . This computation of g^* avoids a complete factorization of g . Its correctness follows from the fact that the multiplicity of an irreducible factor in g reduces by one in $\gcd(g, g')$. Notice that the proposition yields no information on the position of the zeros of g^* in $\mathbb{C} = \mathbb{R}^2$, since this would require inequalities between absolute values; in other words this is in fact a question about real zeros of two bivariate real polynomials representing the real and imaginary part of $g^*(x+iy)$ (comp. Section 4.2). Subsequent factorization of g^* into irreducible polynomials in $\mathbb{F}[x]$ [Mignotte 1992] may split $V_{\mathbb{C}}(g^*)$ into disjoint parts and hence may help actually to solve the equation by numeric methods.

In order to determine the number and position of the real zeros of g^* and hence of F one may proceed as follows: For two given non-zero polynomials $f, g \in \mathbb{R}[x]$ the iterated reduction of f by means of g as defined above will lead to the *remainder* $\text{rem}(f, g)$ of f upon division by g . Using iterated formation of *negative* remainders, one obtains the *Sturm-Sylvester sequence* $S(f, g)$ of (f, g) as follows: Define f_k recursively by $f_0 := f$, $f_1 := g$; $f_{k+1} := -\text{rem}(f_k, f_{k-1})$. Then $S(f, g) = (f_0, f_1, \dots, f_k)$,

where f_k is the last non-zero polynomial in the recursive sequence. For a real number c let $\mathbf{f}(c)$ denote the sequence of real values taken by the polynomials in the sequence $\mathbf{f}$ at the point c . Let $\text{sc}(\mathbf{f}(a))$ denote the number of sign changes in this sequence of real numbers, when zeros are ignored. Then a beautiful theorem due to Sturm and Sylvester [Ben-Or, Kozen and Reif 1986] says:

4.2. THEOREM. *Let f, g be non-zero relatively prime polynomials in $\mathbb{R}[x]$, let f be square-free, and let $a < b$ be real numbers that are non-zeros of f . Let $\mathbf{f}$ be the Sturm-Sylvester sequence of $(f, f' \cdot g)$. Denote the number of zeros of f in the interval $[a, b]$, where g is either positive or negative, by $N(f, g, 1, a, b)$ and $N(f, g, -1, a, b)$ respectively. Then*

$$\text{sc}(\mathbf{f}(a)) - \text{sc}(\mathbf{f}(b)) = N(f, g, 1, a, b) - N(f, g, -1, a, b). \quad (4.2)$$

In particular, for $g = 1$

$$\text{sc}(\mathbf{f}(a)) - \text{sc}(\mathbf{f}(b)) \quad (4.3)$$

is the number of zeros of f in the interval $[a, b]$.

With the obvious sign evaluation of real polynomials at $\pm\infty$ this theorem can also be applied to the case $a = -\infty$ and/or $b = \infty$. The proof employs only the most elementary algebraic facts about real polynomials. By applying the theorem to the pairs f, g and $f, 1$ one easily obtains the separate numbers $N(f, g, 1, a, b), N(f, g, -1, a, b)$. The hypothesis that f is square-free and g is relatively prime can be circumvented by first computing the square-free part f^* of f and $h = f^* / \gcd(f^*, g)$, and then applying the theorem to h in place of f .

Another calculation of $N(f, g, 1, a, b) - N(f, g, -1, a, b)$ due to Hermite has recently found a beautiful generalization to the multivariate case (comp. [Becker and Wörmann 1994, Pedersen, Roy and Szpirglas 1993] and Section 4.2 below): It does not require f to be square-free and g to be relatively prime to f . It asserts that the number $N(f, g, 1, a, b) - N(f, g, -1, a, b)$ is the signature (i.e. the number of positive eigenvalues minus the number of negative eigenvalues) of a symmetric $n \times n$ matrix (a_{ij}) that is computed as follows: $n = \deg(f)$ and a_{ij} is the trace of the linear map $h \mapsto x^{i+j}gh$ on the real vector space $\mathbb{R}[x]/f$ of residues modulo f with respect to the canonical basis $\{1, x, \dots, x^{n-1}\}$.

Moreover, there is a far-reaching generalization of the corollary to the counting of real zeros c of f satisfying several side conditions, say $g_1(c) > 0, \dots, g_m(c) > 0$. Again this generalization is derived in a purely combinatorial way by applying the Sturm-Sylvester theorem to finitely many pairs of polynomials $(f, \tilde{g})$, where $\tilde{g}$ is a product of certain g_i and its squares. The number of these pairs is a priori exponential in the number m of side conditions. This exponential growth can be avoided by a combinatorial divide-and-conquer argument [Ben-Or et al. 1986]. Since over the reals the common zeros of a finite polynomial system $F = \{f_1, \dots, f_r\}$ can be expressed as the zeros of a single polynomial $f = f_1^2 + \dots + f_r^2$, this generalization solves all point counting problems for finite systems of univariate polynomial inequalities.

A priori bounds for the size of the real zeros of a univariate polynomial in terms of its coefficients are given by the *Cauchy bounds* [Mishra 1993]. Other a priori bounds are known e.g. for the distance between roots [Mignotte 1992].

Once a Sturm-Sylvester sequence for (f, f') has been computed for a square-free polynomial f , one can evaluate the number of sign changes of $S(f, f')$ at the endpoints of intervals obtained from the starting interval $[-C, C]$, where C is a rational number greater than a Cauchy bound for f by successive bisection until one has found a finite sequence of pairwise disjoint open intervals with rational endpoints each of which contains exactly one real root of f . Call these the *isolating intervals* for the real roots of f . Using only the endpoints a, b of such an isolating interval and the coefficients of f , one can now evaluate any other polynomial $g \in \mathbb{Q}[x]$ at the unique zero α of f in the open interval $]a, b[$ using again the theorem of Sturm-Sylvester: Assume without restriction that f is square-free and let $\mathbf{h}$ be defined as above. Then

$$g(\alpha) > 0 \ (= 0, < 0) \iff \text{sc}(\mathbf{h}(a)) - \text{sc}(\mathbf{h}(b)) = 1 \ (= 0, = -1). \quad (4.4)$$

This shows that the *isolating interval* $]a, b[$ for α can be used as an exact algebraic code for the algebraic real number α in algebraic computations. Moreover by iterating the bisection of intervals and the evaluation of the number of sign changes of $S(f, f')$ at the endpoints of these intervals, one obtains arbitrarily good numeric approximations of α . This coding of real algebraic numbers is used extensively in the SAC-2 computer algebra system [Collins 1985]. Another possible unique code for a real zero α of f is the sequence of signs of all higher derivatives of f at α (Thom's Lemma [Roy 1996]). The latter has the advantage that it works in arbitrary, not necessarily Archimedean, real-closed fields.

4.2. Zero-sets of multivariate polynomials

In the univariate case of Section 4.1, the computation of the variety of a finite system F of polynomials was reduced to that of a single polynomial by taking a greatest common divisor. While gcd's still can be computed in multivariate polynomial rings R over fields, they do not serve the same purpose for solving equations, since $\text{gcd}(F)$ is no longer a R -linear combination of polynomials in F , because R is not a principal ideal ring. So the role of the greatest common divisor is taken over by the ideal generated by F . The fundamental transition from geometric properties of the variety of F in some extension field $\mathbb{K}$ of $\mathbb{F}$ to arithmetic properties of polynomials in the polynomial ring R over the base field $\mathbb{F}$ is achieved by a "*Nullstellensatz*", i.e. a theorem on zeros for the field $\mathbb{K}$.

We consider the ring $R = \mathbb{F}[x_1, \dots, x_n]$ of multivariate polynomials in the indeterminates $x_1, \dots, x_n$ over $\mathbb{F}$. As in Section 4.1, we let $\text{Id}(F)$ denote the *ideal* generated by F , i.e. the set of all linear combinations of polynomials in F using arbitrary polynomials from R as factors. We let $V_{\mathbb{K}}(F)$ denote the *variety* of F in some extension field $\mathbb{K}$ of $\mathbb{F}$, i.e. the set of all common zeros of F in $\mathbb{K}^n$. For a single polynomial g we put $V_{\mathbb{K}}(g) = V_{\mathbb{K}}(\{g\})$. We say the equation $g = 0$ is *implied*

by the system $F = 0$ over $\mathbb{K}$ if $V_{\mathbb{K}}(F) \subseteq V_{\mathbb{K}}(g)$. To begin with we state *Hilbert's Nullstellensatz* for algebraically closed fields. It applies in particular to the case $\mathbb{F} = \mathbb{K} = \mathbb{C}$.

4.3. THEOREM (Solvability and entailment in algebraically closed fields). *Let $\mathbb{K}$ be an algebraically closed extension field of $\mathbb{F}$, let $R = \mathbb{F}[x_1, \dots, x_n]$, and let $I = \text{Id}(F)$ denote the ideal generated by F in R . Then the following hold:*

1. $V_{\mathbb{K}}(F) = V_{\mathbb{K}}(I)$.
2. The system $F = 0$ is solvable in $\mathbb{K}^n$ iff $1 \notin I$.
3. Let g be a further polynomial in R . Then the equation $g = 0$ is implied by $F = 0$ iff there exists a $k \in \mathbb{N}$ such that $g^k \in I$.
4. Let g be a further polynomial in R , let y be a new indeterminate, and let $S = \mathbb{F}[x_1, \dots, x_n, y]$. Then the equation $g = 0$ is implied by $F = 0$ iff $1 \in \text{Id}(I \cup \{1 - yg\})$, where the ideal is formed in S .

The first statement is obvious. The second is an immediate consequence of the third by taking $g = 1$. The last statement is an immediate consequence of the second; it is known as the *Rabinovich trick*. In the third statement the implication from right to left is again obvious. The converse is a theorem of considerable depth that requires either a considerable amount of ideal theory [Becker et al. 1993] or an elementary but intricate induction on the number of indeterminates [Seidenberg 1956b, Seidenberg 1956a]. The second and the last statement reduce the relevant geometrical questions to the *ideal membership problem* in polynomial rings over $\mathbb{F}$. An algorithmic solution of this problem is offered by the construction of Gröbner bases described below in Section 4.2.2.

In order to classify the varieties $V_{\mathbb{K}}(I)$ for an algebraically closed field $\mathbb{K}$, it is natural to consider the (maximal) geometric dimension occurring in $V_{\mathbb{K}}(I)$. This concept is captured algebraically by the *dimension* of the ideal I : Let $\mathcal{Y} \subseteq \{x_1, \dots, x_n\}$ be a set of indeterminates; then $\mathcal{Y}$ is called *independent* modulo I if $\mathbb{F}[\mathcal{Y}] \cap I = \{0\}$. The dimension of the ideal I is then the cardinality of the largest independent set of indeterminates. Again Gröbner basis theory will provide an algorithmic way of computing the dimension of an ideal (see Section 4.2.2 below).

Hilbert's Nullstellensatz has the following counterpart for real closed fields, which is valid in particular for the field $\mathbb{K}$ of reals. Recall that an ordered field $\mathbb{K}$ is real closed if the intermediate value theorem for polynomials holds in $\mathbb{K}$.

4.4. THEOREM (Solvability and entailment in real closed fields). *Let $\mathbb{K}$ be a real closed extension field of $\mathbb{F}$, let $R = \mathbb{F}[x_1, \dots, x_n]$, and let $I = \text{Id}(F)$ denote the ideal generated by F in R . Then the following hold:*

1. $V_{\mathbb{K}}(F) = V_{\mathbb{K}}(I)$.
2. The system $F = 0$ is solvable in $\mathbb{K}^n$ iff for all $m \in \mathbb{N}$, all positive $p_1, \dots, p_m \in \mathbb{F}$, and all $h_1, \dots, h_m \in R$, $1 + \sum_{i=1}^m p_i h_i^2 \notin I$.
3. Let g be a further polynomial in R . Then the equation $g = 0$ is implied by $F = 0$ iff there exist $k, m \in \mathbb{N}$, positive $p_1, \dots, p_m \in \mathbb{F}$, and $h_1, \dots, h_m \in R$, such that $g^{2k} + \sum_{i=1}^m p_i h_i^2 \in I$.

Here again the first statement, the implication from left to right in statement two, and the implication from right to left in statement three are obvious. Moreover, statement two is the special case of statement three, where $g = 1$. The implication from left to right in statement three is a deep fact of real algebra [Bochnak, Coste and Roy 1987, Weispfenning 1975]. While the real Nullstellensatz is formally analogous to Hilbert's Nullstellensatz, it does not adapt as easily to algorithmic procedures due to the presence of the p_i , and h_i [Lombardi 1991].

Next we study algorithmic approaches to the questions addressed in the theorems. What is required is a replacement of the $\gcd(F)$ in the solution of univariate systems of equations. In fact, the role of the greatest common divisor can be simulated to some extent by certain finite polynomial systems G that have the same R -linear combinations as F , but exhibit the structural properties of the complex or real variety of F more clearly.

There are two essentially different generalizations of the Euclidean algorithm (and hence the $\gcd$ -concept):

- The computation of *characteristic sets* (Ritt-Wu) [Chou 1988, Mishra 1993] and
- the computation of *Gröbner bases* (Buchberger) [Becker et al. 1993, Cox, Little and O'Shea 1992, Cox, Little and O'Shea 1998].

They are based on two different generalizations of the univariate reduction step to the multivariate case corresponding to the recursive and the distributive representation of multivariate polynomials. In the recursive representation, polynomials f over a field $\mathbb{F}$ in indeterminates $x_1, \dots, x_n$ are viewed as objects in $(\dots(\mathbb{F}[x_1])\dots)[x_n]$, whereas in the distributive representation, f is regarded as a $\mathbb{F}$ -linear combination of *terms*, i.e. power-products $x_1^{e_1} \dots x_n^{e_n}$ of the indeterminates. We begin with a sketch of the *characteristic set method*.

4.2.1. Characteristic Sets

In the recursive representation of polynomials one can define a quasi-order on polynomials by putting $f < g$ if g contains an indeterminate with higher index than the highest indeterminate occurring in f , or both have the same highest indeterminate x_i , but g has higher degree in x_i than f . If f is written as univariate polynomial in its highest indeterminate x_j with coefficients in the remaining indeterminates, then the highest coefficient of f is called the *initial* I_f of f . Suppose now g is another polynomial with $m = \deg_{x_j}(g) \geq \deg_{x_j}(f) = k$ and g is written as univariate polynomial in the indeterminate x_j with coefficients in the remaining indeterminates. (We do not require x_j to be the highest indeterminate in g .) Then we have the following (*Ritt*-)reduction of g by the *divisor* f :

$$g \rightarrow_f g^* = I_f g - I_g x_j^{m-k+1} f \quad \text{with} \quad \deg_{x_j}(g) \geq \deg_{x_j}(g^*). \quad (4.5)$$

g is called *reduced w.r.t. f* if no such reduction is possible, i.e. if $\deg_{x_j}(g) < \deg_{x_j}(f)$. Notice that any chain of reductions using the same divisor f will terminate after finitely many steps, since $\deg_{x_j}$ decreases in each step.

An *ascending set* is either a non-zero constant or a finite sequence A of non-constant polynomials with strictly increasing highest indeterminates, such that each polynomial in A is reduced w.r.t. all the previous ones. Ascending sets are quasi-ordered by first differences in their entries from left to right. If no such difference in quasi-order is encountered, the longer sequence is declared smaller than the shorter one. This yields a well-founded quasi-order on ascending sets. As a consequence, every non-empty set F of polynomials in the given polynomial ring contains a minimal ascending set C . Any such C is called a *characteristic set* of F . For finite F such a characteristic set can be determined in an obvious way by inspection.

Let now F be finite and let $C \subseteq F$ be a characteristic set of F . Then each polynomial $f \in F$ can be reduced as long as possible, using the polynomials in C successively in decreasing order, to a polynomial f^* (which may coincide with f if no such reduction is possible). Call f^* the *remainder* of f upon reduction by the ascending set C . Adding all the non-zero remainders f^* of polynomials $f \in F$ obtained in this way to the set F , we obtain a finite set F_1 extending F . If F_1 is strictly larger than F , then F_1 contains a new characteristic set C_1 smaller than C in the quasi-order of ascending sets.

Iterating this procedure we arrive at an increasing chain of polynomial sets $F = F_0 \subset F_1 \subset F_2 \subset \dots$ with an associated strictly decreasing chain of characteristic sets $C = C_0 > C_1 > C_2 > \dots$. Since the quasi-order on characteristic sets is well-founded, this iteration will terminate after finitely many steps, say with F_k and C_k . By construction, every polynomial in F_k is still in the ideal $\text{Id}(F)$, and so $\text{Id}(F_k) = \text{Id}(F)$, and so $V_{\mathbb{K}}(F_k) = V_{\mathbb{K}}(F)$ for $\mathbb{K} = \mathbb{C}$ or $\mathbb{K} = \mathbb{R}$. Moreover, the remainder of every $f \in F_k$ upon reduction by C_k is zero. As a consequence, we have the following relation between the variety of F_k and the variety of C_k : Let J be the product of all the initials of the polynomials in C_k . Then

$$V_{\mathbb{K}}(F) \setminus V_{\mathbb{K}}(J) = V_{\mathbb{K}}(C_k) \setminus V_{\mathbb{K}}(J) \quad \text{for } \mathbb{K} = \mathbb{C} \text{ or } \mathbb{K} = \mathbb{R}. \quad (4.6)$$

Call the characteristic set $C' = C_k$ of $F' = F_k$ obtained in this way an *extended characteristic set* of F . Notice that C' has triangular shape. So $V_{\mathbb{C}}(C')$ is finite iff every x_j is the highest indeterminate of the j -th polynomial in C' . If this is the case, then the individual points in $V_{\mathbb{C}}(C')$ can be computed e.g. numerically by solving successive univariate polynomials. In particular $V_{\mathbb{C}}(C') = \emptyset$ iff C' consists of a constant only. If C' contains less than n polynomials, there will be gaps in the sequence of highest variables of these polynomials, which are related to the dimension of $V_{\mathbb{C}}(C')$. So we have got some structural information on the set $V_{\mathbb{C}}(F) \setminus V_{\mathbb{C}}(J) = V_{\mathbb{C}}(C_k) \setminus V_{\mathbb{C}}(J)$.

In order to determine the remaining part of $V_{\mathbb{C}}(F)$, i.e. $V_{\mathbb{C}}(F) \cap V_{\mathbb{C}}(J)$, we proceed as follows: For each initial I_h of some polynomial h in C' , we consider the set $F_h = F' \cup \{I_h\}$ and compute as before an extended characteristic set C''_h . By construction $C''_h < C'$.

Iterating these steps of adding initials and computing extended characteristic sets will lead to a finitely branching tree of ascending sets. Each branch of the tree is finite, since the ascending sets in this branch are decreasing, and so by König's lemma the whole tree is finite. Let $C_1, \dots, C_m$ be the extended characteristic sets

that form the leaves of this tree. Then for any extension field $\mathbb{K}$ of $\mathbb{F}$ we have a disjoint decomposition

$$V_{\mathbb{K}}(F) = \bigcup_{i=1}^m (V_{\mathbb{K}}(C_i) \setminus V_{\mathbb{K}}(J_i)), \quad (4.7)$$

where the systems C_i are ascending sets (and hence of triangular form) and the J_i are products of all initials of the polynomials in C_i .

This decomposition can then be used for numeric computations of individual elements of $V_{\mathbb{C}}(F)$ and of $V_{\mathbb{R}}(F)$ in case these sets are finite. Alternatively, individual solutions in $\mathbb{C}$ can be computed symbolically in iterated algebraic extension fields of $\mathbb{Q}$. This requires, however, factorization of polynomials over iterated algebraic extension fields and can get quite complex. Individual real solutions can be computed symbolically from the decomposition using their description by isolating rational intervals described in the previous section.

The method has been employed most successfully in order to show for given polynomial f that $V_{\mathbb{C}}(f) \subseteq V_{\mathbb{C}}(F)$ and hence $V_{\mathbb{R}}(f) \subseteq V_{\mathbb{R}}(F)$ by showing that f Ritt-reduces to zero w.r.t. all C_i . This yields automatic proofs of many interesting geometrical theorems [Chou 1988, Mishra 1993].

Next we give a quick sketch of the *Gröbner basis method* and its applications to equation solving [Becker et al. 1993].

4.2.2. Gröbner Bases

The basic idea of the Gröbner basis method is similar to the Ritt-Wu method: namely to modify a given finite system of polynomials F to another finite system G with the property that the varieties of F and G are closely related, and structural properties of the complex and real variety of G can be derived from the shape of G rather directly. In contrast to the construction of characteristic sets, the construction of a Gröbner basis G from F does not yield a decomposition of the variety of F . Instead, the variety remains the same, $V_{\mathbb{K}}(G) = V_{\mathbb{K}}(F)$. In fact even the ideal generated by F is preserved, $\text{Id}(F) = \text{Id}(G)$.

A central concept is again some kind of polynomial reduction, here *Buchberger reduction*. Recall that Ritt-reduction regarded polynomials as univariate polynomials in the highest indeterminate allowing multiplication of the dividend with polynomials in lower variables. For Buchberger reduction we think of polynomials as finite $\mathbb{Q}$ -linear combinations of terms (power-products of the x_i). In order to perform some kind of division with remainder, we should know what we shall regard as the “highest” term in the divisor polynomial.

This is achieved by introducing a *term order*, i.e. a linear order on the set T of terms which has 1 as smallest element and for which multiplication is monotonic. Examples are the *lexicographic* term order (induced by the lexicographic order of the exponent tuples), the *inverse lex* term order (just as the lex order with the order of variables reversed), the *degree-lex* term order (which orders by total-degree first and breaks ties using the lex order on the set T of terms), and the *degree-reverse-lex*

term order (which orders by total-degree first and breaks ties using the lex order backward on the set T of terms). Given a fixed term order, we can speak of the “highest monomial” $\text{HM}(f)$ of a non-zero polynomial f just as in the univariate case. (By convention all monomials will have non-zero coefficients.)

The resulting reduction concept is straightforward:

$$g \xrightarrow{f} g_1 = g - \frac{b}{a}uf, \quad (4.8)$$

if g contains a monomial bt ($b \in \mathbb{K}, t \in T$) and $\text{HM}(f) = as$ and $t = us$ in T . Iterated Buchberger reduction of a polynomial g using divisors f from a given set F is denoted by $g \xrightarrow{F}^* h$. Any chain of reductions is finite. This can be seen as follows: By Dickson’s lemma, divisibility on T is a well-quasi-order; this is just another way of saying that every non-empty subset of $\mathbb{N}^n$ has a finite Hilbert basis (compare Section 3.6). Thus Buchberger reduction is a terminating reduction relation on $\mathbb{K}[x_1, \dots, x_n]$ in the sense of term rewriting. Since any term order refines the divisibility relation on T , term orders are well-orders; any reduction decreases the highest term of the polynomials w.r.t. the chosen term order.

At first glance, Buchberger reduction for the inverse lexicographical term order appears to be very close to Ritt reduction. Notice that Ritt reduction will be applicable much more often than Buchberger reduction since it allows multiplication of the dividend by the initial of the divisor. So it may not be surprising that the addition of all remainders of polynomials in F by iterated Buchberger reduction with divisors in F will in general (except in the univariate case) not yield much more structural information on $\text{Id}(F)$ and hence on $V_{\mathbb{K}}(F)$.

So another process is needed for generating new polynomials in $\text{Id}(F)$. It takes account of cancellations of highest monomials in R -linear combinations of elements of F that are not discovered by the reduction process. Take e.g. the set $F = \{f, g\} \subset \mathbb{Q}[x, y]$ with $f = x^2y + 1$, $g = xy^2 + 1$. Here no Buchberger reduction is possible, but $yf - xg = y - x \in \text{Id}(F)$. For every pair of polynomials $f, g \in \mathbb{F}[x_1, \dots, x_n]$ there is an obvious way of constructing the “minimal” such cancellation, as in the example. Call the resulting “subtraction”-polynomial the *S-polynomial* $\text{spol}(f, g)$ of f and g .

The construction of a Gröbner basis G (w.r.t a fixed term order) from F proceeds by an iteration of adding all non-zero remainders of all S-polynomials w.r.t. reduction using the set of polynomials at hand as possible divisors. Again Dickson’s lemma guarantees that this iteration stops after finitely many steps. The output will be a finite set G with $\text{Id}(G) = \text{Id}(F)$ and the property that all S-polynomials of pairs from G can be reduced to zero using polynomials from G . Such a set G is called a *Gröbner basis* of $\text{Id}(F)$ (w.r.t the given term order). There are many equivalent definitions of Gröbner bases [Becker et al. 1993]; one of them asserts that G is a Gröbner basis iff Buchberger reduction w.r.t. G is confluent in the sense of term rewriting.

In practice the computation of normal forms of S-polynomials w.r.t. the Buchberger reduction is the bottleneck of the construction, since it may involve computations with very large coefficients. So a number of sophisticated criteria and methods

has been developed in order to recognize without lengthy computations superfluous S-polynomials, i.e. S-polynomials whose normal form will not contribute to the construction of the Gröbner basis.

The general strategy of computing Gröbner bases by adding normal forms of S-polynomials may be regarded as a critical-pair (or Knuth-Bendix) completion procedure as defined in term rewriting. Notice, however, that polynomials in the Buchberger algorithm correspond to ground terms, i.e. variable-free terms in term rewriting. For a detailed study of Gröbner bases in the framework of term rewriting see [Le Chenadec 1986, Bündgen 1996, Madlener and Reinert 1998].

In order to simplify a given Gröbner basis G one may interreduce all polynomials in G (by Buchberger reduction) as long as possible. This process terminates as well and yields a *reduced Gröbner basis* of $\text{Id}(F)$. Rather surprisingly reduced Gröbner bases are uniquely determined by $\text{Id}(F)$ and the term order, provided all polynomials are made monic.

What is the information on $V_{\mathbb{C}}(F)$ that can be obtained from a Gröbner basis G of $\text{Id}(F)$ without much further effort?

- A solution of the ideal membership problem: $g \in \text{Id}(F) \iff g$ reduces to zero w.r.t. G .
- A solution of the word problem for the congruence relation generated by $I = \text{Id}(F)$: $g \equiv_I h \iff g - h$ reduces to zero w.r.t. G .
- The decision, whether $V_{\mathbb{C}}(F) = \emptyset$ (check if G contains a non-zero constant).
- The decision, whether $V_{\mathbb{C}}(F)$ is finite (check if for all $1 \leq i \leq n$ some power of x_i occurs in $\text{HM}(g)$ for some $g \in G$). If this is not the case, the computation of the dimension of $V_{\mathbb{C}}(F)$ and of a maximal set of independent variables (inspection of the highest terms in G [Becker et al. 1993, Kredel and Weispfenning 1988]).
- If $V_{\mathbb{C}}(F)$ is finite, explicit computations in the residue class ring $\mathbb{Q}[x_1, \dots, x_n]/\text{Id}(F)$, computation of the radical of $\text{Id}(F)$, computation of the number of complex zeros of F .

For a set F of homogeneous polynomials, $V_{\mathbb{C}}(F)$ is always non-empty, since it contains the origin. Here the modified problem is to determine, whether the set $V_{\mathbb{C}}^*(F)$ of non-trivial common complex zeros of F is non-empty. By the homogeneity of F this is the case iff $V_{\mathbb{C}}(F)$ is infinite. So by the above this holds iff for some $1 \leq i \leq n$ no power of x_i occurs in $\text{HM}(g)$ for any $g \in G$.

All the above works for arbitrary term orders. Other applications such as the computation of elimination ideals [Becker et al. 1993] requires a block-order, where the indeterminates are grouped into two blocks that such that the first block is lexicographically smaller than the second. The complexity of a Gröbner basis computation for a given system F may vary enormously according to the term order chosen. There are some heuristic principles but no definite theoretical results for the choice of a “good” term order. For numerical (or symbolic algebraic) computation of individual solutions, one often requires the reduced Gröbner basis G to be in triangular form; this is guaranteed for the lexicographical term orders. For a detailed study of solutions of algebraic systems in triangular form by symbolic methods, see [Lazard 1991, Lazard 1992].

Moreover, Gröbner bases have numerous other applications in computational commutative algebra [Becker et al. 1993]. At present computations of $V_{\mathbb{C}}(F)$ via Gröbner bases are limited – as a rule of thumb – to systems of say at most 10 variables and of total degree at most 10. This scope can sometimes be extended considerably by combining the Gröbner bases construction with intermediate factorization of polynomials [Melenk, Möller and Neun 1989, Gräbe 1994]. This yields a decomposition of $V_{\mathbb{C}}(F)$ as a finite union of smaller varieties $V_{\mathbb{C}}(G_i)$ given by their respective Gröbner bases G_i .

For zero-dimensional systems F (i.e. systems with finite complex variety) the computation of an *involution basis* provides sometimes a faster alternative to the Buchberger algorithm for the computation of a Gröbner basis [Zharkov and Blinkov 1993, Apel 1995, Apel 1998]. The method is an adaptation of ideas developed by Janet for non-commutative rings of differential operators to the case of commutative polynomial rings.

In case $V_{\mathbb{C}}(F)$ is finite, the real zero set $V_{\mathbb{R}}(F)$ can be computed e.g. as follows, compare [Becker et al. 1993, Section 8.8]: One introduces a new variable Z as random linear combination ℓ of $x_1, \dots, x_n$ such that the variety $V_{\mathbb{C}}(F \cup \{\ell\})$ is in normal position with respect to Z , i.e. different zeros of this system have different Z -components. This fact is checked by a Gröbner basis computation. Then Z satisfies a univariate polynomial equation $f(Z) = 0$ over $\mathbb{Q}$ and all x_i are given as polynomial functions of Z . Hence the real zeros of F can now be determined from the real zeros of the univariate polynomial f .

An alternative method dealing directly with a Gröbner basis of a real zero dimensional ideal is based on a *multivariate version of the Sturm-Sylvester theorem* discovered recently [Becker and Wörmann 1994, Pedersen et al. 1993]. It uses the Gröbner basis to compute the signature of a quadratic form on the residue ring defined by I , similar to the univariate case described in the previous section.

4.3. Parametric zero-sets

In many applications of constraint solving one wants to study not only the zero sets of fixed polynomial systems, but also a whole class of such problems given by a polynomial system $F = 0$, where the coefficients of the polynomials in the system depend on (complex or real) parameters. Then the complex or real zero-set of F depends on these parameters and the goal is to describe this dependence on the parameters as uniformly as possible.

Another source of parametric zero-sets are polynomial systems F that are parameter-free but where $V_{\mathbb{C}}(F)$ has positive dimension and one wants to determine the zeros of F depending on a set $\mathcal{Y} \subseteq \{x_1 \dots x_n\}$ of independent variables acting as parameters. This situation may be subsumed under the former case by regarding the variables in $\mathcal{Y}$ as parameters.

So we consider a finite set $F \subseteq \mathbb{Q}[U_1, \dots, U_m][x_1, \dots, x_n]$, and we want to determine $V_{\mathbb{C}}(F_{\mathbf{a}})$ and $V_{\mathbb{R}}(F_{\mathbf{a}})$ depending on arbitrary complex or real values $\mathbf{a} = (a_1, \dots, a_m)$ of the parameters $\mathbf{U} = (U_1, \dots, U_m)$. Here $F_{\mathbf{a}} = \{f(\mathbf{a}, \mathbf{x}) \mid$

$f(\mathbf{U}, \mathbf{x}) \in F\}$. What is the answer we expect for questions like $|V_{\mathbb{C}}(F_{\mathbf{a}})| = d$ or $|V_{\mathbb{R}}(F_{\mathbf{a}})| = d$ or $\dim V_{\mathbb{C}}(F_{\mathbf{a}}) = d$ for a natural number d , for arbitrary values of $\mathbf{a}$? Obviously it should consist of conditions on the parameters $\mathbf{U}$ that are easy to evaluate for a given value $\mathbf{a}$ of $\mathbf{U}$. Typically such a condition is a Boolean combination of polynomial equations in $\mathbf{U}$ in the complex case, and of a Boolean combination of polynomial inequalities in the real case. Such a condition will then constitute a *uniform solution* to the parametric problem. It is well-known from algebraic model theory that conditions of this form do indeed exist. In the following, we sketch some methods that can be useful for computing such conditions in practice.

The basic idea is to use the methods applied in the non-parametric cases in such a way as to split the computation into a tree depending on case distinctions concerning the vanishing (or the signs in the real case) of certain coefficients that enter the computation step in an essential way. In principle it suffices to treat systems of parametric univariate polynomial equations and disequations (equations and inequalities in the real case), since the case of several main variables can then be handled by induction. Recall from the univariate case in Section 4.1 that the Euclidean algorithm and its variants (Sturm-Sylvester sequences) form the centerpiece of the non-parametric univariate algorithms. Unfortunately in the parametric case, the Euclidean algorithm will require an enormous number of case distinctions on the possible vanishing of highest coefficients of polynomials used as divisors. So in general it is not advisable for complexity reasons to follow this route, see however the next section for a counterexample. Various replacements for the Euclidean algorithm and Sturm-Sylvester sequences have been considered in the literature that behave better in the parametric case. Important concepts arising in this problem area are various types of *resultants*, *subresultants* and for real zeros *Sturm-Habicht sequences* [González, Lombardi, Recio and Roy 1989, González, Lombardi, Recio and Roy 1990, González-Vega, Recio, Lombardi and Roy 1998, González-Vega 1998, Renegar 1991, Kapur and Lakshman 1992, Cox et al. 1998].

For multivariate parametric systems, the method for computing an extended characteristic set is, by its inherent case distinctions, well fitted for parameters and does not require essential changes from the non-parametric case [Gao and Chou 1991, Kapur 1995a]. But again the number of case distinctions may be prohibitive in practice. For parametric systems of linear equations, a symbolic solution method has been presented in [Sit 1991].

For the rest of this section we consider the use of *Gröbner bases in the parametric case* and of various types of *resultants*.

4.3.1. Comprehensive Gröbner Bases

The basic observation here is that it is not sufficient to compute a Gröbner basis G of F with respect to the variables $\mathbf{x}$ and indeterminate parameters $\mathbf{U}$, since under specialization of $\mathbf{U}$ to some value $\mathbf{a}$, $G_{\mathbf{a}}$ may no longer be a Gröbner basis. This problem is remedied by the more subtle construction of a *comprehensive Gröbner basis* for F that does in fact remain stable under specialization of parameters [Weispfenning 1992]. The idea of the construction is to perform a tree of

Gröbner basis computations w.r.t. the necessary case distinctions on the parameters, but to keep all the coefficients in the computation that have been asserted to vanish in some case. In this manner all the Gröbner bases obtained at the leaves of the tree stay inside the given input ideal; so the whole case distinction can be discarded and the different Gröbner bases at the leaves can be joint into one single polynomial system which turns out to be a comprehensive Gröbner basis of F . Alternatively, one may attach to each node in the tree the corresponding condition on the parameters that have led to this node and drop coefficients that are zero as a consequence of this condition. The result is then a (modified) *Gröbner system* in the sense of [Weispfenning 1992].

A related approach appears in [Kapur 1995a]. Here conditions on the parameters are attached to individual polynomials, resulting in *constrained polynomials*. Parametric Gröbner bases are then finite sets of constrained polynomials. Using this approach the characteristic set construction can also be modified in such a way that it remains stable under specializations [Kapur 1995a].

Once a comprehensive Gröbner basis G has been computed from F , the structure of the variety $V_{\mathbb{C}}(F_{\mathbf{a}})$ can easily and uniformly be obtained from G depending on the complex values $\mathbf{a}$ of the parameters $\mathbf{U}$. In particular there are Boolean combinations of polynomial equations in the parameters that yield necessary and sufficient conditions for the following properties: $V_{\mathbb{C}}(F_{\mathbf{a}}) = \emptyset$, $|V_{\mathbb{C}}(F_{\mathbf{a}})| = d$, $\dim(V_{\mathbb{C}}(F_{\mathbf{a}})) = d$ for a given natural number d .

The corresponding problem concerning the structure of $V_{\mathbb{R}}(F_{\mathbf{a}})$ is much more subtle. Due to the presence of parameters, it is in practice impossible to reduce the problem to the univariate case as in the non-parametric case, since this would involve too many different Gröbner basis calculations.

Progress has, however, been made recently in an important special case, viz. when $V_{\mathbb{C}}(F_{\mathbf{a}})$ is finite for all relevant values of $\mathbf{a}$. In this case the construction of comprehensive Gröbner bases can be combined with the powerful multivariate Sturm theorem mentioned in Section 4.1 [Becker and Wörmann 1994, Pedersen et al. 1993] to obtain the structure of $V_{\mathbb{R}}(F_{\mathbf{a}})$ depending on the relevant values of $\mathbf{a}$ [Weispfenning 1998]. One proceeds as follows: Using a comprehensive Gröbner basis of $\text{Id}(F)$, one computes $\mathbb{Q}$ -linear bases and structure constants for the finitely many residue class rings $\mathbb{Q}[x_1, \dots, x_n]/(F_{\mathbf{a}})$ that may occur for all values of the parameters. In each case one computes a certain quadratic form on the respective $\mathbb{Q}$ -linear space. Then the number of real zeros of F in each case is determined by the signature of the corresponding quadratic form in much the same way as in the univariate case described in Section 4.1. For any fixed $k \in \mathbb{N}$ one can write down a Boolean combination of polynomial inequalities in the parameters $\mathbf{U}$ expressing the fact that this signature equals k . In this way one gets conditions on the parameters that express the fact that $|V_{\mathbb{R}}(F_{\mathbf{a}})| = d$. Unfortunately these conditions tend to become very long for large dimensions of the $\mathbb{Q}$ -linear space; so applications so far are limited. Again as in the non-parametric case, side conditions can in principle be incorporated into the method [Dolzmann 1994].

4.3.2. Resultants

Consider a finite set $F \subseteq \mathbb{Q}[U_1, \dots, U_m][x_1, \dots, x_n]$ of parametric polynomials in $x_1, \dots, x_n$. We have seen above that there is a Boolean combination of polynomial equations in the parameters that are necessary and sufficient conditions for the property $V_C(F_a) \neq \emptyset$. In general, such a Boolean combination cannot be replaced by a conjunction of polynomial equations in the parameters, as the simple example of the polynomial $f(U, x) = Ux - 1$ shows: Here $V_C(f(a, x)) \neq \emptyset$ iff $a \neq 0$. In the special case of testing a system of homogeneous parametric polynomials for a common non-trivial zero, there is indeed a test consisting of a conjunction of polynomial equations in the parameters. The polynomials appearing in such a conjunction are known as a *resultant system* for F [van der Waerden 1940, Section 78].

For an inhomogeneous set of parametric polynomials

$$F \subseteq \mathbb{F}[U_1, \dots, U_m][x_1, \dots, x_n]$$

one may pass to the homogenized system $F^* \subseteq \mathbb{F}[U_1, \dots, U_m][x_0, x_1, \dots, x_n]$ by replacing each $f \in F$ by $f^* = x_0^{\deg(f)} f(x_1/x_0, \dots, x_n/x_0)$. Then a resultant system for F^* constitutes a necessary, but in general not a sufficient criterion for $V_C(F_a) \neq \emptyset$.

A special situation occurs if we take F as the most general set of n homogeneous polynomials $f_1, \dots, f_n$ in n variables $x_1, \dots, x_n$ of fixed degrees $d_1, \dots, d_n$. Then the parameters are the indeterminate coefficients of these polynomials. We call such a system F *generic*. For the case of a generic system F , there exists a resultant system for F consisting of a single polynomial R in the parameters. If R is taken of minimal degree, then it is uniquely determined up to a constant factor. This polynomial R is called the *resultant* of F [van der Waerden 1940, Section 78] and [Cox et al. 1998]), and is denoted by $\text{Res}_{d_1, \dots, d_n}(f_1, \dots, f_n)$. In the special case, where $n = 2$ and F consists of two homogeneous generic polynomials f_1, f_2 of degree d_1, d_2 , respectively, then $\text{Res}_{d_1, d_2}(f_1, f_2)$ is the well-known *Sylvester resultant*, given by the determinant of the Sylvester-matrix of the coefficients of f_1, f_2 . This resultant is also known as the Sylvester-resultant of the univariate polynomials $f_1(x, 1), f_2(x, 1)$ obtained from f_1, f_2 by dehomogenization.

Due to the very large number of coefficients in a generic system $F = \{f_1, \dots, f_n\}$, the computation of $\text{Res}_{d_1, \dots, d_n}(f_1, \dots, f_n)$ is a major problem of elimination theory. Macauley has given two representations of the resultant, one as a gcd of certain determinants and another as a quotient of two determinants, both having coefficients from F as entries. The first is impractical for computations, and the second approach may fail if the denominator vanishes; moreover for non-generic systems extraneous factors may occur in the second approach [Kapur 1995b]. For ways to circumvent these difficulties see [Canny and Manocha 1993].

Applications of the resultant (for inhomogeneous systems) include the computation of all complex zeros of systems of n polynomials in n variables with finitely many complex zeros, either via the formation of a *u-resultant* or by hiding one variables in the ground field [van der Waerden 1940, Cox et al. 1998].

The complexity of the resultant that is due to dealing with dense polynomials of given degrees is circumvented by *sparse resultants* and more generally *mixed sparse*

resultants [Canny and Emiris 1993, Cox et al. 1998]. Here one restricts attention to sparse multivariate polynomials with specified sets of admissible exponent vectors in the monomials. The polynomials may be inhomogeneous. Call an n -tuple c of complex numbers strongly non-trivial, if all components of c are non-zero. The vanishing of the mixed sparse resultants of a finite (Laurent) polynomial system F with respect to fixed sets of exponent vectors in $\mathbb{Z}^n$ is then a necessary condition for F to have a common strongly non-trivial zero. It is also sufficient for F to have a common zero in a certain (possibly larger) toric variety V that can be computed from the sets of exponent vectors. A possible way of computing the mixed sparse resultant is as the gcd of certain determinants associated with mixed volumes of tuples of polytopes [Canny and Emiris 1993, Emiris and Canny 1995].

The resultants for homogeneous systems considered above are a special case of sparse resultants, where the set of exponents vectors is given by a bound on the sum of the entries. Another special case are *Dixon resultants*, where the set of exponents vectors is given by individual bounds for each entry. They have been studied in great detail by Kapur and his group [Kapur, Saxena and Yang 1994, Kapur 1995b, Kapur 1998].

4.4. Complex and real quantifier elimination

The first-order theory T_F of fields is expressed in the language L_F of fields having 0, 1 as constants, $+$, $-$, $\cdot$ as binary operations, and equality $=$ as binary relation. For the first-order theory T_{OF} of ordered fields the corresponding language L_{OF} of ordered fields has in addition order $<$ as a binary relation. The axioms for these theories are well-known. So *terms* in L_F or L_{OF} can be written in the form of polynomials with integer coefficients. Atomic formulas in L_F are equations between such terms; in L_{OF} atomic formulas include also inequalities. Formally, integer coefficients of variables and the constant 1 are written in unary notation.

The language L_F is rich enough to express most basic facts in algebraic geometry about zero-sets of polynomial systems. Even more expressive and important for application problems is the language L_{OF} . The sets definable in $\mathbb{R}$ by a L_{OF} -formula are called *semi-algebraic sets*. Among the application areas of semi-algebraic sets are: Geometric theorem proving, computer-aided design and solid modeling, robot kinematics and motion planning, non-linear optimization problems, simulation of technical networks [Chou 1988, Sturm 1997, Sturm and Weispfenning 1997, Sturm and Weispfenning 1998, Weispfenning 1994, Weispfenning 1997c, Dolzmann, Sturm and Weispfenning 1998a, Dolzmann, Sturm and Weispfenning 1998b]. For these applications quantifier elimination is a very powerful tool.

The existence of effective quantifier elimination procedures both for the theory of complex numbers in L_F and the theory of real numbers in L_{OF} is due to Tarski [1948]. An analysis of the properties required in the course of these quantifier elimination procedures provides easily explicit first-order axioms for the theory of $\mathbb{C}$ and of $\mathbb{R}$. The theory of $\mathbb{C}$ can be axiomatized by first-order axioms for algebraically closed fields of characteristic zero [Tarski 1948]; in fact quantifier elimination holds

for algebraically closed fields of arbitrary characteristic. The theory of $\mathbb{R}$ can be axiomatized by first-order axioms for real closed fields [Tarski 1948].

For applications, the complexity of real or complex quantifier elimination is of decisive importance. Unfortunately, the asymptotical worst-case complexity is doubly exponential [Weispfenning 1988, Davenport and Heintz 1988], compare Section 2.4.2. For practical applications, the situation is not quite as bad. Here we encounter as a rule only formulas with a fixed number of quantifier blocks in its prefix. Under this restriction the lower bound of the worst-case complexity of real or complex quantifier elimination is singly exponential.

Matching upper bounds for the complexity of real quantifier elimination have been proved in [Renegar 1992a, Renegar 1992b, Renegar 1992c], and in a more refined version in [Basu, Pollack and Roy 1998], compare also [Grigoriev 1988, Heintz, Roy and Solernó 1990] for the complexity of the real decision problem. Implementation of these asymptotically optimal algorithms is still at a very early stage [Roy 1996, Rouillier 1996, González-Vega, Roy, Rouillier and Trujillo 1998].

In the following we list some quantifier elimination methods that have been implemented and proven to be useful in applications. They may be specialized or theoretically less efficient in the worst case. For complex elimination a classical method by A. Seidenberg [Seidenberg 1956a] has been refined and implemented in [Wang 1993b, Wang 1993a, Wang 1995]. Another method [Weispfenning 1992, Weispfenning 1998] is based on the construction of *comprehensive Gröbner bases*. As explained in Section 4.3, these bases provide a test for the solvability of a conjunction of polynomial equations with parametric coefficients in $\mathbb{C}$. This test consists in a Boolean combination of polynomial equations in the parameters, in other words in a quantifier-free formula in these parameters. So we have a complex quantifier elimination for existential formulas of the form

$$\exists x_1 \dots \exists x_n (\bigwedge f_i(x, u) = 0), \quad (4.9)$$

where $u = u_1 \dots u_m$ is an m -tuple of parameters.

Quantifier elimination for an arbitrary existential formula is reduced to this case by formation of a disjunctive normal form and the elimination of negated polynomial equations by the Rabinovich trick (compare Section 1.3):

$$f(x_1 \dots x_n) \neq 0 \iff \exists y(1 - yf(x_1 \dots x_n) = 0) \quad (4.10)$$

This method is implemented in the computer algebra systems MAS [Kredel 1993, Pesch 1994] and REDUCE by A. Dolzmann, W. Neun and T. Sturm (see <http://www.fmi.uni-passau.de/~reduce/cgb/>). An analogous approach for real quantifier elimination is based on parametric multivariate real root counting as described in the previous section [Weispfenning 1998]. It is much more complex and is also implemented in MAS [Lippold 1993, Dolzmann 1994].

The oldest and most comprehensive implemented real quantifier elimination method is quantifier elimination by *partial cylindrical algebraic decomposition* [Collins 1975, Collins 1998, Collins and Hong 1991]. It is based on a cylindrical

decomposition of the variable space of the given formula into finitely many connected semi-algebraic cells in such a way that all polynomials in the input formula are sign invariant on each cell. It includes special strategies for formulas of special types [Hong 1998, Brown 1998].

A special real quantifier elimination system for input formulas with polynomials of low degree is implemented in the REDLOG-package of REDUCE [Dolzmann and Sturm 1996, Dolzmann and Sturm 1997, Dolzmann, Sturm and Weispfenning 1998b]. It is based on the virtual substitution of test points described in Sections 2.2.6 and 4.7.1. This approach is particularly successful for input formulas with many variables and at most quadratic polynomials [Weispfenning 1997b, Weispfenning 1997c, Dolzmann, Sturm and Weispfenning 1998a, Dolzmann, Sturm and Weispfenning 1998b].

Another special real quantifier elimination system for input formula involving only a single quantifier and a single polynomial is described in [González-Vega 1998]; it is based on parametric Sturm-Habicht sequences.

4.5. Further Topics

4.5.1. Quadratic systems of inequalities and quadratic programming.

Quadratic programming is concerned with the minimization of a quadratic objective function $q(x) = x^T Hx + c^T x$ subject to a system $Ax \geq b$ of linear constraints. The method of virtual substitution of test points described in Section 2.2.6 can be extended to Boolean combinations of quadratic inequalities [Weispfenning 1997b]. Here successive elimination of variables by virtual substitution of test points may lead to an increase of degrees beyond two. This situation can sometimes be overcome by implicit factorization of polynomials [Dolzmann, Sturm and Weispfenning 1998a]; otherwise it can lead to an incomplete quantifier elimination. If, however, the constraints except for one quadratic constraint are all linear, this feature will persist throughout successive elimination of variables by virtual substitution of test points. The reason is the following: The test points used for the elimination of a variable x all come from the linear constraints or from a zero of the partial derivative of the objective function with respect x . Hence they are linear polynomials in the remaining variables. Analogous remarks apply to *hyperbolic programming*, i.e. the minimization of a rational linear objective function with respect to linear constraints. The method can also be applied to the corresponding parametric problems.

Under the additional hypothesis that the matrix H is positive definite, the methods of convex programming can be applied yielding Karush–Kuhn–Tucker conditions that lead to a solution by an extension of the Simplex algorithm for linear programming [Peressini, Sullivan and Uhl 1988].

Quadratic programming can be viewed as a decision problem, namely the solvability of a system $Ax \geq b$ of linear constraints together with a quadratic constraint $q(x) \leq c$ derived from the objective function $q(x)$ and a parameter c . As such it is NP-complete [Vavasis 1990]. Indeed, the Boolean satisfiability problem SAT

is easily coded as a quadratic programming problem. On the other hand, as the elimination argument above suggests, satisfiability of the quadratic programming problem can be decided by testing the solvability of certain systems of linear equations derived from the linear constraints and the partial derivatives of $q(x)$. Since the size of the coefficients of these systems is polynomial in the data, this shows that quadratic programming is in NP.

Quadratic constraints in constraint logic programming have been studied in [Pesant and Boyer 1994, Pesant 1995].

4.5.2. Semidefinite programming.

Semidefinite programming is an extension of linear programming where the componentwise inequalities between vectors are replaced by matrix inequalities, or, equivalently the first orthant is replaced by the cone of positive semi-definite matrices. A *semidefinite optimization problem* has the form

$$\min\{c^T x \mid F(x) \succeq 0, x \in \mathbb{R}^n\}, \text{ with } F(x) = F_0 + \sum_{j=1}^n x_j F_j. \quad (4.11)$$

Here $F_0, F_1, \dots, F_n$ are symmetric matrices in $\mathbb{R}^{m \times m}$ and $F(x) \succeq 0$ means that $F(x)$ is positive semidefinite. Such a constraint is nonlinear, but convex; therefore semidefinite programs are convex optimization problems. Semidefinite programming unifies several standard problems, e.g. linear programming and convex quadratically constrained quadratic programming. Although semidefinite programs are much more general than linear programs, they are not much harder to solve. Most interior-point methods for linear programming have been generalized to semidefinite programming. Like in linear programming, these methods have worst-case polynomial complexity and perform well in practice, see [Vandenberghe and Boyd 1996] for an overview.

5. Non-linear diophantine constraints

5.1. Hilbert's Tenth Problem

Non-linear diophantine equations have a long tradition in mathematics, which can be traced back at least to the Greek mathematician Diophantus of Alexandria. Pierre de Fermat, when reading Diophantus' book *Arithmetica*, founded the study of what we call now Diophantine equations. A *diophantine equation* is an equation between polynomials in several variables which has to be solved over the integer numbers. The most famous example is Fermat's equation

$$x^n + y^n = z^n. \quad (5.1)$$

Fermat claimed that for $n \geq 3$ this equation has only the trivial solutions $(a, 0, a)$ and $(0, a, a)$, where a is an arbitrary integer. Only in 1994, this was finally proven by Wiles [1995] and Taylor and Wiles [1995].

In 1900 at the International Congress of Mathematicians in Paris, David Hilbert presented twenty-three challenge problems for the upcoming century. *Hilbert's Tenth Problem* [Matiyasevich 1993] was to find a general method to determine the solvability of a Diophantine equation:

Given a diophantine equation with any number of unknown quantities and with rational integral numerical coefficients: to devise a process according to which it can be determined by a finite number of operations whether the equation is solvable in rational integers.

Only in 1970, this problem was finally proven to be undecidable by Matiyasevich. Let us call a set $D \subseteq \mathbb{Z}^n$ *diophantine* if there exists a polynomial $p \in \mathbb{Z}[x_1, \dots, x_n, y_1, \dots, y_m]$ such that

$$D = \{(a_1, \dots, a_n) \in \mathbb{Z}^n \mid \exists (b_1, \dots, b_m) \in \mathbb{Z}^m \\ p(a_1, \dots, a_n, b_1, \dots, b_m) = 0\}. \quad (5.2)$$

Based on previous work on exponential diophantine equations by Davis, Putnam and Robinson [1961], Matiyasevich showed that every recursively enumerable set $S \subseteq \mathbb{Z}^n$ is diophantine. Since there exist recursively enumerable sets that are not recursive, this implies:

5.1. THEOREM (Matiyasevich 70). *Hilbert's Tenth Problem is undecidable.*

Note that the intersection and union of two diophantine sets is diophantine, since $p = 0 \wedge q = 0$ is equivalent to $p^2 + q^2 = 0$. Moreover, $p = 0 \vee q = 0$ is equivalent to $p \cdot q = 0$. The set of non-negative integers is diophantine by Lagrange's theorem which states that an integer is non-negative if and only if it is the sum of four integer squares. Finally, the set of nonzero integers is diophantine because an integer x is nonzero if and only if there exists a non-negative integer y such that $x = y + 1$ or $x = -y - 1$. This implies that the solvability of systems of diophantine equations, inequalities, and disequalities is equivalent to the solvability of a single diophantine equation.

Matiyasevich's theorem is a very strong result. For example, it implies that there exists a polynomial $p \in \mathbb{Z}[x, y_1, \dots, y_m]$ such that $a \in \mathbb{Z}$ is a prime number iff the diophantine equation $p(a, y_1, \dots, y_m) = 0$ has an integer solution.

Various results have been obtained to characterize the boundary between decidable and undecidable problems in terms of the degree of the polynomial and the number of variables. These can be summarized as follows [Davis, Matiyasevich and Robinson 1976, Tung 1987, Matiyasevich 1993, Mazur 1994, Pheidas 1994]:

- Hilbert's Tenth Problem over $\mathbb{N}$ for a single quadratic diophantine equation in arbitrarily many variables is decidable [Siegel 1972, Grunewald and Segal 1981]. It is undecidable for systems of quadratic equations.
- Hilbert's Tenth Problem over $\mathbb{N}$ for cubic equations is open.
- Hilbert's Tenth Problem over $\mathbb{N}$ for diophantine equations of degree 4 is undecidable.
- Hilbert's Tenth Problem over $\mathbb{N}$ or $\mathbb{Z}$ for systems of equations in one variable is decidable.

- Hilbert's Tenth Problem over $\mathbb{N}$ for equations in 2 to 8 variables is open.
- Hilbert's Tenth Problem over $\mathbb{N}$ for equations in 9 or more variables is undecidable.
- Hilbert's Tenth Problem over $\mathbb{Z}$ for equations in 27 or more variables is undecidable.
- Hilbert's Tenth Problem for solving diophantine equations over the *rational* numbers is a major open problem in the area.

Several authors applied Hilbert's Tenth problem to prove the undecidability of unification problems, see e.g. [Goldfarb 1981, Bockmayr 1987, Siekmann and Szabó 1989].

5.2. Positive results

5.2.1. Effective solution of diophantine equations in two variables

Even in decidable cases, the effective resolution of non-linear diophantine equations is very hard [Smart 1998].

Manders and Adleman [1978] showed that deciding the solvability of diophantine equations of the form

$$ax^2 + by = c, \quad a, b, c \in \mathbb{N} \quad (5.3)$$

over the natural numbers is NP-complete.

In a fundamental contribution, Baker [1968] developed a non-deterministic exponential-time algorithm for deciding the solvability of the *Thue equation*

$$f(x, y) = m, \quad (5.4)$$

where $f(x, y)$ is a homogeneous polynomial of degree ≥ 3 irreducible over the rationals. Baker's method extends to various other equations in two unknowns. The first systematic algorithm for solving Thue's equation was developed by Tzanakis and Weger [1989]. Recent work on Thue's equation includes [Bilu and Hanrot 1996, Bilu and Hanrot 1999], who studied also *superelliptic* diophantine equations of the form

$$ay^p = f(x), \quad (5.5)$$

with $p \geq 3$, a a positive integer, and $f(x) \in \mathbb{Z}[x]$ a separable polynomial of degree $n \geq 2$ [Bilu and Hanrot 1998].

5.2.2. Non-linear integer programming over convex sets

Porkolab and Khachiyan [1997] study integral points in convex subsets of $\mathbb{R}^n$ that are defined by polynomial equations and inequalities. They derive an upper bound for the size of some integral solution - if there is one - and show how this yields a polynomial-time algorithm for these systems in fixed dimension. This result generalizes Lenstra's theorem on the polynomial-time solvability of integer linear programming in fixed dimension.

5.2.3. Non-linear 0-1 programming

Non-linear constraints in 0-1 variables have been studied in the optimization literature. There are four major approaches: linearization, algebraic methods, enumerative methods, and cutting planes, see [Hansen, Jaumard and Mathon 1993] for an overview.

5.3. Decision problems for first-order theories of the integers

The algorithmic unsolvability of Hilbert's Tenth Problem constitutes an enormous strengthening of Gödel's undecidability result [Gödel 1931]. Let T denote the first-order theory of the integers in the language L of rings having 0, 1 as constants, $+$, $-$, $\cdot$ as binary operations, and equality as binary relation. Then Gödel's undecidability theorem says (among other things) that T is undecidable, i.e. there is no algorithm that decides upon input of a closed L -formula φ , whether or not φ holds in $\mathbb{Z}$. The algorithmic unsolvability of Hilbert's Tenth Problem implies that even the positive existential theory of the integers in L is undecidable: Call an L -formula positive existential if it is existential and contains no negation. Then there is no algorithm that decides upon input of a closed positive existential L -formula φ , whether or not φ holds in $\mathbb{Z}$.

In fact this result holds even if the inputs are restricted to closed positive existential L -formulas in which all terms are at most quadratic polynomials. To see this, one observes that by introducing new existentially quantified variables the degree of terms can be reduced to two or less. Consider e.g. the equation $x^2yz - 1 = 0$. Then any integer solution of this equation can be expanded to an integer solution of the system $uv - 1 = 0$, $u = yz$, $v = x^2$. By way of contrast, a restriction to linear polynomials (even when order is included in the language) leads to a decidable theory, see Section 3.8. A few restricted non-linear fragments of T turn out to be decidable [Weispfenning 1990a]. On the other hand, the linear theory of the integers with the divisibility relation is undecidable [Lipshitz 1978], since multiplication of integers is definable in this theory as follows: $x \cdot y = z \iff 2z = (x+y)^2 - x^2 - y^2$ and $y = x^2 \iff y + x = \text{lcm}(x, x+1)$. The main result of [Lipshitz 1978] shows, however, that the positive existential fragment of this theory is again decidable.

Acknowledgments

We are grateful to the second reader Michael Maher whose detailed comments helped a lot to improve this chapter. Thanks also to Eric Domenjoud, Fritz Eisenbrand, Guillaume Hanrot, Thomas Kasper, Claude Kirchner, and Nicolai Pissaruk for various useful remarks.

Bibliography

- AARDAL K., HURKENS C. AND LENSTRA A. K. [1998a], Solving a linear diophantine equation with lower and upper bounds on the variables, in 'Integer Programming and Combinatorial Optimization, IPCO'98', Springer, LNCS 1412, pp. 229 – 242.
- AARDAL K., HURKENS C. AND LENSTRA A. K. [1998b], Solving a system of diophantine equations with lower and upper bounds on the variables, Technical Report UU-CS-1998-36, Department of Computer Science, Utrecht University.
- AARDAL K., WEISMANTEL R. AND WOLSEY L. A. [1999], Non-standard approaches to integer programming, Technical Report UU-CS-1999-41, Department of Computer Science, Utrecht University.
- ABDULRAB H. AND MAKSIMENKO M. [1995], General solution of systems of linear diophantine equations and inequations, in 'Rewriting Techniques and Applications, RTA'95, Kaiserslautern', Springer, LNCS 914, pp. 339 – 351.
- ABDULRAB H. AND PÉCUCHE J.-P. [1989], Solving systems of linear diophantine equations and word equations, in 'Rewriting Techniques and Applications, RTA'89, Chapel Hill', Springer, LNCS 355, pp. 530 – 532.
- ADI M. AND KIRCHNER C. [1992], 'AC-Unification Race: The System Solving Approach, Implementation and Benchmarks', *J. Symbolic Computation* **14**(1), 51–70.
- AJILI F. [1994], Etude de la résolution de contraintes diophantiennes linéaires sur les entiers naturels, Mémoire de DEA, Univ. Henri Poincaré, Nancy, France.
- AJILI F. [1998], Contraintes diophantiennes linéaires: résolution et coopération inter-résolveurs, PhD thesis, Univ. Henri Poincaré, Nancy, France.
- AJILI F. AND CONTEJEAN E. [1997], 'Avoiding slack variables in the solving of linear diophantine equations and inequations', *Theoretical Computer Science* **173**, 183 – 208.
- AJILI F. AND LOCK H. C. R. [1998], Integrating constraint propagation in complete solving of linear diophantine systems, in 'Principles of Declarative Programming, ALP/PLILP'98, Pisa', Springer, LNCS 1490, pp. 463 – 480.
- AJTAI M. [1998], The shortest vector problem in l_2 is NP-hard for randomized reduction, in '30th ACM Symposium on the Theory of Computing, STOC'98, Dallas', pp. 10 – 19.
- ANAI H. AND WEISPFENNING V. [2000], Deciding linear-trigonometric problems, in C. Traverso, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC 2000', ACM-Press, pp. 14–22.
- APEL J. [1995], 'A Gröbner approach to involutive bases', *J. Symb. Comp.* **19**, 441–457.
- APEL J. [1998], 'The theory of involutive divisions and an application to Hilbert function computations', *J. Symb. Comp.* **25**, 683–704.
- ARMANDO A., MELIS E. AND RANISE S. [1998], Constraint solving in logic programming and in automated deduction: a comparison, in 'Artificial Intelligence: Methodology, Systems, Applications, AIMS'98', Springer, LNCS 1480, pp. 28–38.
- ARMANDO A. AND RANISE S. [1998], From integrated reasoning specialists to "plug-and-play" reasoning components, in 'Artificial Intelligence and Symbolic Computation, AISC'98', Springer, LNCS 1476, pp. 42–54.
- ARORA S., BABAI L., STERN J. AND SWEEDYK Z. [1993], The hardness of approximate optima in lattices, codes, and systems of linear equations, in 'Foundations of Computer Science FOCS'93, Palo Alto'.
- AZULAY D.-O. AND PIQUE J.-F. [1998], Optimized Q-pivot for exact linear solvers, in 'Principles and Practice of Constraint Programming, CP'98, Pisa', Springer, LNCS 1520, pp. 55 – 71.
- BAADER F. AND SNYDER W. [2001], Unification theory, in A. Robinson and A. Voronkov, eds, 'Handbook of Automated Reasoning', Vol. I, Elsevier Science, chapter 8, pp. 445–532.
- BACHMAIR L. AND GANZINGER H. [2001], Resolution theorem proving, in A. Robinson and A. Voronkov, eds, 'Handbook of Automated Reasoning', Vol. I, Elsevier Science, chapter 2, pp. 19–99.

- BAKER A. [1968], 'Contributions to the theory of diophantine equations. I/II', *Philos. Trans. Roy. Soc. London, Ser. A* **263**, 173 – 208.
- BALAS E. [1979], 'Disjunctive programming', *Annals of Discrete Mathematics* **5**, 3 – 51.
- BALAS E. [1998], 'Disjunctive programming: Properties of the convex hull of feasible points', *Discrete Applied Mathematics* **89**(1-3), 3 – 44.
- BALAS E., CERIA S. AND CORNUÉJOLS G. [1993], 'A lift-and-project cutting plane algorithm for mixed 0-1 programs', *Mathematical Programming* **58**, 295 – 324.
- BALAS E., CERIA S. AND CORNUÉJOLS G. [1996], 'Mixed 0-1 programming by lift-and-project in a branch-and-cut framework', *Management Science* **42**(9), 1229 – 1246.
- BALAS E., CERIA S., CORNUÉJOLS G. AND NATRAJ N. R. [1996], 'Gomory cuts revisited', *Operations Research Letters* **19**.
- BALAS E. AND PERREGAARD M. [2000], A precise correspondence between lift-and-project cuts, simple disjunctive cuts, and mixed integer Gomory cuts for 0-1 programming, Management Science Research Report # MSRR-631, GSIA, Carnegie Mellon University.
- BAREISS E. H. [1968], 'Sylvester's identity and multistep integer-preserving Gaussian elimination', *Mathematics of Computation* **22**, 565 – 578.
- BARTH P. [1996], *Logic-based 0-1 constraint programming*, Operations Research/Computer Science Interfaces Series, Kluwer.
- BARTH P. AND BOCKMAYR A. [1995], Finite domain and cutting plane techniques in CLP($\mathcal{PB}$), in L. Sterling, ed., 'Logic Programming. 12th International Conference, ICLP'95, Kanagawa, Japan', MIT Press, pp. 133 – 147.
- BARTH P. AND BOCKMAYR A. [1998], 'Modelling discrete optimisation problems in constraint logic programming', *Annals of Operations Research* **81**, 467–496.
- BASU S., POLLACK R. AND ROY M.-F. [1998], A new algorithm to find a point in every cell defined by a family of polynomials, in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 341–350.
- BECKER E. AND WÖRMANN T. [1994], On the trace formula for quadratic forms, in W. B. Jacob, T.-Y. Lam and R. O. Robson, eds, 'Recent Advances in Real Algebraic Geometry and Quadratic Forms', Vol. 155 of *Contemporary Mathematics*, American Mathematical Society, Providence, Rhode Island, pp. 271–291. Proceedings of the RAGSQUAD Year, Berkeley, 1990–1991.
- BECKER T., WEISPFENNING V. AND KREDEL H. [1993], *Gröbner Bases, a Computational Approach to Commutative Algebra*, Vol. 141 of *Graduate Texts in Mathematics*, Springer, New York.
- BEN-OR M., KOZEN D. AND REIF J. [1986], 'The complexity of elementary algebra and geometry', *Journal of Computer and System Sciences* **32**, 251–264.
- BERMAN L. [1980], 'The complexity of logical theories', *Theoretical Computer Science* **11**, 71–77.
- BERTOLI P. G., CALMET J., GIUNCHIGLIA F. AND HOMANN K. [1998], Specification and integration of theorem provers and computer algebra systems, in 'Artificial Intelligence and Symbolic Computation, AISC'98', Springer, LNCS 1476, pp. 94–106.
- BERTSIMAS D. AND TSITSIKLIS J. N. [1997], *Introduction to linear optimization*, Athena Scientific, Belmont, Mass.
- BILU Y. AND HANROT G. [1996], 'Solving Thue equations of high degree', *J. Number Th.* **60**, 373–392.
- BILU Y. AND HANROT G. [1998], 'Solving superelliptic diophantine equations by Baker's method', *Compositio Math.* **112**, 273–312.
- BILU Y. AND HANROT G. [1999], 'Thue equations with composite fields', *Acta Arithmetica* **88**(4), 311–326.
- BJØRNER N. S., STICKEL M. E. AND URIBE T. E. [1997], A practical integration of first-order reasoning and decision procedures, in '14th International Conference on Automated Deduction, CADE-14, Townsville', Springer, LNCS 1249.

- BLAIR C. AND JEROSLOW R. G. [1978], 'A converse for disjunctive constraints', *Journal of Optimization Theory and Applications* **25**, 195 – 206.
- BLAND R. [1977], 'New finite pivoting rules for the simplex method', *Mathematics of Operations Research* **2**, 103 – 107.
- BLEDSE W. W. [1975], A new method for proving certain Presburger formulas, in 'Fourth Int. Joint Conf. on Artificial Intelligence, IJCAI'75, Tbilisi'.
- BLÖMER J. AND SEIFERT J.-P. [1999], On the complexity of computing short linearly independent vectors and short bases in a lattice, in 'ACM Symposium on Theory of computing, STOC'99, Atlanta, GA', pp. 711 – 720.
- BOCHNAK J., COSTE M. AND ROY M.-F. [1987], *Géométrie algébrique réelle*, Springer, Berlin, Heidelberg, New York.
- BOCKMAYR A. [1987], 'A note on a canonical theory with undecidable unification and matching problem', *Journal of Automated Reasoning* **3**, 379 – 381.
- BOCKMAYR A. [1993], Logic programming with pseudo-Boolean constraints, in F. Benhamou and A. Colmerauer, eds, 'Constraint Logic Programming. Selected Research', MIT Press, chapter 18, pp. 327 – 350.
- BOCKMAYR A. [1995], Solving pseudo-Boolean constraints, in 'Constraint Programming: Basics and Trends', Springer, LNCS 910, pp. 22 – 38.
- BOCKMAYR A. AND EISENBRAND F. [1999], Cutting planes and the elementary closure in fixed dimension, Research Report MPI-I-99-2-008, Max-Planck-Institut für Informatik, Im Stadtwald, D-66123 Saarbrücken, Germany. To appear in *Mathematics of Operations Research*.
- BOCKMAYR A., EISENBRAND F., HARTMANN M. AND SCHULZ A. S. [1999], 'On the Chvátal rank of polytopes in the 0/1 cube', *Discrete Applied Mathematics* **98**, 21 – 27.
- BOCKMAYR A. AND KASPER T. [1998], 'Branch-and-infer: A unifying framework for integer and finite domain constraint programming', *INFORMS J. Computing* **10**(3), 287 – 300.
- BORGWARDT K.-H. [1982], 'Some distribution-independent results about the asymptotic order of the average number of pivot steps of the simplex method', *Mathematics of Operations Research* **7**, 441 – 462.
- BOUDET A. AND COMON H. [1996], Diophantine equations, Presburger arithmetic and finite automata, in 'Trees in Algebra and Programming, CAAP'96, Linköping, Sweden', Springer, LNCS 1059, pp. 30 – 44.
- BOYER R. S. AND MOORE J. S. [1988], Integrating decision procedures into heuristic theorem provers: A case study of linear arithmetic, in J. E. Hayes, D. Michie and J. Richards, eds, 'Machine Intelligence 11: Logic and the acquisition of knowledge', Clarendon Press, chapter 5, pp. 83–124.
- BROWN C. [1998], Simplification of truth-invariant cylindrical algebraic decomposition, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC'98', ACM Press, pp. 295–301.
- BRUNS W., GUBELADZE J., HENK M., MARTIN A. AND WEISMANTEL R. [1999], 'A counterexample to an integer analogue of Carathéodory's theorem', *J. Reine Angew. Math.* **510**, 179–185.
- BÜNDGEN R. [1996], 'Buchberger's algorithm: the term rewriter's point of view', *Theor. Comp. Science* **159**(2), 143–190.
- BÜRCKERT H.-J. [1991], *A Resolution principle for a logic with restricted quantifiers*, Springer, LNCS 568.
- BÜRCKERT H.-J., HEROLD A., KAPUR D., SIEKMANN J. H., STICKEL M. E., TEPP M. AND ZHANG H. [1988], 'Opening the AC-unification race', *Journal of Automated Reasoning* **4**(4), 465–474.
- BURG J., STUCKEY P. J., TAI J. C. H. AND YAP R. H. C. [1995], Linear equation solving for constraint logic programming, in 'Logic Programming. 12th International Conference, ICLP'95, Kanagawa, Japan', MIT Press, pp. 33 – 47.
- BÜRGISSER P., CLAUSEN M. AND SHOKROLLAHI M. A. [1997], *Algebraic Complexity Theory*, Vol. 315 of *Grundlehren der Mathematischen Wissenschaften*, Springer.

- BURHENNE K.-D. [1990], Implementierung eines Algorithmus zur Quantorenelimination für lineare reelle Probleme, Diploma thesis, Universität Passau, D-94030 Passau, Germany.
- CANNY J. AND EMIRIS I. [1993], An efficient algorithm for the sparse resultant, in 'Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, AAEECC-93', Vol. 263 of *LNCS*, Springer, pp. 89–104.
- CANNY J. AND MANOCHA D. [1993], 'Multipolynomial resultant algorithms', *Journal of Symbolic Computation* **15**, 99–122.
- CAPRARA A., FISCHETTI M. AND LETCHFORD A. N. [2000], 'On the separation of maximally violated mod- k cuts', *Mathematical Programming* **87**, 37–56.
- ČERNIKOV R. [1963], 'Contraction of finite systems of linear inequalities', *Soviet Mathematics Doklady* **4**(5), 1520–1524.
- CHANDRU V. [1993], 'Variable elimination in linear constraints', *The Computer Journal* **36**(5), 463 – 472.
- CHANDRU V. AND HOOKER J. N. [1999], *Optimization Methods for Logical Inference*, Wiley.
- CHOU S.-C. [1988], *Mechanical Geometry Theorem Proving*, Mathematics and its applications, D. Reidel Publishing Company, Dordrecht, Boston, Lancaster, Tokyo.
- CHOU T.-W. AND COLLINS G. E. [1982], 'Algorithms for the solution of systems of linear diophantine equations', *SIAM J. Computing* **11**(4), 687 – 708.
- CHVÁTAL V. [1973], 'Edmonds polytopes and a hierarchy of combinatorial problems', *Discrete Mathematics* **4**, 305 – 337.
- CLAUSEN M. AND FORTENBACHER A. [1989], 'Efficient solution of linear diophantine equations', *Journal of Symbolic Computation* **8**, 201–216.
- COHEN E. AND MEGIDDO N. [1994], 'Improved algorithms for linear inequalities with two variables per inequality', *SIAM J. Computing* **23**(6), 1313–1347.
- COLLINS G. E. [1975], Quantifier elimination for the elementary theory of real closed fields by cylindrical algebraic decomposition, in H. Brakhage, ed., 'Automata Theory and Formal Languages. 2nd GI Conference', Vol. 33 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, Heidelberg, New York, pp. 134–183.
- COLLINS G. E. [1985], The SAC-2 computer algebra system, in B. F. Caviness, ed., 'EUROCAL '85; Proceedings of the European Conference on Computer Algebra', Vol. 104 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, New York, Tokyo, pp. 34–35.
- COLLINS G. E. [1998], Quantifier elimination by cylindrical algebraic decomposition - twenty years of progress, in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 8–23.
- COLLINS G. E. AND HONG H. [1991], 'Partial cylindrical algebraic decomposition for quantifier elimination', *Journal of Symbolic Computation* **12**(3), 299–328.
- COLMERAUER A. [1987], Introduction to PROLOG III, in '4th Annual ESPRIT Conference, Bruxelles', North Holland.
- COMON H. [1991], Disunification: A survey, in J. Lassez and G. Plotkin, eds, 'Computational Logic: Essays in Honor of A. Robinson', MIT Press, chapter 9, pp. 322–359.
- COMON H., DINCBAŞ M., JOUANNAUD J.-P. AND KIRCHNER C. [1999], 'A methodological view of constraint solving', *Constraints* **4**, 337 – 361.
- CONTEJEAN E. AND DEVIE H. [1994], 'An efficient incremental algorithm for solving systems of linear diophantine equations', *Information and Computation* **113**, 143 – 172.
- CONTI P. AND TRAVERSO C. [1991], Buchberger algorithm and integer programming, in 'Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, AAEECC-9', Springer, LNCS 539.
- COOK W., COULLARD C. R. AND TURÁN G. [1987], 'On the complexity of cutting plane proofs', *Discrete Applied Mathematics* **18**, 25 – 38.
- COOK W., GERARDS A. M. H., SCHRIJVER A. AND TARDOS E. [1986], 'Sensitivity theorems in integer linear programming', *Mathematical Programming* **34**, 251 – 264.

- COOK W., RUTHERFORD T., SCARF H. E. AND SHALLCROSS D. [1993], 'An implementation of the generalized basis reduction algorithm for integer programming', *ORSA Journal on Computing* 5(2), 206 – 212.
- COOPER D. [1972], 'Theorem-proving in arithmetic without multiplication', *Machine Intelligence* 7, 91–100.
- COPPERSMITH D. AND WINOGRAD S. [1990], 'Matrix multiplication via arithmetic progressions', *Journal of Symbolic Computation* 9(3), 251 – 280.
- COX D., LITTLE J. AND O'SHEA D. [1992], *Ideals, Varieties and Algorithms*, Undergraduate Texts in Mathematics, Springer-Verlag, New York, Berlin, Heidelberg.
- COX D., LITTLE J. AND O'SHEA D. [1998], *Using algebraic geometry*, Graduate Texts in Mathematics, Springer-Verlag, New York, Berlin, Heidelberg.
- DANTZIG G. [1963], *Linear Programming and Extensions*, Princeton Univ. Press, Princeton.
- DANTZIG G. AND EAVES B. [1973], 'Fourier-Motzkin elimination and its dual', *Journal of Combinatorial Theory, Series A* 14, 288–297.
- DAVENPORT J. H. AND HEINTZ J. [1988], 'Real quantifier elimination is doubly exponential', *Journal of Symbolic Computation* 5(1–2), 29–35.
- DAVIS M., MATIYASEVICH Y. AND ROBINSON J. [1976], 'Hilbert's tenth problem. Diophantine equations: Positive aspects of a negative solution', *Proc. Symp. Pure Math.* 28, 323–378.
- DAVIS M., PUTNAM H. AND ROBINSON J. [1961], 'The decision problem for exponential Diophantine equations', *Annals of Mathematics* 74(3).
- DEGTYAREV A. AND VORONKOV A. [2001], Equality reasoning in sequent-based calculi, in A. Robinson and A. Voronkov, eds, 'Handbook of Automated Reasoning', Vol. I, Elsevier Science, chapter 10, pp. 609–704.
- DIKIN I. [1967], 'Iterative solution of problems of linear and quadratic programming', *Soviet Math. Doklady* 8, 674–675.
- DINCBAŞ M., VAN HENTENRYCK P., SIMONIS H., AGGOUN A. AND GRAF T. [1988], The constraint logic programming language CHIP, in 'Fifth Generation Computer Systems, Tokyo, 1988', Springer.
- DINES L. [1919], 'Systems of linear inequalities', *Annals of Mathematics* 20, 191–199.
- DOLZMANN A. [1994], Reelle Quantorenelimination durch parametrisches Zählen von Nullstellen, Diploma thesis, Universität Passau, D-94030 Passau, Germany.
- DOLZMANN A., GLOOR O. AND STURM T. [1998], Approaches to parallel quantifier elimination, in O. Gloor, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC 98', ACM Press, Rostock, Germany, pp. 88–95.
- DOLZMANN A. AND STURM T. [1996], Redlog User manual, Technical Report MIP-9616, FMI, Universität Passau, D-94030 Passau, Germany. Edition 1.0 for Version 1.0.
- DOLZMANN A. AND STURM T. [1997], 'Redlog: Computer algebra meets computer logic', *ACM SIGSAM Bulletin* 31(2), 2–9.
- DOLZMANN A., STURM T. AND WEISPFENNING V. [1998a], 'A new approach for automatic theorem proving in real geometry', *Journal of Automated Reasoning* 21(3), 357–380.
- DOLZMANN A., STURM T. AND WEISPFENNING V. [1998b], Real quantifier elimination in practice, in B. H. Matzat, G.-M. Greuel and G. Hiss, eds, 'Algorithmic Algebra and Number Theory', Springer, Berlin, pp. 221–247.
- DOMENJOU E. [1991], Solving systems of linear diophantine equations: An algebraic approach, in 'Mathem. Foundations of Computer Science, Kazimierz Dolny, Poland', Springer, LNCS 520.
- DOMENJOU E. [1992], 'A technical note on AC-unification. the number of minimal unifiers of the equation $\alpha x_1 + \dots + \alpha x_p \doteq_{ac} \beta y_1 + \dots + y_q$ ', *Journal of Automated Reasoning* 8, 39–44.
- DOMENJOU E. AND TOMÁS A. P. [1995], From Elliott-MacMahon to an algorithm for general linear constraints in naturals, in 'Principles and Practice of Constraint Programming - CP'95, Cassis, France', Springer, LNCS 976, pp. 18 – 35.

- DORMICH P. D., KANNAN R. AND TROTTER L. E. [1987], 'Hermite normal form computation using modulo determinant arithmetic', *Mathematics of Operations Research* **12**(1), 50 – 59.
- DOWEK G., HARDIN T. AND KIRCHNER C. [1998], Theorem proving modulo, Rapport de recherche 3400, INRIA. To appear in *J. Autom. Reas.*
- DRIES V. D. L. AND HOLLY J. [1992], 'Quantifier elimination for modules with scalar variables', *Annals of Pure and Applied Logic* **57**, 161–179.
- DUFFIN R. J. [1974], 'On Fourier's analysis of linear inequality systems', *Mathematical Programming Study* **1**, 71–95.
- DURAND A., HERMANN M. AND JUBAN L. [1999], On the complexity of recognizing the Hilbert basis of a linear diophantine system, in '24th International Symposium on Mathematical Foundations of Computer Scienc, MFCS'99', Springer, LNCS 1672, pp. 92–102.
- EAVES B. AND ROTHBLUM U. [1992], 'Dines-Fourier-Motzkin quantifier elimination and an application of corresponding transfer principles over ordered fields', *Mathematical Programming* **53**, 307–321.
- EDMONDS J. [1967], 'Systems of distinct representatives and linear algebra', *J. Res. Natl. Bur. Stand., Sect. B* **71**, 241–245.
- EISENBRAND F. [1999], 'On the membership problem for the elementary closure of a polyhedron', *Combinatorica* **19**(2), 297–300.
- EISENBRAND F. [2000], Gomory-Chvátal cutting planes and the elementary closure of polyhedra, PhD thesis, Naturwissenschaftlich-Technische Fakultät I, Univ. des Saarlandes.
- EISENBRAND F. AND SCHULZ A. S. [1999], Bounds on the Chvátal rank of polytopes in the 0/1 cube, in 'Integer Programming and Combinatorial Optimization, IPCO 99', Springer, LNCS 1610, pp. 137–150.
- ELLIOTT E. B. [1903], 'On linear homogeneous diophantine equations', *Quarterly Journal of Pure and Applied Mathematics* **34**, 348 – 377.
- EMIRIS I. AND CANNY J. [1995], 'Efficient incremental algorithms for the sparse resultant and the mixed volume', *J. Symbolic Computation* **20**, 117–149.
- FANG X. G. AND HAVAS G. [1997], On the worst-case complexity of integer Gaussian elimination, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC'97, Maui, HI', ACM Press, pp. 28 – 31.
- FERRANTE J. AND RACKOFF C. W. [1979], *The Computational Complexity of Logical Theories*, number 718 in 'Lecture Notes in Mathematics', Springer-Verlag, Berlin.
- FILGUEIRAS M. AND TOMÁS A. P. [1991], Solving linear constraints on finite domains through parsing, in '5th Portuguese Conference on Artificial Intelligence, EPIA'91', Springer, LNCS 541, pp. 1–16.
- FILGUEIRAS M. AND TOMÁS A. P. [1993], Fast methods for solving linear diophantine equations, in '6th Portuguese Conference on AI, EPIA'93', Springer, LNCS 727, pp. 297 – 306.
- FISCHER M. J. AND RABIN M. [1974], 'Super-exponential complexity of Presburger arithmetic', *SIAM-AMS Proceedings* **7**, 27–41.
- FORTENBACHER A. [1993], Algebraische Unifikation (in German), Diplomarbeit, Univ. Karlsruhe.
- FOURIER J. [1826], 'Solution d'une question particulière du calcul des inégalités', *Nouveau Bulletin des Sciences par la Société Philomathique de Paris* pp. 99–100.
- FRANK A. AND TARDOS E. [1987], 'An application of simultaneous diophantine approximation in combinatorial optimization', *Combinatorica* **7**, 49–65.
- FRUMKIN M. A. [1976], 'An algorithm for the reduction of a matrix of integers to triangular form with power complexity of computations (in Russian)', *Ekonom. i mat. Metody* **12**, 173 – 178.
- FÜRER M. [1982], 'The complexity of Presburger arithmetic with bounded quantifier alternation depth', *Theoretical Computer Science* **18**, 105–111.
- GAO X. AND CHOU S. [1991], Computing with parametric equations, in S. Watt, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC'91', ACM Press, pp. 122–127.

- GATHEN V. Z. J. AND SIEVEKING M. [1976], Weitere zum Erfüllungsproblem polynomial äquivalente kombinatorische Aufgaben, in E. Specker and V. Strassen, eds, 'Komplexität von Entscheidungsproblemen', Vol. 43 of *LNCS*, Springer.
- GATHEN V. Z. J. AND SIEVEKING M. [1978], 'A bound on solutions of linear integer equalities and inequalities', *Proceedings AMS* **72**, 155–158.
- GEDDES K. O., CZAPOR S. R. AND LABAHN G. [1992], *Algorithms for computer algebra*, Kluwer.
- GERARDS A. M. H. [1990], On cutting planes and matrices, in W. Cook and P. D. Seymour, eds, 'Polyhedral Combinatorics', DIMACS, pp. 29–32.
- GÖDEL K. [1931], 'Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I', *Monatsh. Math. Phys.* **38**, 173–198.
- GOLDFARB W. [1981], 'The undecidability of the second-order unification problem', *Theoretical Computer Science* **13**, 225–230.
- GOMORY R. E. [1958], 'Outline of an algorithm for integer solutions to linear programs', *Bull. AMS* **64**, 275 – 278.
- GONZÁLEZ L., LOMBARDI H., RECIO T. AND ROY M.-F. [1989], Sturm-Habicht sequence, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC89', ACM Press, pp. 136–146.
- GONZÁLEZ L., LOMBARDI H., RECIO T. AND ROY M.-F. [1990], 'Sous-résultants et spécialisation de la suite de Sturm I', *Informatique théorique et applications* **24**(6), 561–588.
- GONZÁLEZ-VEGA [1998], A combinatorial algorithm solving some quantifier elimination problems, in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 365–375.
- GONZÁLEZ-VEGA L., RECIO T., LOMBARDI H. AND ROY M.-F. [1998], Sturm-Habicht sequences determinants and real roots of univariate polynomials, in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 300–316.
- GONZÁLEZ-VEGA L., ROY M.-F., ROULLIER F. AND TRUJILLO G. [1998], Symbolic recipes for real solutions, in A. Cohen, H. Cuypers and H. Sterk, eds, 'Some tapas of computer algebra', Springer, Berlin.
- GORDAN P. [1873], 'Über die Auflösung linearer Gleichungen mit reellen Coefficienten', *Mathematische Annalen* **6**, 23 – 28.
- GRÄBE H.-G. [1994], On factorized Gröbner bases, in 'Proceedings of the workshop: "Computer Algebra in Science and Engineering"', Bielefeld, Aug'94', World Scientific, Singapore, pp. 77–89.
- GRÄDEL E. [1987], The complexity of subclasses of logical theories, Doctoral dissertation, Basel.
- GRIGORIEV D. [1988], 'Complexity of deciding Tarski algebra', *Journal of Symbolic Computation* **5**(1), 65–108.
- GRÖTSCHHEL M., LOVÁSZ L. AND SCHRIJVER A. [1988], *Geometric algorithms and combinatorial optimization*, Vol. 2 of *Algorithms and Combinatorics*, Springer.
- GRUNEWALD F. J. AND SEGAL D. [1981], 'How to solve a quadratic equation in integers', *Math. Proc. Camb. Philos. Soc.* **89**, 1 – 5.
- GUCKENBIEHL T. AND HEROLD A. [1985], Solving linear diophantine equations, Technical Report Memo SEKI-85-IV-KL, Univ. Kaiserslautern.
- GURARI E. [1985], 'Decidable problems for powerful programs', *J. ACM* **32**, 466–483.
- GURARI E. AND IBARRA O. [1979], 'An NP-complete number-theoretic problem', *J. ACM* **26**, 567–581.
- GURARI E. AND IBARRA O. [1981], 'The complexity of the equivalence problem for two characterizations of Presburger sets', *Theoretical Computer Science* **13**, 295–314.
- HAFNER J. L. AND MCCURLEY K. S. [1991], 'Asymptotically fast triangularization of matrices over rings', *SIAM J. Computing* **20**(6), 1068 – 1083.
- HAKEN A. [1985], 'The intractability of resolution', *Theoretical Computer Science* **39**, 297 – 308.

- HAKEN A. AND COOK S. A. [1999], 'An exponential lower bound for the size of monotone real circuits', *Journal of Computer and System Sciences* **58**, 326 – 335.
- HANSEN P., JAUMARD B. AND MATHON V. [1993], 'Constrained nonlinear 0-1 programming', *ORSA Journal on Computing* **5**(2), 97 – 119.
- HAVAS G., HOLT D. F. AND REES S. [1993], 'Recognizing badly presented $\mathbb{Z}$ -modules', *Linear Algebra Appl.* **192**, 137 – 163.
- HAVAS G. AND MAJEWSKI B. S. [1994], 'Hermite normal form computation for integer matrices', *Congressus Numerantium* **105**, 87 – 96.
- HAVAS G. AND MAJEWSKI B. S. [1997], 'Integer matrix diagonalization', *Journal of Symbolic Computation* **24**, 399 – 408.
- HAVAS G., MAJEWSKI B. S. AND MATTHWES K. R. [1998], 'Extended GCD and Hermite normal form algorithms via lattice basis reduction', *Experimental Mathematics* **7**(2), 125 – 136.
- HEINTZ J., ROY M.-F. AND SOLERNÓ P. [1990], 'Sur la complexité du principe de Tarski-Seidenberg', *Bull. Soc. Math. France* **118**, 101–126.
- HENK M. [1997], 'Note on shortest and nearest lattice vectors', *Information Processing Letters* **61**, 183 – 188.
- HENK M. AND WEISMANTEL R. [1997], 'The height of minimal Hilbert bases', *Result. Math.* **32**(3-4), 298 – 303.
- HENK M. AND WEISMANTEL R. [2000], 'On minimal solutions of linear diophantine equations', *Contrib. Algebra and Geometry* **41**(1), 49–55.
- HERMANN M., JUBAN L. AND KOLAITIS P. G. [1999], 'On the complexity of counting the Hilbert basis of a linear diophantine system', in '6th International Conference on Logic for Programming and Automated Reasoning, LPAR'99', Springer, LNCS 1705, pp. 13–32.
- HERMITE C. [1851], 'Sur l'introduction de variables continues dans la théorie des nombres', *J. reine u. angewandte Mathematik* **41**, 191 – 216.
- HILBERT D. [1890], 'Über die Theorie der algebraischen Formen', *Mathematische Annalen* **36**, 473 – 534.
- HOCHBAUM D. AND NAOR J. [1994], 'Simple and fast algorithms for linear and integer programs with two variables per inequality', *SIAM J. Computing* **23**(6), 1179–1192.
- HOLZBAUR C. [1995], OEFAI clip(Q,R) manual, Edition 1.3.3, Technical Report TR-95-09, Austrian Research Institute for Artificial Intelligence, Vienna.
- HONG H. [1998], 'An improvement of the projection operator in cylindrical algebraic decomposition', in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 166–173.
- HOOKE J. N. [1988a], 'Generalized resolution and cutting planes', *Annals of Operations Research* **12**, 217 – 239.
- HOOKE J. N. [1988b], 'A quantitative approach to logical inference', *Decision Support Systems* **4**, 45 – 69.
- HOOKE J. N. [1992], 'Generalized resolution for 0-1 linear inequalities', *Annals of Mathematics and Artificial Intelligence* **6**, 271–286.
- HOOKE J. N. [2000], *Logic-based methods for optimization*, John Wiley.
- HUET G. [1972], 'Constrained Resolution: A Complete Method for Higher Order Logic', PhD thesis, Case Western Reserve University, USA.
- HUET G. [1978], 'An algorithm to generate the basis of solutions to homogeneous linear diophantine equations', *Information Processing Letters* **7**(3), 144 – 147.
- HUYNH T., JOSKOWICZ L., LASSEZ C. AND LASSEZ J. L. [1990], 'Reasoning about constraints using parametric queries', in K. Nori and C. Veni Madhavan, eds, 'Foundations of Software Technology and Theoretical Computer Science', number 472 in 'LNCS', Springer, pp. 1–20.
- HUYNH T., LASSEZ C. AND LASSEZ J.-L. [1992], 'Practical issues on the projection of polyhedral sets', *Annals of Mathematics and Artificial Intelligence* **6**, 295–316.
- HUYNH T. AND MARRIOTT K. [1995], 'Incremental constraint deletion in systems of linear constraints', *Information Processing Letters* **55**, 111 – 115.

- ILIOPOULOS C. S. [1989a], 'Worst-case complexity bounds on algorithms for computing the canonical structure of finite abelian groups and the Hermite and Smith normal forms of an integer matrix', *SIAM J. Computing* **18**(4), 658 – 669.
- ILIOPOULOS C. S. [1989b], 'Worst-case complexity bounds on algorithms for computing the canonical structure of infinite abelian groups and solving systems of linear diophantine equations', *SIAM J. Computing* **18**(4), 670–678.
- IMBERT J. L. [1993], 'Variable elimination for disequations in generalized linear constraint systems', *The Computer Journal* **36**(5), 473 – 484.
- IMBERT J.-L. [1995a], Fourier's elimination: which to choose, in 'Principles and Practice of Constraint Programming. The Newport Papers', MIT Press, pp. 245 – 268.
- IMBERT J.-L. [1995b], Linear constraint solving in CLP languages, in 'Constraint Programming: Basics and Trends', Springer, LNCS 910, pp. 108 – 127.
- IMBERT J.-L. [1997], Redundancy, variable elimination and linear disequations, in 'International Symposium on Logic Programming, ISLP'94, Ithaca', MIT Press, pp. 139 – 153.
- JAFFAR J. AND LASSEZ J.-L. [1987], Constraint logic programming, in 'Proc. 14th ACM Symp. Principles of Programming Languages', Munich.
- JAFFAR J. AND MAHER M. J. [1994], 'Constraint logic programming: A survey', *Journal of Logic Programming* **19/20**, 503 – 581.
- JAFFAR J., MAHER M. J., STUCKEY P. J. AND YAP R. H. C. [1993], 'Projecting $CLP(\mathcal{R})$ constraints', *New Generation Computing* **11**, 449–469.
- JAFFAR J., MICHAYLOV S., STUCKEY P. J. AND YAP R. H. C. [1992], 'The $CLP(\mathcal{R})$ language and system', *ACM Transactions on Programming Languages and Systems* **14**(3), 339–395.
- JANIČIĆ P., BUNDY A. AND GREEN I. [1999], A framework for flexible integration of a class of decision procedures into theorem provers, in 'Automated Deduction, CADE-16, Trento', Springer, LNAI 1632, pp. 127 – 141.
- JEROSLOW R. G. [1977], 'Cutting-plane theory: disjunctive methods', *Annals of Discrete Mathematics* **1**, 293 – 330.
- JOHNSON E. L., NEMHAUSER G. L. AND SAVELSBERGH M. W. P. [2000], 'Progress in linear programming-based algorithms for integer programming: An exposition', *INFORMS J. Computing* **12**, 2 – 23.
- JOUANNAUD J. P. AND KIRCHNER C. [1991], Solving equations in abstract algebras: A rule-based survey of unification, in J. Lassez and G. Plotkin, eds, 'Computational Logic: Essays in Honor of A. Robinson', MIT Press, chapter 8, pp. 257–321.
- KANNAN R. AND BACHEM A. [1979], 'Polynomial algorithms for computing the Smith and the Hermite normal forms of an integer matrix', *SIAM J. Computing* **8**(4), 499 – 507.
- KAPPERT M. [1994], Eliminationsverfahren zur linearen und quadratischen Optimierung, Master's thesis, Universität Passau, FMI, Universität Passau.
- KAPUR D. [1995a], An approach for solving systems of parametric polynomial equations, in Saraswat and V. Hentenryck, eds, 'Principles and Practice of Constraint Programming', MIT press.
- KAPUR D. [1995b], Comparison of various multivariate resultant formulations, in A. Levelt, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC'95', ACM Press, pp. 187–194.
- KAPUR D. [1998], Automated geometric reasoning: Dixon resultants, Gröbner bases, and characteristic sets, in D. Wang, ed., 'Automated Deduction in Geometry', Vol. 1360 of *LNAI*, Springer.
- KAPUR D. AND LAKSHMAN Y. N. [1992], Elimination methods: An introduction, in B. Donald, D. Kapur and J. Mundy, eds, 'Symbolic and Numerical Computation for Artificial Intelligence', Academic Press.
- KAPUR D., MUSSER D. R. AND NIE X. [1994], 'An overview of the Tecton proof system', *Theoretical Computer Science* **133**, 307–339.

- KAPUR D. AND NIE X. [1994], Reasoning about numbers in Tecton, Technical report, Dept. of Computer Science, State Univ. of New York, Albany.
- KAPUR D., SAXENA T. AND YANG L. [1994], Algebraic and geometric reasoning using Dixon resultants, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC'94', ACM Press, pp. 108–113.
- KARMAKAR N. [1984], A new polynomial-time algorithm for linear programming, in '16th ACM Symposium on the Theory of Computing, STOC'84, Washington D.C.', pp. 302 – 311.
- KASPER T. [1998], A unifying logical framework for integer linear programming and finite domain constraint programming, PhD thesis, Technische Fakultät, Univ. des Saarlandes.
- KÄUFL T. [1988], Reasoning about systems of linear inequalities, in '9th International Conference on Automated Deduction, CADE-9, Argonne, USA', Springer, LNCS 310, pp. 563–572.
- KHACHIVAN L. G. [1979], 'A polynomial algorithm in linear programming', *Soviet Math. Doklady* 20, 191 – 194.
- KIRCHNER C., KIRCHNER H. AND RUSINOWITCH M. [1990], 'Deduction with symbolic constraints', *Revue d'Intelligence Artificielle* 4(3), 9–52.
- KIRCHNER H. [1995], On the use of constraints in automated deduction, in 'Constraint Programming: Basics and Trends', Springer, LNCS 910, pp. 128 – 146.
- KLEE V. AND MINTY G. J. [1972], How good is the simplex algorithm?, in O. Shisha, ed., 'Inequalities III', Academic Press, pp. 159 – 175.
- KOUBARAKIS M. [1996], Tractable disjunctions of linear constraints, in 'Principles and Practice of Constraint Programming, CP'96, Cambridge, MA', Springer, LNCS 1118, pp. 297 – 307.
- KREDEL H. [1993], MAS modula-2 algebra system, interactive usage, Technical report, Universität Passau. Available by anonymous ftp from [alice.fmi.uni-passau.de](ftp://alice.fmi.uni-passau.de).
- KREDEL H. AND WEISPFENNING V. [1988], 'Computing dimension and independent sets for polynomial ideals', *Journal of Symbolic Computation* 6, 232–247.
- LAGARIAS J. C. [1985], 'The computational complexity of simultaneous diophantine approximation problems', *SIAM J. Computing* 14(1), 196 – 209.
- LAMBERT J.-L. [1987], 'Une borne pour les générateurs des solutions entières positives d'une équation diophantienne linéaire', *C. R. Acad. Sci., Paris, Ser. I* 305, 39 – 40.
- LANKFORD D. [1989], 'Non-negative integer basis algorithms for linear equations with integer coefficients', *Journal of Automated Reasoning* 5, 25 – 35.
- LASSEZ J.-L. [1990], Querying constraints, in 'Proc. ACM Symp. on Principles of Database Systems', ACM-Press, pp. 288–298.
- LASSEZ J.-L. AND MAHER M. [1992], 'On Fourier's algorithm for linear arithmetic constraints', *Journal of Automated Reasoning* 9, 373 – 379.
- LASSEZ J.-L. AND MCALOON K. [1992], 'A canonical form for generalized linear constraints', *Journal of Symbolic Computation* 13, 1–24.
- LAZARD D. [1991], 'A new method for solving algebraic systems of positive dimension', *Discrete Applied Mathematics* 33, 147–160.
- LAZARD D. [1992], 'Solving zero-dimensional algebraic systems', *Journal of Symbolic Computation* 13, 117–131.
- LE CHENADEC P. [1986], *Canonical forms in finitely presented algebras*, Research Notes in TCS, Pitman.
- LENGAUER C. [1993], Loop parallelization in the polytope model, in 'CONCUR'93', Vol. 715 of LNCS, Springer, pp. 398–416.
- LENSTRA A. K., LENSTRA H. W. AND LOVÁSZ L. [1982], 'Factoring polynomials with rational coefficients', *Math. Annalen* 261, 515 – 534.
- LENSTRA H. W. [1983], 'Integer programming with a fixed number of variables', *Mathematics of Operations Research* 8(4), 538 – 548.
- LIPPOLD F. [1993], Implementierung eines Verfahrens zum Zählen reeller Nullstellen multivariater Polynome, Diploma thesis, Universität Passau, D-94030 Passau, Germany.

- LIPSHITZ L. [1978], 'The diophantine problem for addition and divisibility', *Trans. AMS* **235**, 271–283.
- LOMBARDI H. [1991], Effective real Nullstellensatz and variants, in M. T. and C. Traverso, eds, 'Effective Methods in Algebraic Geometry, MEGA'90', Birkhauser, pp. 263–288.
- LOOS R. AND WEISPFENNING V. [1993], 'Applying linear quantifier elimination', *The Computer Journal* **36**(5), 450–462. Special issue on computational quantifier elimination.
- LOVÁSZ L. AND SCARF H. E. [1992], 'The generalized basis reduction algorithm', *Mathematics of Operations Research* **17**(3), 751 – 764.
- MACKWORTH A. K. [1977], 'Consistency in networks of relations', *Artificial Intelligence* **8**, 99 – 118.
- MACMAHON P. A. [1916], *Combinatory Analysis*, Vol. II, The University Press, Cambridge.
- MADLENER K. AND REINERT B. [1998], String rewriting and Gröbner bases - a general approach to monoid and group rings, in M. Bronstein, J. Grabmeier and V. Weispfenning, eds, 'Symbolic Rewriting Techniques', Birkhäuser, Basel.
- MAKANIN G. [1979], 'Systems of standard equations in words in an n-layer alphabet of unknowns', *Sib. Math. J.* **19**, 448 – 454.
- MANDERS K. L. AND ADLEMAN L. [1978], 'NP-complete decision problems for binary quadratics', *Journal of Computer and System Sciences* **16**, 168 – 184.
- MARRIOTT K. AND STUCKEY P. J. [1998], *Programming with constraints*, MIT Press.
- MATIYASEVICH Y. V. [1970], 'Enumerable sets are Diophantine', *Soviet Math. Doklady* **11**(2), 354–358.
- MATIYASEVICH Y. V. [1993], *Hilbert's Tenth Problem*, MIT Press.
- MAZUR B. [1994], 'Questions of decidability and undecidability in number theory', *Journal of Symbolic Logic* **59**(2), 353 – 371.
- MEGIDDO N. [1984], 'Linear programming in linear time when the dimension is fixed', *Journal of the Association for Computing Machinery* **31**, 182–196.
- MELENK H., MÖLLER H. AND NEUN W. [1989], 'Symbolic solution of large stationary chemical kinetics problems', *IMPACT Comput. Sci. Eng.* **1**, 138–167.
- MIGNOTTE M. [1992], *Mathematics for computer algebra*, Springer.
- MISHRA B. [1993], *Algorithmic Algebra*, Springer Verlag.
- MOTZKIN T. S. [1936], Beiträge zur Theorie der linearen Ungleichungen, Doctoral dissertation, Universität Zürich.
- NELSON G. AND OPPEN D. C. [1979], 'Simplification by cooperating decision procedures', *ACM Transactions on Programming Languages and Systems* **1**(2), 245–257.
- NEMHAUSER G. L. AND WOLSEY L. A. [1988], *Integer and Combinatorial Optimization*, John Wiley.
- NIEUWENHUIS R. AND RUBIO A. [2001], Paramodulation-based theorem proving, in A. Robinson and A. Voronkov, eds, 'Handbook of Automated Reasoning', Vol. I, Elsevier Science, chapter 7, pp. 371–443.
- OPPEN D. [1973], Elementary bounds for Presburger arithmetic, in 'Proceedings 5th ACM Symposium on Theory of Computing', ACM, pp. 34–37.
- PAN V. [1991], Complexity of algorithms for linear systems of equations, in E. Spedicato, ed., 'Computer algorithms for solving linear algebraic equations: the state of the art', Vol. F 77 of *NATO ASI series*, Springer.
- PASECHNIK D. V. [1998], On computing Hilbert bases via the Elliott-MacMahon algorithm. To appear in *Theoretical Computer Science*.
- PEDERSEN P., ROY M.-F. AND SZPIRGLAS A. [1993], Counting real zeroes in the multivariate case, in F. Eysette and A. Galigo, eds, 'Computational Algebraic Geometry', Vol. 109 of *Progress in Mathematics*, Birkhäuser, Boston, Basel; Berlin, pp. 203–224. Proceedings of the MEGA 92.
- PERESSINI A., SULLIVAN F. AND UHL J. [1988], *The Mathematics of Nonlinear Programming*, Undergraduate Texts in Mathematics, Springer.

- PESANT G. [1995], Une approche géométrique aux contraintes arithmétiques quadratiques en programmation logique avec contraintes, PhD thesis, Université de Montréal.
- PESANT G. AND BOYER M. [1994], QUAD-CLP(R): Adding the power of quadratic constraints, in 'Principles and Practice of Constraint Programming', Springer, LNCS 874, pp. 95–108.
- PESCH M. [1994], Computing Comprehensive Gröbner Bases using MAS. User Manual.
- PHEIDAS T. [1994], 'Extensions of Hilbert's tenth problem', *Journal of Symbolic Logic* **59**(2), 372 – 397.
- PLOTKIN G. [1972], Building-in equational theories, in B. Meltzer and D. Michie, eds, 'Machine Intelligence 7', Edinburgh Univ. Press, pp. 73–90.
- PORKOLAB L. AND KHACHIYAN L. [1997], Computing integral points in convex semi-algebraic sets, in 'Foundations of Computer Science, FOCS'97, Miami Beach', pp. 162–171.
- POTTIER L. [1991a], Minimal solutions of linear diophantine systems: bounds and algorithms, in 'Rewriting Techniques and Applications, RTA'91', Springer, LNCS 488.
- POTTIER L. [1991b], Sub-groups of $\mathbb{Z}^n$, standard basis, and linear diophantine systems, Technical Report RR-1520, INRIA, Sophia-Antipolis.
- PRESBURGER M. [1929], Über die Vollständigkeit eines gewissen Systems der Arithmetik, in 'Comptes rendus du 1er Congrès des Mathématiques des Pays Slaves', Vol. 395, Warsaw, pp. 92–101.
- PUDLÁK P. [1997], 'Lower bounds for resolution and cutting plane proofs and monotone computations', *Journal of Symbolic Logic* **62**(3), 981–988.
- PUDLÁK P. [1999], On the complexity of the propositional calculus, in 'Sets and Proofs, Invited papers from Logic Colloquium'97', Cambridge Univ. Press, pp. 197 – 218.
- REDDY C. AND LOVELAND D. [1978], Presburger arithmetic with bounded quantifier alternation, in 'Symposium on the Theory of Computing, STOC'78', ACM, pp. 320–325.
- RENEGAR J. [1991], Recent progress on the complexity of the decision problem for the reals, in 'Discrete and computational geometry, Proc. DIMACS Spec. Year Workshops 1989-90', Vol. 6 of DIMACS, Ser. Discret. Math. Theor. Comput. Sci., AMS, pp. 287–308.
- RENEGAR J. [1992a], 'On the computational complexity and geometry of the first-order theory of the reals. Part I: Introduction. Preliminaries. The geometry of semi-algebraic sets. The decision problem for the existential theory of the reals', *Journal of Symbolic Computation* **13**(3), 255–299.
- RENEGAR J. [1992b], 'On the computational complexity and geometry of the first-order theory of the reals. Part II: The general decision problem. Preliminaries for quantifier elimination', *Journal of Symbolic Computation* **13**(3), 301–327.
- RENEGAR J. [1992c], 'On the computational complexity and geometry of the first-order theory of the reals. Part III: Quantifier elimination', *Journal of Symbolic Computation* **13**(3), 329–352.
- ROMEUF J.-F. [1990], 'A polynomial algorithm for solving systems of two linear diophantine equations', *Theoretical Computer Science* **74**(3), 329–340.
- ROUILLIER F. [1996], Algorithmes efficaces pour l'étude des zéros réels des systèmes polynomiaux, PhD thesis, Université Rennes.
- ROY M.-F. [1996], Basic algorithms in real algebraic geometry: from Sturm theorem to the existential theory of real numbers, in 'Lectures on Real Geometry in memoriam of Mario Raimondo', Vol. 23 of *Expositions in Mathematics*, de Gruyter, pp. 1–67.
- SARASWAT V. A. [1993], *Concurrent constraint programming*, MIT Press.
- SCARPELLINI B. [1984], 'Complexity of subcases of Presburger arithmetic', *Trans AMS* pp. 203–218.
- SCHRIJVER A. [1980], 'On cutting planes', *Annals of Discrete Mathematics* **9**, 291 – 296.
- SCHRIJVER A. [1986], *Theory of Linear and Integer Programming*, John Wiley.
- SEBŐ A. [1990], Hilbert bases, Carathéodory's theorem and combinatorial optimization, in R. Kannan and W. R. Pulleyblank, eds, 'Integer Programming and Combinatorial Optimization, IPCO'90, Waterloo, ON, Canada', University of Waterloo Press.

- SEIDENBERG A. [1956a], 'An elimination theory for differential algebra', *Univ. California Publ. Math (N.S.)* **3**, 31–66.
- SEIDENBERG A. [1956b], 'Some remarks on Hilbert's Nullstellensatz', *Arch. Math.* **7**, 235–240.
- SHERALI H. D. AND SHETTY C. M. [1980], *Optimization with disjunctive constraints*, Vol. 181 of *Lecture Notes in Economics and Mathematical Systems*, Springer.
- SHOSTAK R. E. [1977], 'On the sup-inf method for proving Presburger formulas', *Journal of the Association for Computing Machinery* **24**(4), 529–543.
- SHOSTAK R. E. [1979], 'An efficient decision procedure for arithmetic with function symbols', *Journal of the Association for Computing Machinery* **26**(2), 351–360.
- SHOSTAK R. E. [1984], 'Deciding combinations of theories', *Journal of the Association for Computing Machinery* **31**(1), 1 – 12.
- SIEGEL C.-L. [1972], 'Zur Theorie der quadratischen Formen', *Nachr. Akad. Wiss. Goettingen, II. math.-phys. Kl.* pp. 21–46.
- SIEKMANN J. AND SZABÓ P. [1989], 'The undecidability of the D_A -unification problem', *Journal of Symbolic Logic* **54**(2), 402–414.
- SIT W. [1991], A theory for parametric linear systems, in S. Watt, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC'91', ACM Press, pp. 112–121.
- SMART N. P. [1998], *The Algorithmic Resolution of Diophantine Equations - A Computational Cookbook*, Cambridge Univ. Press.
- SMITH H. J. S. [1861], 'On systems of linear indeterminate equations and congruences', *Phil. Trans. Royal Soc. London (A)* **151**, 293 – 326.
- SONTAG E. D. [1985], 'Real addition and the polynomial hierarchy', *Information Processing Letters* **20**, 115 – 120.
- SRIVASTAVA D. [1993], 'Subsumption and indexing in constraint query languages with linear arithmetic constraints', *Annals of Mathematics and Artificial Intelligence* **8**, 315 – 343.
- STANLEY R. P. [1973], 'Linear homogeneous diophantine equations and magic labelings of graphs', *Duke Math. J.* **40**, 607 – 632.
- STANLEY R. P. [1986], *Enumerative combinatorics*, Vol. I, The Wadsworth & Brooks/Cole Mathematics Series.
- STICKEL M. E. [1985], 'Automated deduction by theory resolution', *Journal of Automated Reasoning* **1**, 333–356.
- STORJOHANN A. [1997], Nearly optimal algorithms for computing Smith normal forms of integer matrices, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC'96, Zurich', ACM Press, pp. 267 – 274.
- STORJOHANN A. AND LABAHN G. [1996], Asymptotically fast computation of Hermite normal forms of integer matrices, in 'International Symposium on Symbolic and Algebraic Computation, ISSAC'96, Zurich', ACM Press, pp. 259 – 266.
- STRASSEN V. [1969], 'Gaussian elimination is not optimal', *Numerische Mathematik* **13**, 354–356.
- STUCKEY P. J. [1991], 'Incremental linear constraint solving and detection of implicit equalities', *ORSA J. Comput.* **3**(4), 269 – 274.
- STURM T. [1997], Reasoning over networks by symbolic methods, Technical Report MIP-9719, FMI, Universität Passau, D-94030 Passau, Germany.
- STURM T. AND WEISPFENNING V. [1997], Rounding and blending of solids by a real elimination method, in A. Sydow, ed., 'Proceedings of the 15th IMACS World Congress on Scientific Computation, Modelling, and Applied Mathematics (IMACS 97)', Vol. 2, IMACS, Wissenschaft & Technik Verlag, Berlin, pp. 727–732.
- STURM T. AND WEISPFENNING V. [1998], Computational geometry problems in Redlog, in D. Wang, ed., 'Automated Deduction in Geometry', Vol. 1360 of *Lecture Notes in Artificial Intelligence*, Springer-Verlag, Berlin Heidelberg, pp. 58–86.
- STURMFELS B. [1993], *Algorithms in invariant theory*, Springer.
- SUZUKI N. AND JEFFERSON D. [1980], 'Verification decidability of Presburger array programs', *J. ACM* **27**, 191–205.

- TARDOS E. [1986], 'A strongly polynomial algorithm to solve combinatorial linear programs', *Operations Research* **34**, 250 – 256.
- TARSKI A. [1948], A decision method for elementary algebra and geometry, Technical report, University of California. 2nd ed., 1951.
- TAYLOR R. AND WILES A. [1995], 'Ring-theoretic properties of certain Hecke algebras', *Ann. Math., II. Ser.* **141**(3), 553–572.
- THOMAS R. [1995], 'A geometric Buchberger algorithm for integer programming', *Mathematics of Operations Research* **20**(4), 864 – 884.
- THOMAS R. [1998], Gröbner bases in integer programming, in D.-Z. Du and P. M. Pardalos, eds, 'Handbook of Combinatorial Optimization', Vol. 1, Kluwer, pp. 533–572.
- TOMÁS A. P. [1997], On solving linear diophantine constraints, PhD thesis, Univ. Porto.
- TOMÁS A. P. AND FILGUEIRAS M. [1997], Solving linear diophantine equations using the geometric structure of the solution space, in 'Rewriting Techniques and Applications, RTA-97, Sitges, Spain', Springer, LNCS 1232, pp. 269 – 283.
- TSANG E. [1993], *Foundations of Constraint Satisfaction*, Academic Press.
- TUNG S. P. [1987], 'Computational complexities of diophantine equations with parameters', *Journal of Algorithms* **8**, 324–336.
- TZANAKIS N. AND WEGER B. M. M. D. [1989], 'On the practical solution of the Thue equation', *J. Number Theory* **31**(2), 99–132.
- VAN DER WAERDEN B. L. [1940], *Moderne Algebra II*, 2nd edn, Springer, Berlin.
- VAN EMDE BOAS P. [1981], Another NP-complete partition problem and the complexity of computing short vectors in a lattice, Technical Report Report 81-04, Math. Inst., Univ. Amsterdam.
- VAN HENTENRYCK P. [1989], *Constraint satisfaction in logic programming*, MIT Press.
- VAN HENTENRYCK P. AND GRAF T. [1992], 'Standard forms for rational linear arithmetic in constraint logic programming', *Annals of Mathematics and Artificial Intelligence* **5**, 303–320.
- VAN HENTENRYCK P. AND SARASWAT V. [1996], 'Strategic directions in constraint programming', *ACM Computing Surveys* **28**(4), 701 – 726.
- VANDENBERGHE L. AND BOYD S. [1996], 'Semidefinite programming', *SIAM Review* **38**, 49 – 95.
- VANDERBEI R. J. [1997], *Linear programming : Foundations and extensions*, Kluwer.
- VAVASIS S. A. [1990], 'Quadratic programming is in NP', *Information Processing Letters* **36**(2), 73 – 77.
- VOLGER H. [1983], 'Turing machines with linear alternation, theories of bounded concatenation and the decision problem of first order theories', *Theoretical Computer Science* **23**, 333–337.
- VON ZUR GATHEN J. AND GERHARD J. [1999], *Modern computer algebra*, Cambridge University Press.
- WALLACE M. [1996], 'Practical applications of constraint programming', *Constraints* **1**, 139 – 168.
- WANG D. [1993a], An elimination method based on Seidenberg's theory and its applications, in F. Eyssette and A. Galligo, eds, 'Computational Algebraic Geometry (MEGA '92)', Birkhäuser Verlag, pp. 301–328.
- WANG D. [1993b], 'An elimination method for polynomial systems', *Journal of Symbolic Computation* **16**(2), 83–114.
- WANG D. [1995], Reasoning about geometric problems using an elimination method, in J. Pfalzgraf, ed., 'Automatic Practical Reasoning', Springer-Verlag, Wien, pp. 147–185.
- WAYNE K. D. [1999], A polynomial combinatorial algorithm for generalized minimum cost flow, in '31th ACM Symposium on the Theory of Computing, STOC'99, Atlanta,', pp. 11 –18.
- WEISPFENNING V. [1975], 'Two model theoretic proofs of Rückert's Nullstellensatz', *Transactions AMS* **203**, 331–342.
- WEISPFENNING V. [1988], 'The complexity of linear problems in fields', *Journal of Symbolic Computation* **5**(1–2), 3–27.

- WEISPFENNING V. [1990a], 'The complexity of almost linear diophantine problems', *Journal of Symbolic Computation* 10(5), 395–403.
- WEISPFENNING V. [1990b], 'Existential equivalence of ordered abelian groups with parameters', *Archive for Mathematical Logic* 29, 237–248.
- WEISPFENNING V. [1992], 'Comprehensive Gröbner bases', *Journal of Symbolic Computation* 14, 1–29.
- WEISPFENNING V. [1994], Parametric linear and quadratic optimization by elimination, Technical Report MIP-9404, FMI, Universität Passau, D-94030 Passau, Germany.
- WEISPFENNING V. [1997a], Complexity and uniformity of elimination in Presburger arithmetic, in W. W. Kuchlin, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC'97, Maui, Hawaii', ACM Press, New York, pp. 48–53.
- WEISPFENNING V. [1997b], 'Quantifier elimination for real algebra—the quadratic case and beyond', *Applicable Algebra in Engineering Communication and Computing* 8(2), 85–101.
- WEISPFENNING V. [1997c], 'Simulation and optimization by quantifier elimination', *Journal of Symbolic Computation* 24(2), 189–208. Special issue on applications of quantifier elimination.
- WEISPFENNING V. [1998], A new approach to quantifier elimination for real algebra, in B. Caviness and J. Johnson, eds, 'Quantifier Elimination and Cylindrical Algebraic Decomposition', Springer, Wien, New York, pp. 376–392.
- WEISPFENNING V. [1999], Mixed real-integer linear quantifier elimination, in S. Dooley, ed., 'International Symposium on Symbolic and Algebraic Computation, ISSAC'99', ACM Press, pp. 129–136.
- WEISPFENNING V. [2000], Deciding linear-transcendental problems, in V. Ganzha, E. Mayr and E. Vorozhtshov, eds, 'Computer Algebra in Scientific Computation - CASC 2000', Springer, pp. 423–438.
- WILES A. [1995], 'Modular elliptic curves and Fermat's last theorem', *Ann. Math., II. Ser.* 141(3), 443–551.
- WILLIAMS H. [1986], 'Fourier's method of linear programming and its dual', *American Mathematical Monthly* 93, 681–695.
- WOLSEY L. [1998], *Integer programming*, Wiley.
- YE Y. [1997], *Interior point algorithms : theory and analysis*, Wiley.
- ZHARKOV A. Y. AND BLINKOV Y. A. [1993], Involution approach to solving systems of algebraic equations, in 'Proc. IMACS'93', pp. 11–16.
- ZIEGLER G. M. [1999], Gröbner bases and integer programming, in A. M. Cohen, H. Cuypers and H. Sterk, eds, 'Some Tapas of Computer Algebra', Springer, pp. 168 – 183.

Index

A

affine
 combination 755
 hull 755
 subspace 755, 759
 affine scaling algorithm 768
 affinely independent 755
 algebraically closed 757
 algorithm
 affine scaling 768
 cutting plane 785
 Euclidean 779, 807
 univariate polynomials 803
 generalized basis reduction 784
 interior point 768
 lift-and-project 788
 LLL 781
 path following 769
 primal-dual 769
 potential reduction 769
 rectangles 795
 Simplex 764
 slopes 795
 arithmetic model 756
 ascending set 808
 atomic formula 757

B

barrier
 function 769
 optimization problem 769
 basic
 feasible
 index set 764
 solution 763
 solution 763
 basis
 Gröbner 810
 Hilbert 794
 involution 812
 lattice 781
 Lovász-Scarf reduced 784
 of a matrix 763
 reduced Gröbner 811
 binomial 793
 Bland's rule 765
 bound occurrence 757
 bound reasoning 790
 bounded quantifier 801
 branch-and-bound 791

branch-and-cut 792
 branch-and-infer 788
 branch-and-relax 791
 Buchberger reduction 809

C

canonical form
 generalized linear constraints 774
 Cauchy bounds 805
 center
 of an ellipsoid 766
 central path 770
 characteristic set 807
 extended 808
 Chvátal rank 787
 clausal inequality 782
 clause
 extended 799
 prime extended 800
 propositional 782
 closed formula 757
 combination
 affine 755
 conic 755
 convex 755
 linear 755
 complex and real quantifier elimination 816
 conic
 combination 755
 hull 755
 subspace 755
 constrained optimization 791
 constraint
 branching 790
 generalized linear 774
 lower bounding 791
 negative linear 774
 non-primitive 788
 numerical 754
 positive linear 774
 primitive 788
 constraint programming 788
 constraint satisfaction problem 790
 constraint store 789
 convex
 combination 755
 hull 755
 subspace 755
 coprime 761
 cutting plane 785, 792

algorithm	785
disjunctive	788
Gomory-Chvátal	786
proof	786
cylindrical algebraic decomposition ...	817

D

decision procedure	758
dependent	
affinely	755
linearly	755
dimension	
ideal	806
subspace	756
diophantine equation	819
superelliptic	821
Thue	821
diophantine set	820
disjunctive	
cutting plane	788
normal form	757
optimization	787
domain	
reasoning	790
reduction	789
dual	
linear program	763
duality theorem	763, 772

E

elementary	
integer column operations	780
elementary closure	787
elimination	
by substitution of test points	772
Fourier-Motzkin	771
Gauss-Jordan	761
Gaussian	759
ellipsoid	766
method	766
encoding length	
arithmetic model	756
Turing machine model	756
equation	
diophantine	819
linear	758
linear diophantine	779, 782, 794
polynomial	805
equivalent formula	757
Euclidean algorithm	779, 807
univariate polynomials	803
existential formula	757
extended clause	799

extreme ray	763
-------------------	-----

F

face	762
facet	762
Farkas Lemma	762, 772
fast matrix multiplication	760
feasible solution	763
first-order	
language	757
theory	758
formula	757
atomic	757
closed	757
equivalent	757
existential	757
prenex	757
quantifier-free	757
universal	757
Fourier-Motzkin elimination	771
free occurrence	757
full	
column rank	756
row rank	756
full-dimensional	756

G

Gauss-Jordan elimination	761
Gaussian elimination	759
generalized	
basis reduction	784
linear constraints	774
canonical form	774
resolution	799
generic system	815
Gomory-Chvátal cutting plane	786
Gröbner basis	807, 810
comprehensive	813, 817
reduced	811
Gröbner system	814

H

half space	762
height of Hilbert basis	798
Hermite normal form	780
Hilbert basis	794
height	798
Hilbert's Nullstellensatz	806
Hilbert's Tenth Problem	820
homogeneous polynomial	757
hull	
affine	755
conic	755
convex	755

integer	787
linear	755
hyperbolic programming	818
hyperplane	759

I

ideal	
toric	793
ideal membership problem	806
independence	
of negative constraints	774
inequality	
clausal	782
linear	761
valid	762
integer	
hull	787
linear optimization	785
linear programming	785, 788
interior point algorithm	768
isolating intervals	805

K

Karush-Kuhn-Tucker conditions	770
-------------------------------------	-----

L

lattice	781
basis	781
reduced	781
lift-and-project method	788
linear	
combination	755
diophantine equation ..	779, 782, 794
equation	758
function	754
hull	755
inequality	761
optimization	763
program	
dual	763
primal	763
programming	763
parametric	775
relaxation	785
quantifier elimination	776
subspace	755
theory of real numbers	776
linearly independent	755
LLL algorithm	781
local consistency	789
Lovász-Scarf reduced basis	784

M

matrix	
--------	--

fast multiplication	760
positive definite	766
minimal	
solutions	794
minimum basis problem	782
monomial	793
multivariate polynomial ring	756

N

nearest lattice vector problem	781
negative linear constraint	774
non-decomposable solution	794
non-primitive constraint	788
normal form	
disjunctive	757
Hermite	780
Smith	781
Nullstellensatz	805
Hilbert	806
numerical	
constraint	754
domain	754
function	754

O

occurrence	
bound	757
free	757
optimal solution	763
optimization	791
disjunctive	787
integer linear	785
linear	763
problem	
barrier	769
semidefinite	819
order	
term	809

P

parameter	757
parametric	
linear programming	775
strongly	776
weakly	776
partial cylindrical algebraic	
decomposition	817
path	
central	770
path following algorithm	769
pigeon-hole problem	798
pivot element	760
pivoting rules	765
pointed	762

polyhedron	762
pointed	762
polynomial	754
binomial	793
constrained	814
equation	805
homogeneous	757
monomial	793
S-	810
polynomial ring	
multivariate	756
univariate	756
polytope	762
positive definite matrix	766
positive linear constraint	774
potential reduction algorithm	769
precise negative constraints	775
prefix	757
prenex formula	757
Presburger arithmetic	800
uniform	801
primal	
linear program	763
primal-dual path following algorithm	769
prime extended clause	800
primitive constraint	788
problem	
constrained optimization	791
constraint satisfaction	790
programming	
constraint	788
hyperbolic	818
integer linear	785, 788
linear	763
propagation	789
propositional clauses	782

Q

quantifier	
bounded	801
quantifier elimination	758
complex and real	816
linear	776
quantifier-free formula	757

R

Rabinovich trick	806
rank	756
ray	763
extreme	763
real-closed	757
reasoning	
bound	790

domain	790
rectangles algorithm	795
reduced lattice base	781
reduction	
Buchberger	809
Ritt	807
relaxation	789
linear programming	785
tight	792
remainder	803
resolution	798
generalized	799
resultant	815
u-	815
Dixon	816
mixed sparse	816
sparse	815
Sylvester	815
system	815
Ritt reduction	807

S

S-polynomial	810
semidefinite optimization	819
sensitivity analysis	775
set	
ascending	808
characteristic	807
diophantine	820
shortest lattice vector problem	781
Simplex algorithm	764
simplifying resolvents	800
slack variable	764
sliding objective function	772
slopes algorithm	795
Smith normal form	781
solution	
basic feasible	763
feasible	763
minimal	794
non-decomposable	794
optimal	763
square-free part	803
standard form	
linear optimization problem	764
strongly	
parametric	776
polynomial	756
Sturm-Sylvester	
sequence	803
theorem	
multivariate	812
subspace	

affine755, 759
conic755
convex755
linear755
superelliptic diophantine equation821
Sylvester resultant815

T

term757
term order809
 degree-lex809
 degree-reverse-lex809
 inverse lex809
 lexicographic809
theory
 first-order758
 linear of real numbers776
Thue equation821
tight relaxation792
toric ideal793

U

Uniform Presburger Arithmetic801
univariate polynomial ring756
universal formula757

V

valid inequality762
variety757, 802
vertex762
virtual substitution of test points774
volume
 of an ellipsoid766

W

weakly parametric776

Z

zero756
 non-trivial757